

Contents Page

<i>Contents Page</i>	1
PROJECT ANALYSIS	3
Description of the Problem	3
Project Scope	3
Project Boundaries	4
Project Constraints	5
UML Use Case Diagram	6
Requirements Specification	7
End User Requirements	7
Functional Requirements	7
Project Plan	11
Resources Required	11
Gantt Chart	12
PROJECT DESIGN	13
Rough Initial Concept Sketch	13
User Interface Design	13
Main Menu UI Design	14
Login Menu UI Design	15
Sign-up Menu UI Design	16
Playing Game UI Design	17
Leaderboard Menu UI Design	20
Printed Leaderboard UI Design	20
TLA (Top Level Algorithm)	21
Refined TLA Design	22
Further Refined TLA Code Design	25
Integration Design	25
Queries	25
Data Dictionary	29
Advanced Higher Concepts Designs	30
Array Of Records [TLA 1.1]	30
Bubble Sort [TLA 21.6.2]	30

2D Array [TLA 17.1]	31
Project Implementation	32
Ongoing Testing, New Skills, & Research	32
Completed Program Code	69
Program User Interface	83
Main Menu	83
Sign-up Menu	83
Log-in Menu	83
Connect 4 Board	84
Leaderboard Menu	85
Database Entries	87
Project Testing	88
Test Plan	88
Persona 1 (aka Player 1 & Score Checker)	88
Persona 2 (aka Player 2)	88
Testing: Main Menu	91
Testing: Leaderboard Submenu	100
Testing: Signup Menu	107
Testing: Log-in Menu	113
Testing: Connect 4 Board	120
PROJECT EVALUATION	147
Fitness For Purpose Evaluation	147
Maintainability Evaluation	150
Commentary & Whitespace	150
Modularity	151
Variable Names	154
Robustness Evaluation	155

PROJECT ANALYSIS

Description of the Problem

Project Idea: Connect 4 Game (SDD Project w/ Database Integration)

I will be developing a 2-player recreation of the well-known board game Connect 4 using the languages Python & SQL.

1	2	3	4	5	6	7
Yellow				Yellow	Yellow	
Red			Yellow	Red	Red	
Red	Red	Yellow	Red	Yellow	Red	
Red	Yellow	Red	Yellow	Yellow	Yellow	Red

Diagram of an example game

Connect 4 is a game played in a 7x6 grid where you insert your colour of counter, turn by turn, into one of 7 slots on top of the board where it will then fall to the bottom. You win the game by being the first to get 4 of your colour counters in a line (vertically, horizontally, or diagonally).

The game will automatically detect wins/losses/draws, keep track of all signed up players and all their wins/losses/draws stats (the user account information will be stored in a database which I will interface with). I intend to include a text-based menu with login, logout, signup, and view leaderboard facilities (which can be sorted by wins/losses/draws). For playing the game and displaying the game board itself, an interactable graphically displayed (non-text based) Connect 4 grid window will be used.

The end-users of my game will predominantly S6s peers in my Advanced Computing class who want to play Connect 4 for entertainment - all of which are adept with technology and familiar with the rules of the game.

The Adv H Computing project requirements will be met for my SSD integrated with DDD project as:

1. The project will have an easy-to-use user interface that will validate all inputs
2. The project will be primarily programmed in Python but will integrate with a database containing all the user account information via SQL
3. The project will use a 2D array to store the “Connect 4” grid
4. The project will use an array of records to store information about the currently logged in players
5. The project will use a bubble sort algorithm to order by wins on the leaderboard menu, and for simplicity, losses and draws will be sorted within the SQL statement itself

Project Scope

All the different sections of the project will be detailed on the contents page with more specific information, however here is the overview of what will be handed over:

1. Analysis Section
 - a. Description of the Problem (Scope, Boundaries, Constraints)
 - b. UML Use Case Diagram
 - c. Requirements Specification (Functional and End-user Requirements)
 - d. Project Plan (Resources Required & Gantt Chart)
2. Design Section

- a. Rough Initial Concept Sketch
 - b. UI Design
 - c. Top Level Algorithm
 - d. Refined Top Level Algorithm
 - e. Further Refined Top Level Algorithm (including integration & Adv H concepts)
3. Implementation Section:
 - a. Ongoing Testing, Skills & Research
 - b. Complete Python Program
 - c. Program User Interface
4. Test Plan & Test Results
5. Evaluation Report
 - a. Fitness for Purpose
 - b. Maintainability
 - c. Robustness

Project Boundaries

1. Player usernames, passwords, wins, losses, and draws will be stored in the database

Text Based Systems

2. Main menu system for navigating between other menus
 - a. This (and all menus) will be navigated to via entering the number of the option you want to select
3. Sign-up system for allowing users to sign up with new username and password
 - a. The username and password must be more than 6 characters, less than 16 characters, and the username must not already exist
 - b. These details will not be encrypted when stored in the database
 - c. There is no limit on how many players can be signed up
4. Login system for allowing 2 users with their unique username and passwords to individually be signed-in at once
 - a. There is no limit to how many attempts you get at entering the password
5. Logout system for logging out each player individually
6. Leaderboard system for viewing the leaderboard sorted by number of wins, losses, or draws per player
 - a. These will be sorted in descending order of score (biggest to smallest)
 - b. The leaderboard will be printed in the console in a formatted text-based table

Graphical Systems

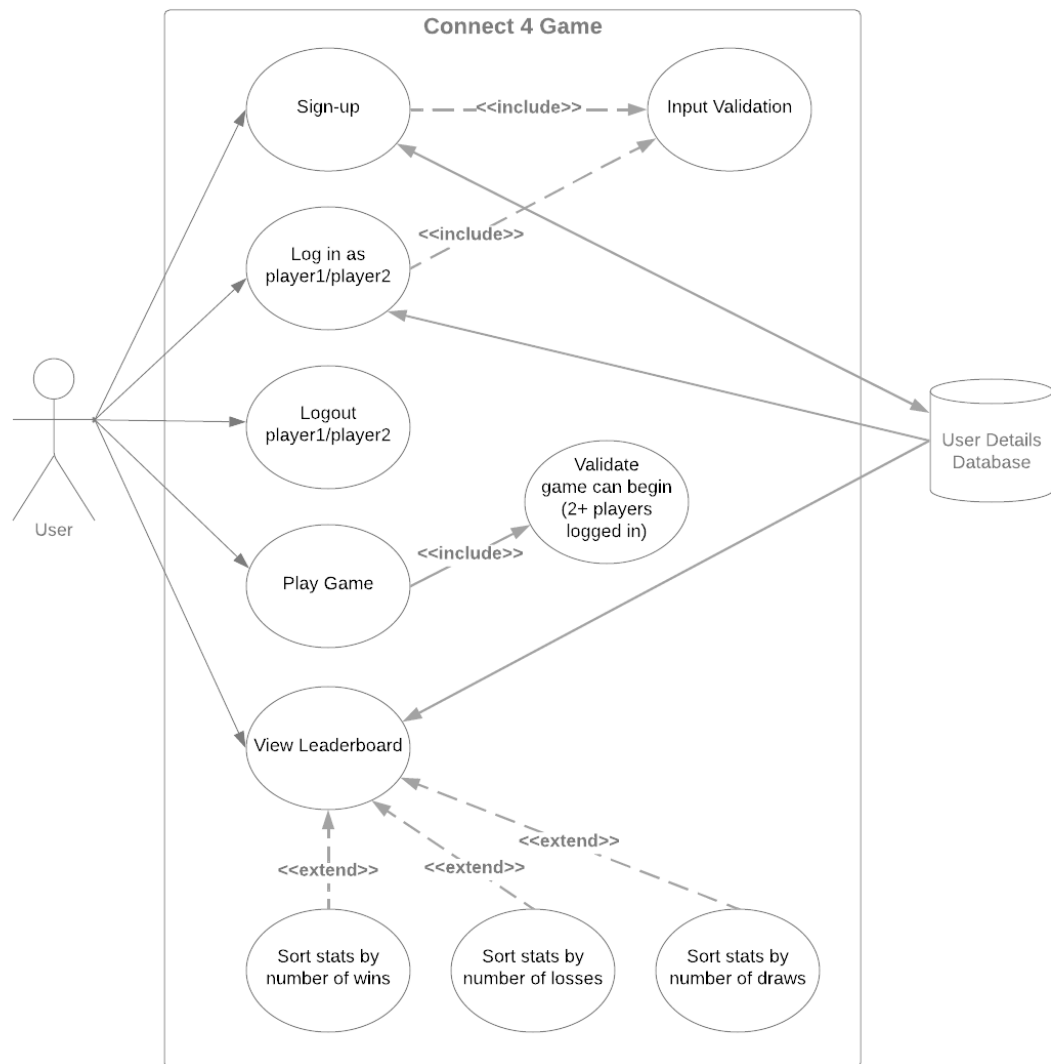
7. When a game has started, the newly opened Connect 4 board window's size will be set by the program
8. 1v1 Connect 4 game system for allowing 2 players to compete in games of Connect 4, turn by turn, until they draw or win
 - a. No bots will be included for solo play – you must have 2 players signed-in to play a game
 - b. You will interact with the board via mouse clicking on the graphically displayed game board
 - c. The game will validate that you click on a valid column for your counter to be dropped in

- d. The program will detect draws and wins

Project Constraints

Technical Constraints	Hardware	The final product will exclusively run on Windows 10 devices – as these are the only operating systems I have access to at home and in school. My computer specs, for reference, are: CPU: i7 7700k GPU: GTX 1060 6GB Memory: 16Gb DDR4 RAM Note: These specs are VERY MUCH overboard for this program which is extremely lightweight.
	Software	I will be using Python v3.10.1 to create the game as it is the language I am most agile in. The following will need installed on the user's computer prior to launching program: 1. Python v3.10.1 2. Python 'pygame' library v2.1.2 3. Python 'mysql-connector' library v2.2.9 4. MySQL v8.0.27
Business Constraints	Time	All aspects of the project must be completed by 5th April 2022 as that is the deadline set to us by our Advanced Computing teacher. The time set for the more specific aspects/sections of the project are detailed in the Gantt chart below.
	Economic	There are no costs for the project as all software, websites, and resources used are available at no cost online (freeware).
	Legal	There are no legal constraints for the project as this project is only intended for the purposes of evidence for the SQA and not for commercial purposes so does not need to meet GDPR required standards.
	Political	There are no political considerations for my project.

UML Use Case Diagram



Requirements Specification

For all the requirements to be met for my project, the following criteria must be met:

End User Requirements

Using the results of my user survey – here are the 5 most popular end-user requirements which I will make sure to meet in development of this project:

1. Users must be able to easily navigate the menus
2. Users need their accounts to be password protected
3. Users must drop a counter by clicking the wanted column – not typing in the number corresponding to the column
4. Users must be able to choose between the leaderboard being shown sorted by wins, losses, or draws
5. Users must be able to continue playing multiple games without needing to restart the program

Functional Requirements

These requirements have been broken down by each main process of the program. I have also categorised the end-user requirements listed above by which main processes they are relevant to.

Main Menu		
Python Requirements		
Inputs	Processes	Outputs
Validated user menu navigation inputs	Validation of user inputs Open connection with database (on program start)	Text-based menus & submenus displayed based on input: - Main Menu - Sign-up Menu - Log-in Menu - Leaderboard Menu Display who is logged in on main-menu Number of users logged in on main-menu Validation error messages Quitting game
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	X	X
End User Requirements		
Users must be able to easily navigate the menus		

Log-in Submenu		
Python Requirements		
Inputs	Processes	Outputs
Validated inputted usernames and passwords which already exist in database Existing user details SQL results	Validation of user inputs (via checking entered details exist in database)	Returning to main menu Log-in success message Validation error messages
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	X	Existing user details SQL results
End User Requirements		
Users must be able to easily navigate the menus Users needs their accounts to be password protected		

Sign-up Submenu		
Python Requirements		
Inputs	Processes	Outputs
Validated inputted usernames (6-16 characters) Validated inputted passwords (6-16 characters) Existing user details SQL results	Validation of user inputs (6-16 characters and ensuring user doesn't already exist using SQL results)	Returning to main menu Sign-up success message Validation error messages
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
Inserting new account details (username and password)	X	Existing user details SQL results
End User Requirements		
Users must be able to easily navigate the menus Users needs their accounts to be password protected		

Connect 4 Board		
Python Requirements		
Inputs	Processes	Outputs
Left clicking window	Check 2 players are logged in before game starts Validation of user inputs (left clicking) on the Connect 4 Board including: - Dropping counter where valid - Exiting to menu after game-ended screen Automatically detect wins and draws on Connect 4 board Randomise which players drops first counter	Successful starting game message Launching graphically displayed Connect 4 board Update following on the board window: - Highlighting columns when hovered - Displaying dropped counters dropped by player - Post-game win/draw screen Exiting back to main menu
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	Update player statistics based on how the game ends (editing wins, losses, and draws for both logged in players)	X
End User Requirements		
1. Users must drop a counter by clicking the wanted column – not typing in the number corresponding to the column 2. Users must be able to continue playing multiple games without needing to restart the program		

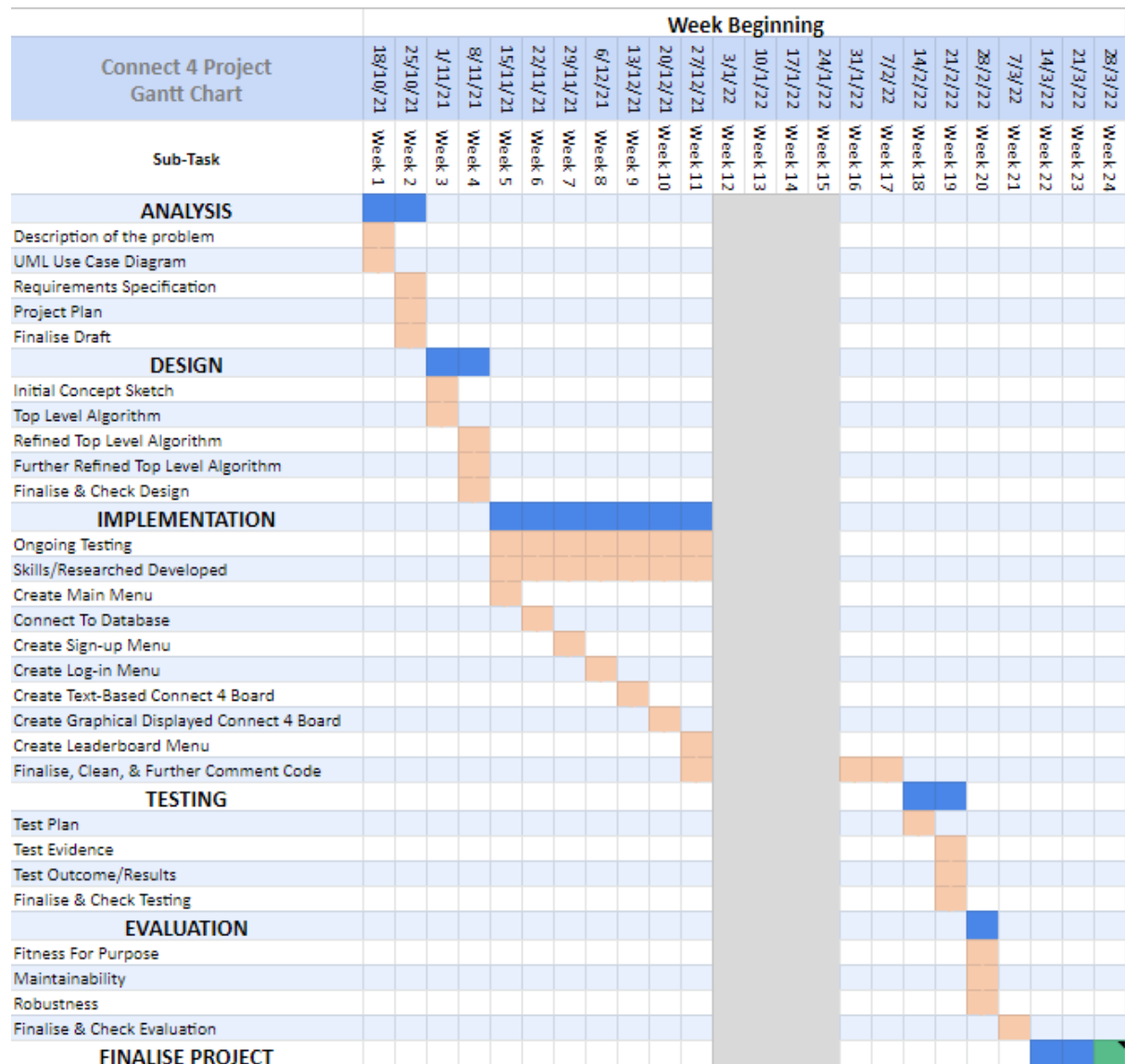
Leaderboard Submenu		
Python Requirements		
Inputs	Processes	Outputs
Validated user menu navigation inputs Leaderboard query results	Validation of user inputs Sort "win" sql results	Sorted leaderboard table based on input Return to main menu Validation error messages
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	Sort "losses" and "draws" sql results	Leaderboard query results
End User Requirements		
Users must be able to easily navigate the menus Users must be able to choose between the leaderboard being shown sorted by wins, losses, or draws		

Project Plan

Resources Required

Analysis	Google Docs Google Sheets (Gantt Chart) Google Chrome Lucidchart Website (https://www.lucidchart.com/)
Design	Google Docs Microsoft Publisher Paper & Pens Google Chrome
Implementation	Python v3.10.1 'pygame' Library v2.1.2 'mysql-connector' Library v2.2.9 Atom MySQL v8.0.27 MySQL Workbench v8.0 Google Chrome Snip & Sketch Google Docs Notepad++
Final Testing	Google Docs Atom Snip & Sketch Google Chrome
Evaluation	Google Docs Google Chrome

Gantt Chart

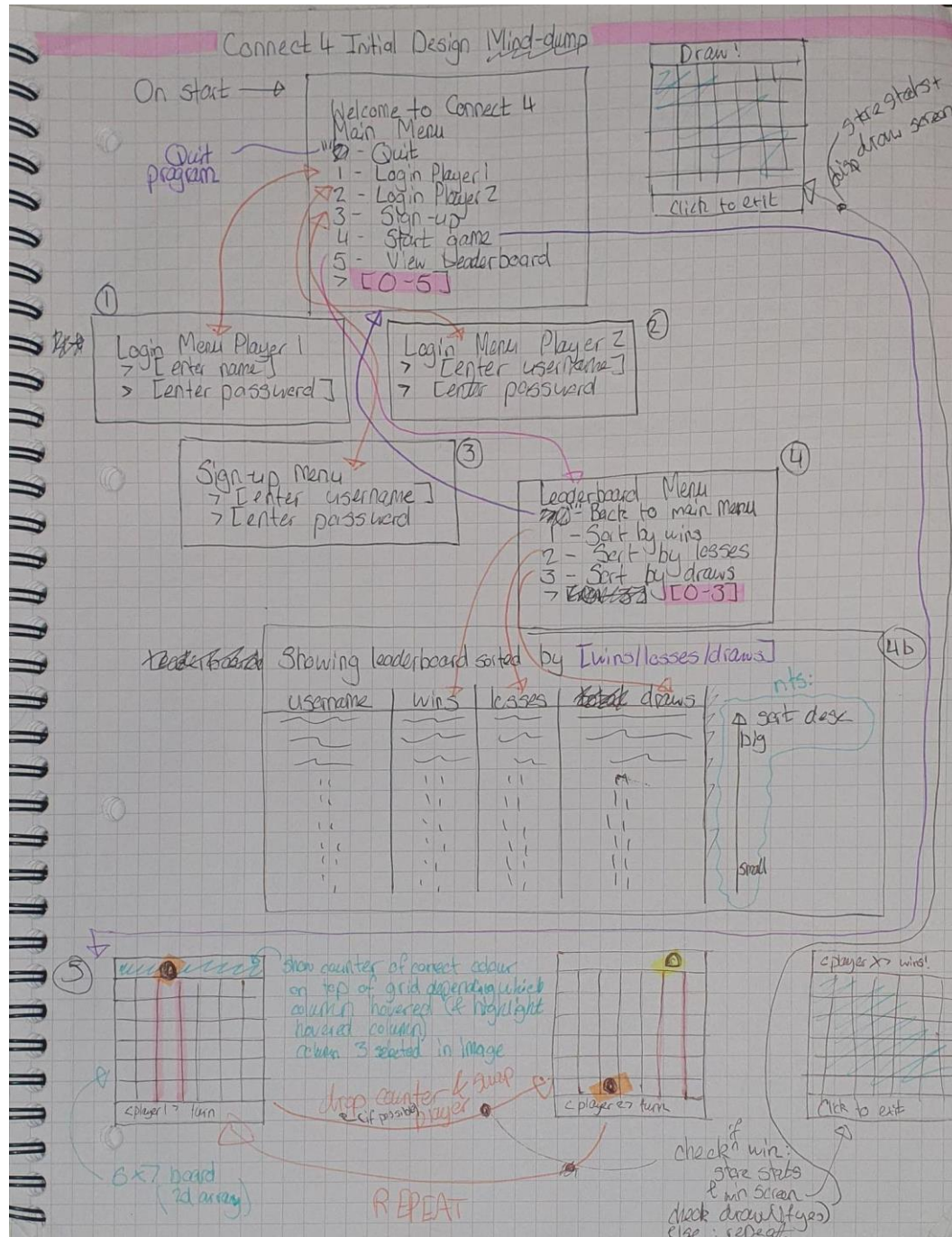


Note: Weeks 12-15 is prelim time so I've decided to keep them free so I can focus on coursework for my other subjects. This will give me a chance to revisit the project thus far with a fresh set of eyes before testing and evaluation.

PROJECT DESIGN

Rough Initial Concept Sketch

This is a rough sketch created to help better image what the final product would look and behave like. Nothing here is finalised. This is to splat the basic vision of the project onto the page so there is a clearer vision of the design when the rest of the design is created.



User Interface Design

After these initial sketches, it felt most natural to move onto creating more detailed wireframes of the user interface. In doing this, all end-user and functional requirements have been carefully considered when developing the user interface. **Any inputs/processes/outputs that occur to me at this stage will also be detailed to a simplified level.**

Although explained below, the UML Use Case diagram within the Analysis section will help to clarify how these menus connect.

Text within the boxes are output to the console.

Main Menu UI Design

Main Menu ←(5)

Player 1: Username/NOT LOGGED IN ←(1)

Player 2: Username/NOT LOGGED IN ←(1)

[1] Login Player 1/Logout USERNAME ←(2)

[2] Login Player 2/Logout USERNAME ←(2)

[3] Sign-up

[4] Play Game [number of logged in players/2 players logged in] ←(3)

[5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu

> [validated user input from 1-5 or blank input] ←(4)

(1) This will either display “NOT LOGGED IN” (in red) or will display the username of the player logged in as Player 1 or Player 2 (in green)

(2) Which message gets displayed (Login [...] / Logout [...]) depends on if a player is logged in. If player 1 is logged in, the logout option is shown. If player 1 is not logged in, the login option is shown. Same goes for Player 2.

(3) The “number of logged in players” message, as suggested, will just show 0, 1, or 2 based on how many players are logged in. The rest of the message on that line, “[” and “/2 players logged in]”, will also match these colours.

(4) As suggested the program will accept only valid options from the user. If an input is invalid the user will be asked to retry. Valid options are: 1, 2, 3, 4, 5, “” ←empty input.

The function of each input is detailed below:

1/2 → Go to Login Menu or logout appropriate player

3 → Go to Sign-up Menu

4 → Start Game (only if 2+ players are logged in. Otherwise, ask the user to log-in first.)

5 → Go to Leaderboard Menu

“” → Quit program

(5) This will be displayed via ASCII art to make text bigger.

Login Menu UI Design

- Player # Login - ←(3)

Leave blank input to exit menu.

Enter Username > [get valid username] ←(1)

Enter Password > [get validated corresponding password] ←(2)

Login successful! Press enter to return to main menu.

(1) & (2) Both inputs will be validated by checking inputted credentials against the database, credentials which will be received from the database with an SQL query. If a username is entered who is already logged in, it will also be invalid.

If a blank input is left for either username or password, you will instantly return to main menu cancelling the login. The login success message will only be displayed after a successful login.

No length checks are done here when receiving input - instead they are directly compared with the credentials from the database.

The inputted password will have to match the corresponding inputted username to successfully login.

(3) Number (#) changes based on who you are logging in as (Player 1/Player 2). So "1" or "2".

Sign-up Menu UI Design

- Sign-up Menu -

Leave blank input to exit menu.

Enter Username (6-16 characters) > [get valid username of 6-16 characters] ←(1)

Enter Password (6-16 characters) > [get valid password of 6-16 characters] ←(2)

Sign-up successful! Press enter to return to main menu. ←(3)

(1) & (2) Both inputs will be length checked before any further validation (6-16 characters). If an invalid input is given by the user they will be asked again after an error.

If a blank input is left for either username or password, you will instantly return to main menu cancelling the sign-up. The sign-up success message will only be displayed after a successful login.

(1) When validating the username, along with the length check, the program must ensure the entered username doesn't already exist in the database - so an SQL query will be necessary to check the username doesn't already exist in the database (usernames must be unique as they will be the primary key of the database)

(3) When the sign-up is successful, an insert SQL query will run to add the new account to the database.

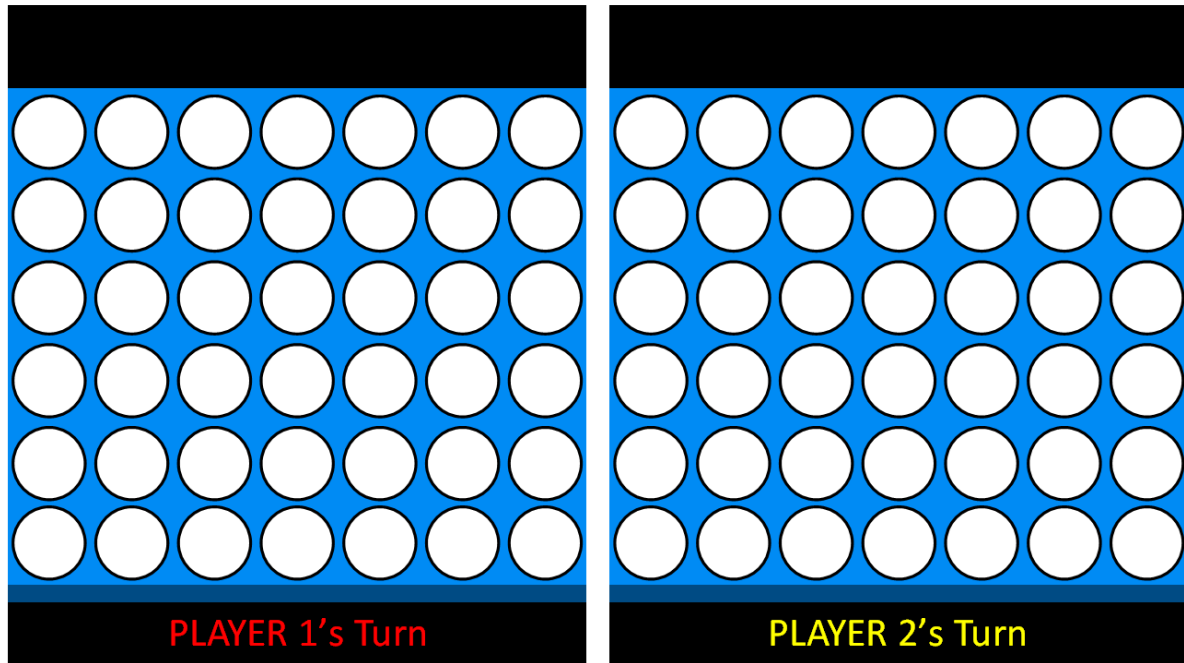
Playing Game UI Design

When the game is started:

- A message will be put in the console along the lines of this:

Connect 4 window has opened!
Select the window and left click the column you'd like to place your counter in.
Good luck!

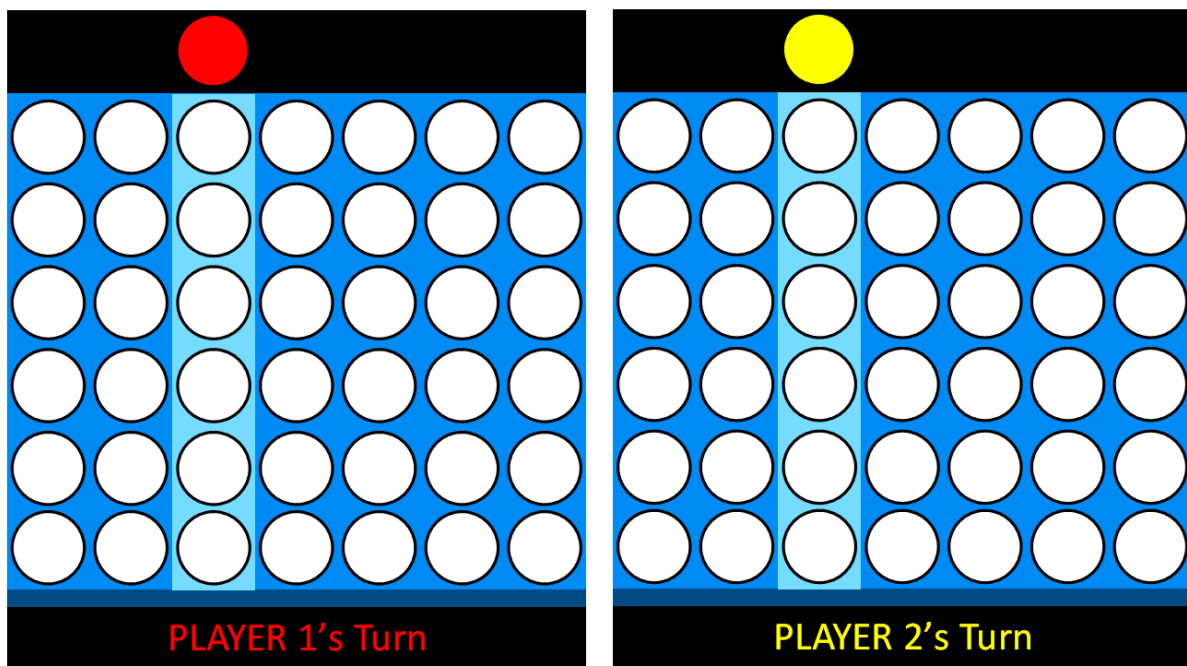
- A new window will pop up and show one of 2 screens (depending on which player is randomly selected to place the first counter)



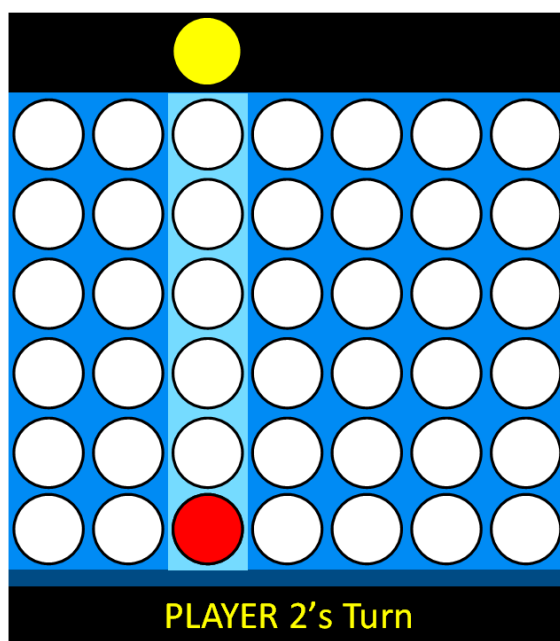
Note 1: The above and following diagrams have been made inside **Microsoft Publisher** with shape, text, layer, and alignment tools. The final project will look as close to what is shown above as possible.

Note 2: PLAYER 1 and PLAYER 2 will be replaced with the username of the appropriate player in the final product. The grids will be stored with a 2D array.

When a column is hovered over a highlighted rectangle will be displayed to indicate which column you are about to drop your counter in. It will also show the colour of counter about to be dropped at the top of the screen:

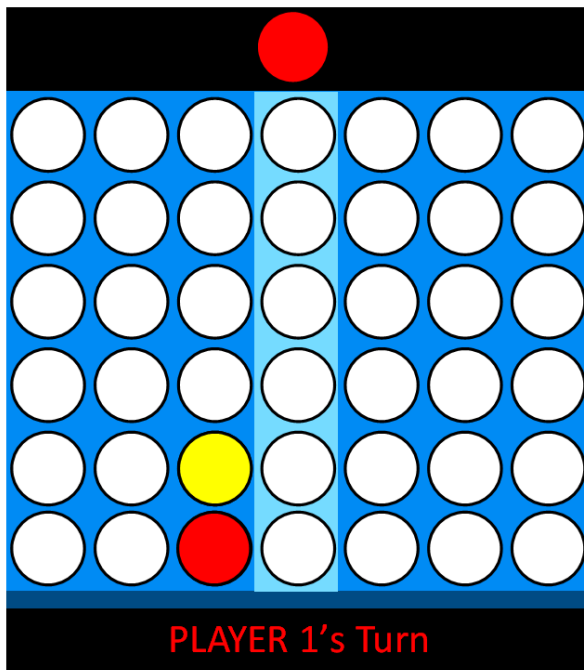


A counter will be dropped by clicking a **valid** column (one not already full with counters). It will be placed at the lowest point in the column. For example, after “Player 1” clicks to drop a counter in the third column in an empty grid, this is what would be shown next:



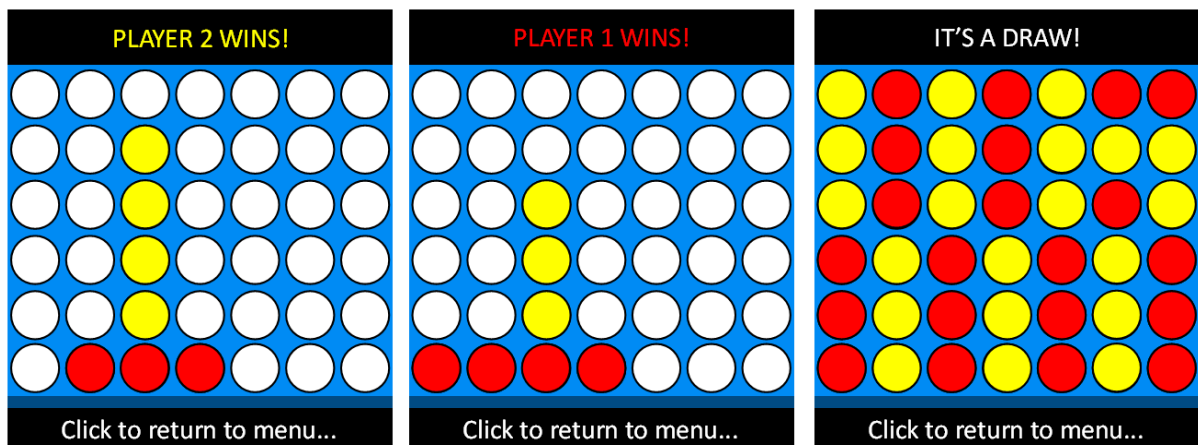
(**Note for above:** The counter has been dropped in column 3, and the turn has swapped to the other player's turn. As soon as any counter is dropped in the grid, the program will run a “win/draw” check to find if there are 4 counters in a row (win) or the board is full with no winner (draw).)

Then if “PLAYER 2” drops counter in the 3rd column, and “PLAYER 1” hovers over the 4th column right after – the screen would look like this:



This process repeats: Drop counter, check for win/draw, swap player. Drop counter, check for win/draw, swap player etc...

When a draw/win is detected – a winner message or a draw message will be displayed with the highlighting column system turned off. This will look like one of the following:



After a game has a winner/is found to be a draw – the database needs to be updated (with an **SQL statement**) to reflect the updated number of wins, losses, or draws for both players.

Player 2 wins: +1 for Player 2 wins in database | +1 for Player 1 losses in database

Player 1 wins: +1 for Player 1 wins in database | +1 for Player 2 losses in database

Draw: +1 for Player 1 & 2 draws in database

When the screen is clicked once again (as indicated above) the game window will shut down and you'll be taken back to the main menu.

Leaderboard Menu UI Design

- Leaderboard Menu -

[1] Sort by wins ←(1)

[2] Sort by losses ←(2)

[3] Sort by draws ←(3)

Enter number from above and press enter to select.

Leave blank to exit menu

> [validated user input from 1-3 or blank input] ←(4)

Printed Leaderboard UI Design

(This will be printed below the above menu)

Showing leaderboard sorted by wins/losses/draws: ←(5)

Username	Wins	Losses	Draws
username1	#	#	#
username2	#	#	#
username3	#	#	#
etc.	etc.	etc.	etc.

Press enter to return to leaderboard menu.

(1) When sort by wins is selected an SQL query will be ran to retrieve all usernames and their stats from the database **unsorted**. This result is then put through a bubble sort in the python program (this is to match the requirement of using a bubble sort for the purposes of the SQA)

(2) When sort by losses is selected an SQL query will be ran to retrieve all usernames and their stats from the database ordered by losses.

(3) When sort by draws is selected an SQL query will be ran to retrieve all usernames and their stats from the database ordered by draws.

(1) & (2) & (3) When any of these options are selected, they format the result obtained from the sql statement so it can be printed as a text based table and the **Printed Leaderboard** layout shown above is printed underneath the leaderboard menu sorted as instructed by the user. This will be displayed in the form of a text based table.

(4) Blank input returns to main menu. Otherwise, the program will do as described above.

(5) This simply displays the correct word based on what the user has sorted by.

TLA (Top Level Algorithm)

1. Initialize Program
2. Connect to database
3. **START WHILE LOOP**
4. Display main menu options
5. Get validated option from user
6. IF option == "" THEN
7. exit program
8. ELSE IF option == 1 or option == 2 THEN
9. IF player logged in THEN
10. logout_player(option) → option will be passed *in* to differentiate between player 1 and 2
11. ELSE THEN
12. login_player(option) → option will be passed *in* to differentiate between player 1 and 2
13. ELSE IF option == 3 THEN
14. open signup menu
15. ELSE IF option == 4 THEN
16. IF 2 players logged in THEN
17. start game
18. ELSE THEN
19. Display Error: 2 players must be logged in
20. ELSE IF option == 5:
21. open leaderboard menu
22. **END WHILE LOOP**

Refined TLA Design

Here I will expand on my TLA to make the process of implementing it easier. This is not pseudocode, but is rather breaking down the processes as much as I can at this stage so I can implement it with code later.

TLA Line 1: **Initialise program**

1. Create array of records for storing details about logged in players
2. + any other variables that will become clearer at the implementation stage

TLA Line 7: **exit program**

3. Display exiting message
4. Prompt user for blank input (so they need to click enter before the program shuts down)
5. Close console and exit program

TLA Line 10: **logout_player(option)**

Note: The "option" value is passed into this subroutine to differentiate between player 1 or player 2 getting logged out

1. In the PLAYERS array update record at index corresponding to logging out player 1 (index 0) or player 2 (index 0). The record values are updated to the following:
 - a. username = None
 - b. password = None
 - c. logged_in = False
2. Exit back to main menu

TLA Line 12: **login_player(option)**

Note: The "option" value is passed into this subroutine to differentiate between player 1 or player 2 getting logged in

1. **START WHILE LOOP**
2. Clear screen
3. Display login menu
4. Receive and store all usernames and passwords from database via SQL query
5. Get valid username from user and check for match from database which isn't already logged in as the other player. If blank input break out of while loop exit to main menu.
6. Get valid password from user and check it corresponds to username from database (if blank input break out of while loop exit to main menu)
7. In the PLAYERS array update record at index corresponding to logging in player 1 (index 1) or player 2 (index 0). The record values are updated to the following:
 - 7.1. username = inputted username
 - 7.2. password = inputted password
 - 7.3. logged_in = True
8. Display log-in successful message
9. **END WHILE LOOP**

TLA Line 14: **open sign-up menu**

1. **START WHILE LOOP**
2. Clear screen
3. Display sign-up menu
4. Receive and store all usernames from database via SQL query
5. Get valid username from user and check there isn't a match from database (6-16 characters, if blank input break out of while loop exit to main menu)
6. Get valid password from user (6-16 characters, if blank input break out of while loop exit to main menu)
7. Run SQL insert query to add new user to database, with default values of 0 for wins/losses/draws
8. Display sign-up successful message
9. **END WHILE LOOP**

TLA Line 17: **start game**

1. Initialise 2D array for the Connect 4 grid (6x7 all filled with 0s)
2. Set turn counter to 0
3. Randomise which player will start (player 1 or player 2) and store in turn_id variable
4. Display game is being launched message in console
5. Open Pygame window
6. **START WHILE LOOP**
7. IF end_game == False:
 - 7.1. Update Pygame window display to reflect the values stored within 2D array and mouse position (every 1/24 second)
 - 7.2. IF player clicks valid column THEN
 - 7.2.1. Drop correct player's counter into grid
 - 7.2.2. Increment turn counter
 - 7.2.3. IF there are 4 counters in a row or 2d array full with counters THEN:
 - 7.2.3.1. Set end_game = True
 - 7.2.3.2. IF player has won THEN
 - 7.2.3.2.1. Update database with SQL statements (+1 wins for winning player, +1 losses for losing player)
 - 7.2.3.3. ELSE THEN
 - 7.2.3.3.1. Update database with SQL statement (+1 draws for both players)
 - 7.2.4. Swap players turn (alter turn_id)
8. ELSE:
 - 8.1. **START WHILE LOOP**
 - 8.2. Update pygame window to show winner/draw screen
 - 8.3. IF window clicked THEN
 - 8.3.1. Shut down pygame window
 - 8.3.2. Break while loop, and exit back to main menu
9. **END WHILE LOOP**

Note for above: Steps 7.1 and 8.2 will require research at the implementation stage as I haven't used Pygame before. The refined code for those steps will be included in my new skills/research section of my implementation.

TLA Line 21: **open leaderboard menu**

1. **START WHILE LOOP**
2. Clear screen
3. Display leaderboard options
4. IF no stored user stats via SQL query THEN
 - 4.1. Display message to ask user to play game before you view leaderboard
 - 4.2. WHEN enter is pressed THEN
 - 4.2.1. Break out while loop and return to main menu
5. Get validated option from user
6. IF option == "" THEN
 - 6.1. Break out of while loop back to main menu
7. IF option == 1 THEN
 - 7.1. SQL query to get usernames, wins, losses, and draws **unsorted**
 - 7.2. Sort results by wins DESC via bubble sort in python
8. ELSE IF option == 2 THEN
 - 8.1. SQL query to get usernames, wins, losses, and draws sorted by losses DESC
9. ELSE IF option == 3 THEN
 - 9.1. SQL query to get usernames, wins, losses, and draws sorted by draws DESC
10. Display text-based formatted table from query result
11. Ask for input from user to prevent the loop resetting before the user has had a chance to look at the leaderboard (an input to halt program until the users says it should continue by pressing enter)
12. **END WHILE LOOP**

Further Refined TLA Code Design

From my Refined Code Design - I'll now develop the steps relevant to being required by the SQA project marking scheme. This will include designing the advanced higher concepts in the project, and also detailing integration design.

Integration Design

Queries

Get Usernames & Passwords Query [TLA 12.4]

Field(s) and Calculation(s)	usernames, passwords
Table(s)	user_details
Search Criteria	
Grouping	
Sort Order	

SQL CODE:

```
SELECT usernames, passwords  
FROM userdetails
```

Get Usernames Query [TLA 14.4]

Field(s) and Calculation(s)	usernames
Table(s)	user_details
Search Criteria	
Grouping	
Sort Order	

SQL CODE:

```
SELECT usernames  
FROM user_details
```

Insert User Query [TLA 14.7]

Fields	usernames, passwords, wins, losses, draws
Values	user inputted username, user inputted password, 0, 0, 0
Table(s)	user_details

SQL CODE:

```
INSERT INTO user_details (usernames, passwords, wins, losses, draws)
VALUES (user inputted username, user inputted password , 0, 0, 0)
```

Leaderboard Check For Results Query [TLA 21.4]

Field(s) and Calculation(s)	usernames
Table(s)	user_details
Search Criteria	
Grouping	
Sort Order	

SQL CODE:

```
SELECT usernames
FROM user_details
```

This query will allow the program to check if there are any entries in the table before the leaderboard submenu gives the user any options.

Leaderboard Get Unsorted Results [TLA 21.7.1]

Field(s) and Calculation(s)	usernames, wins, losses, draws
Table(s)	user_details
Search Criteria	
Grouping	
Sort Order	

SQL CODE:

```
SELECT usernames, wins, losses, draws
FROM user_details
```

Leaderboard Get Results Sorted By Losses [TLA 21.8.1]

Field(s) and Calculation(s)	usernames, wins, losses, draws
Table(s)	user_details
Search Criteria	
Grouping	
Sort Order	losses DESC

SQL CODE:

```
SELECT usernames, wins, losses, draws
FROM user_details
ORDER BY losses DESC
```

Leaderboard Get Results Sorted By Losses [TLA 21.9.1]

Field(s) and Calculation(s)	usernames, wins, losses, draws
Table(s)	user_details
Search Criteria	
Grouping	
Sort Order	draws DESC

SQL CODE:

```
SELECT usernames, wins, losses, draws
FROM user_details
ORDER BY draws DESC
```

Increment Winner Wins [TLA 17.7.2.3.2.1]

Field(s) and Calculation(s)	wins = wins + 1
Table(s)	user_details
Search Criteria	usernames = '<winning player username>'
Grouping	
Sort Order	

SQL CODE:

```
UPDATE user_details
SET wins = wins + 1
WHERE usernames = '<winning player username>'
```

Increment Loser Losses [TLA 17.7.2.3.2.1]

Field(s) and Calculation(s)	losses = losses + 1
Table(s)	user_details
Search Criteria	usernames = '<losing player username>'
Grouping	
Sort Order	

SQL CODE:

```
UPDATE user_details
SET losses = losses + 1
WHERE usernames = '<losing player username>'
```

Increment Player 1 and 2 Draws [TLA 17.7.2.3.3.1]

Field(s) and Calculation(s)	draws = draws + 1
Table(s)	user_details
Search Criteria	usernames = '<player1 username>' OR usernames = '<player2 username>'
Grouping	
Sort Order	

SQL CODE:

```
UPDATE user_details
SET draws = draws + 1
WHERE usernames = '<player1 username>' OR usernames = '<player2 username>'
```


Data Dictionary

This is the table that will store all user account information and win/loss/draw stats.

Entity: user_details					
Attribute Name	PK/FK	Data Type	Size	Required	Validation
usernames	PK	Text	16	Yes	length >= 6
passwords		Text	16	Yes	length >= 6
wins		Integer		Yes	
losses		Integer		Yes	
draws		Integer		Yes	

Advanced Higher Concepts Designs

Array Of Records [TLA 1.1]

In Python, an array of records is created from a class definition. So this piece of code will need to be at the top of the program. This record definition has been designed so that when called with no parameter passing, it will give default values to variables in the record (None, None, False). Note that although username and password are declared as STRING, python allows you to change the data type of a variable at will, which makes setting the string to a None (null) value valid.

```
RECORD Player IS {STRING username=None, STRING password=None, BOOLEAN logged_in=False}
```

This array of records is first created when the program starts. The array will always contain 2 records (player 1 and player 2). This code will initialise this array of 2 records:

```
DECLARE PLAYERS AS ARRAY OF Player INITIALLY [] * 2
```

This array will track who is logged in as each player - and will be altered at different stages of the program (anything related to displaying usernames or logging-in/signing up etc.)

Bubble Sort [TLA 21.6.2]

```
FUNCTION bubble_sort(result)
  DECLARE n INITIALLY length(result)
  DECLARE swapped INITIALLY TRUE
  WHILE swapped AND n >= 0
    SET swapped TO False
    FOR i = 0 to n-2 DO
      IF result[i][1] > result[i+1][1] THEN
        SET result[i], result[i+1] = result[i+1], result[i] #swap
        SET swapped TO TRUE
      END IF
    END FOR
    SET n TO n - 1
  END WHILE
  RETURN result
END FUNCTION
```

This bubble sort algorithm will take the passed in result of the SQL query (formatted in the form of a tuple) and will reorder it by wins, then return it. An example of how the tuple would be formatted:

```
((username1, wins1, losses1, draws1),(username2, wins2, losses2, draws2),(username3, wins3, losses3, draws3))
```

^ Each 'sub-tuple' is what is swapped in the bubble sort (result[i], result[i+1] = result[i+1], result[i]) where result[i][1] points to the wins value of each user from each 'sub tuple'.

2D Array [TLA 17.1]

```
DECLARE board AS ARRAY OF ARRAY OF INTEGER INITIALLY <6 element array, each containing 7  
element array, all set to 0>
```

This piece of code generates the 2D array filled with 0s of size 6x7. it will look like this when printed:

```
[[0, 0, 0, 0, 0, 0, 0],  
[0, 0, 0, 0, 0, 0, 0],  
[0, 0, 0, 0, 0, 0, 0],  
[0, 0, 0, 0, 0, 0, 0],  
[0, 0, 0, 0, 0, 0, 0],  
[0, 0, 0, 0, 0, 0, 0]]
```

In other words this is a 6 item list, with each sub-list having 7 items. The program will use “1” for player 1 having a counter in a slot in the board, “2” for player 2 having their counter in a slot in the board, and “0” for a blank slot. This could look like the following when some counters have been dropped (player 1 has won in this scenario):

```
[[0, 0, 0, 0, 0, 0, 0],  
[0, 0, 0, 0, 0, 0, 0],  
[1, 0, 0, 0, 0, 0, 0],  
[1, 0, 2, 0, 0, 0, 0],  
[1, 0, 2, 1, 0, 0, 0],  
[1, 2, 2, 2, 0, 0, 0]]
```

Project Implementation

Ongoing Testing, New Skills, & Research

The following table includes all of the above.

1

Aim: Ensure the Player class from design works as intended.**Finished Code:**

```
1  class Player:
2      '''This initialises the player class'''
3      #default values are set for class
4      def __init__(self, username=None, password=None, logged_in=False):
5          self.username = username #str/None
6          self.password = password #str/None
7          self.logged_in = logged_in #bool
```

Test Case Example:

```
10  '''Testing'''
11  player1 = Player() #create object of class Player with default variables
12  player2 = Player("username", "password", True) #create object of class Player with passed in vars
13  player3 = Player("scott", "glidden", True) #create object of class Player with passed in vars
14
15  print(player1.username, player1.password, player1.logged_in)
16  print(player2.username, player2.password, player2.logged_in)
17  print(player3.username, player3.password, player3.logged_in)
18
19  input()
```

Expected Test Case Result:

```
None None False
username password True
scott glidden True
```

Test Case Console Output:

```
None None False
username password True
scott glidden True
```

References: None.**Issues Encountered:** None.**Further Comments:**

This class definition was lifted straight from the design with no changes.

2

Aim: Create a clear screen procedure as this will be used throughout the program.

Finished Code:

```
1     import os
11    def cls():
12        '''Procedure to clear console screen'''
13        os.system('cls' if os.name == 'nt' else 'clear')
```

Test Case Example:

```
15    print("hello")
16    cls()
17    print("hello2")
18    input()
```

Expected Test Case Result:

hello2

Test Case Console Output:

hello2

New Skill:

- I learned a method of clearing the console screen. The os library is required along with the snippet of code above. When cls() is called the console is cleared. This will be helpful when switching between menus in the program. (<https://stackoverflow.com/questions/517970/how-to-clear-the-interpreter-console>)

Issues Encountered: None

Further Comments:

Had to do some googling to find a solution here - though the code for clearing the console was quickly found. Was simply a copy paste job here, no issues encountered.

3

Aim: Create function to validate a ranged input (including counting a blank input as valid)**Finished Code:**

```

15 def get_ranged_number_input(min, max):
16     while True: #repeat until valid number is received
17         print("\nEnter number from above and press enter to select.\nLeave blank to exit menu.")
18         user_input = input("> ") #get input
19         fail_message = f"Input must be a number between {min} and {max}, or blank to exit menu. Press enter to retry."
20
21
22         if user_input == "": #check for blank input first
23             return
24         try:
25             if min <= int(user_input) and int(user_input) <= max: #checks input is a number in range
26                 return int(user_input) #return the validated value
27             else: #if not in range, retry
28                 input(RED_TEXT + fail_message + RESET_TEXT)
29         except: #if error has occurred above, retry
30             input(RED_TEXT + fail_message + RESET_TEXT)
31
32
33
34
35
36
37
38
39
40
41
42 ##TEXT COLOUR CONSTANTS
43 RED_TEXT = "\u001b[0;31;40m"
44 RESET_TEXT = "\033[0;0m"

```

Test Case Example:

```

32 def menu():
33     while True:
34         print('menu options:') #DISPLAY MENU
35
36         menu_input = get_ranged_number_input(1, 5)
37         print(menu_input)
38
39
40
41
42
43
44
45 menu()

```

Expected Test Case Result:

The program will only accept values between 1 and 5 or a blank input and will ask the user to retry if unsuccessful.

Test Case Console Output:

menu options:

Enter number from above and press enter to select.

Leave blank to exit menu.

> 0

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> 1

1

menu options:

Enter number from above and press enter to select.

Leave blank to exit menu.

> 2

2

menu options:

Enter number from above and press enter to select.

Leave blank to exit menu.

> 3

3

menu options:

Enter number from above and press enter to select.

Leave blank to exit menu.

> 4

4

menu options:

Enter number from above and press enter to select.

Leave blank to exit menu.

> 5

5

menu options:

Enter number from above and press enter to select.

Leave blank to exit menu.

>

None

menu options:

Enter number from above and press enter to select.

Leave blank to exit menu.

> blank

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> one

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> ■

Note: "None" is intentionally printed when there is a blank input. The program will use this later to know when to exit menus/submenus. Otherwise, the output is as intended.

New Skills/Research:

- Learned how to colour text in python via escape codes to give better looking errors for users (<https://ozzmaker.com/add-colour-to-text-in-python/>)
- Learned how to use the "try except" statement for better input validation that doesn't throw an error if the input cannot be converted to an integer (https://www.w3schools.com/python/python_try_except.asp)

Issues Encountered:

Issue 1: Used incorrect sign for checking user input was less than max value. Line 25 originally looked like below.

```
25         if min <= int(user_input) and int(user_input) >= max: #checks input is a number in range
```

Solution 1: Changed sign around to ensure I was checking the correct range.

Issue 2: Incorrectly had an extra space after the colour escape codes - so there were some double spaces.

Solution 2: Fixed concatenation to get rid of double spaces.

Further Comments:

This code has been made as a function as it will be reused within different submenus.

4

Aim: Print the main menu as described in design, also showing who is logged in and how many are logged in.**Finished Code:**

```
16     def count_players():
17         count = 0 #set counter to 0
18         for record in PLAYERS: #for each record in PLAYERS list...
19             if record.logged_in: #if logged in...
20                 count += 1 #increment counter
21         return count
```

```

39 def menu():
40     while True:
41         cls() #clear screen
42         print(ASCII_MAIN_MENU) #menu ASCII art
43
44         '''Display Login Status'''
45         for index, record in enumerate(PLAYERS): #for each player
46             if record.logged_in == False: #display correct message
47                 print(f"Player {index+1}: " + RED_TEXT + "NOT LOGGED IN" + RESET_TEXT)
48             else:
49                 print(f"Player {index+1}: " + GREEN_TEXT + record.username + RESET_TEXT)
50
51         print("") #new line for neatness
52
53         '''Login/Logout Messages'''
54         for index, record in enumerate(PLAYERS): #for each player
55             if record.logged_in == True: #display correct message
56                 print(f" [{index+1}] Log-out '{record.username}'")
57             else:
58                 print(f" [{index+1}] Log-in {BLUE_TEXT}Player {index+1}\033[0;0m")
59
60         print(" [3] Sign-up")
61
62         logged_in_players = count_players()
63
64         if logged_in_players >= 2:
65             print(f" [4] Play Game! \u001b[32m[{logged_in_players}/2 players logged in]\033[0;0m")
66         else:
67             print(f" [4] Play Game! \u001b[0;31;40m[{logged_in_players}/2 players logged in]\033[0;0m")
68
69         print(" [5] View Leaderboard")
70
71         menu_input = get_ranged_number_input(1, 5)
72
73         if menu_input == None:
74             print("exit program")
75             quit()
76         if menu_input >= 1 and menu_input <= 2:
77             if PLAYERS[menu_input-1].logged_in == False:
78                 print("login menu")
79                 input()
80             else:
81                 PLAYERS[menu_input-1] = Player()
82         elif menu_input == 3:
83             print("signup menu")
84             input()
85
86         elif menu_input == 4:
87             #check if game can begin
88             if PLAYERS[0].logged_in == True and PLAYERS[1].logged_in == True:
89                 print("play game")
90                 input()
91             else:
92                 input(RED_TEXT + "Error: There are not 2 unique players logged in. Press enter to retry." + RESET_TEXT)
93
94         elif menu_input == 5:
95             print("leaderboard")
96             input()

```

```

97  ##TEXT COLOUR CONSTANTS
98  RED_TEXT = "\u001b[0;31;40m"
99  BLUE_TEXT = "\u001b[36m"
100  GREEN_TEXT = "\u001b[32m"
101  RESET_TEXT = "\033[0;0m"

103  ##ASCII ART
104  ASCII_MAIN_MENU = '''
105  |  V  |      ( )      |  V  |
106  | \ / |  _ _ _ _ _  | \ / |
107  | | | | / _ ' _ \  | | | | / _ ' _ \  | | | |
108  | | | | ( | | | | | | | | | | | | | | | | | | | | | |
109  | | | | \ , | | | | | | | | | | | | | | | | | | | |
110  '''
111  # https://patorjk.com/software/taag/#p=display&f=Big&t=Sign%20-%20up%20%20%20Menu
112
113  #setup array of records
114  PLAYERS = [Player()] * 2
115
116  '''Main Program'''
117  menu()

```

Test Cases:

Testing Part 1: I will be testing if each menu option prints its correct message (they only print what they lead to at the moment). See console outputs below.

```

  V      ( )      V
 / \    / \    / \
| | |  | | |  | | |
| | |  | | |  | | |
| | |  | | |  | | |
| | |  | | |  | | |
| | |  | | |  | | |

```

Player 1: NOT LOGGED IN
 Player 2: NOT LOGGED IN

```

[1] Log-in Player 1
[2] Log-in Player 2
[3] Sign-up
[4] Play Game! [0/2 players logged in]
[5] View Leaderboard

```

Enter number from above and press enter to select.
 Leave blank to exit menu.

```

> 1
login menu

```

Main Menu

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

- [1] Log-in Player 1
- [2] Log-in Player 2
- [3] Sign-up
- [4] Play Game! [0/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> 2

login menu

Main Menu

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

- [1] Log-in Player 1
- [2] Log-in Player 2
- [3] Sign-up
- [4] Play Game! [0/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> 3

signup_menu

Main Menu

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

- [1] Log-in Player 1
- [2] Log-in Player 2
- [3] Sign-up
- [4] Play Game! [0/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> 4

Error: There are not 2 unique players logged in. Press enter to retry.

^ Instead of printing "play game", as there are no players logged in an error is given instead. This is correct.

Main Menu

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

- [1] Log-in Player 1
- [2] Log-in Player 2
- [3] Sign-up
- [4] Play Game! [0/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> 5

leaderboard

Main Menu

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

- [1] Log-in Player 1
- [2] Log-in Player 2
- [3] Sign-up
- [4] Play Game! [0/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

>

exit program

Process returned 0 (0x0) execution time : 1.548 s

Press any key to continue . . .

All the above print the correct message based on what number has been entered. The input validation still works, as evidenced in implementation stage 4.

Testing Part 2: I will also be testing, via creating some fake logged in players as the login system doesn't exist yet, whether the menu messages display the correct information, i.e: how many are logged in and who is logged in. I will do this by editing this part of the code:

```
114 #setup array of records
115 #PLAYERS = [Player()] * 2
116 PLAYERS = [Player("scott", "glidden", True), Player("bob", "thebuilder", True)]
```

This allows me to mimic players being logged in. So for the above case the result is:

Main Menu

Player 1: scott

Player 2: bob

- [1] Log-out 'scott'
- [2] Log-out 'bob'
- [3] Sign-up
- [4] Play Game! [2/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> -

Perfect! It is printing the correct number logged in and WHO is logged in. I'll also test only one being logged in:

Main Menu

Player 1: **scott**

Player 2: **NOT LOGGED IN**

- [1] Log-out 'scott'
- [2] Log-in **Player 2**
- [3] Sign-up
- [4] Play Game! **[1/2 players logged in]**
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

>

Success!

New Skills/Research:

- Learned to print ASCII art by assigning a multiline string as a variable to print large main menu text similar to the big text in the design (<https://patorjk.com/software/taag/#p=display&f=Big&t=Sign%20-%20up%20%20%20Menu>)
- Learned to use the build-in python enumerate method to make code easier to read and more maintainable (<https://realpython.com/python-enumerate/>)

Issues Encountered:

Issue 1: Quitting game raises an error. This is as a range check was attempted on a non-integer value.

Solution 1: When selected, ensure quit() is run to stop the running of the python code before the error can occur.

Issue 2: Player numbers were originally printed as "0" and "1" rather than "1" and "2"

Solution 2: Add one to the variable when printed

By some miracle, these were the only issues I encountered. Quite an achievement if you ask me!

Further Comments:

The colour here was added to make the menu attractive and increase clarity for the end-user.

5

Aim: Connect to the database and create the table for storing user details and stats

Finished Code:

```

2      import mysql.connector

39     def connect_db():
40         #get the information for connecting to a database
41         mysql_user = "root"
42         mysql_password = "coolpassword"
43
44         global DB
45         global CURSOR
46
47         try:
48             #Checks details are correct
49             DB = mysql.connector.connect(
50                 host='localhost', user=mysql_user, passwd=mysql_password)
51         except mysql.connector.Error as err:
52             #If details are incorrect
53             print("Error with connection: {}".format(err))
54             input("Incorrect details or MySQL not setup correctly. Exiting program...")
55             quit()
56         else:
57             #Try connect
58             #Connect to SQL with details
59             DB = mysql.connector.connect(
60                 host='localhost', user=mysql_user, passwd=mysql_password)
61             #create database if not existent
62             CURSOR = DB.cursor()
63             #create database if not existent
64             CURSOR.execute("CREATE DATABASE IF NOT EXISTS stg_connect4")
65
66             #Now connect SPECIFICALLY to database that we now know exists
67             DB = mysql.connector.connect(
68                 host='localhost', user=mysql_user, passwd=mysql_password, database="stg_connect4")
69             CURSOR = DB.cursor(buffered=True)
70
71             #DEBUG
72             #CURSOR.execute("DROP TABLE user_details")
73
74             #Create the table if not existent
75             CURSOR.execute("CREATE TABLE IF NOT EXISTS user_details (usernames varChar(16) PRIMARY KEY, passwords varChar(16), wins int, losses int, draws int)")
76
156     '''Main Program'''
157     connect_db()
158     menu()
  
```

Test Case:

Inside MySQL workbench, both testing when the database has been dropped (deleted) and when it already exists, the workbench shows us that the data dictionary from the design section of the project has been implemented correctly - and successfully!

Query 1	Search	stg_connect4	stg_connect4.user_details						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants	DDL		
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges	Extra		
draws	int		YES			select,insert,update,references			
losses	int		YES			select,insert,update,references			
passwords	varchar(24)		YES	utf8mb4	utf8mb4_0900_...	select,insert,update,references			
usernames	varchar(24)		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references			
wins	int		YES			select,insert,update,references			

References:

(all related to database connection)

[Python MySQL Tutorial - Setup & Basic Queries \(w/ MySQL Connector\)](https://youtu.be/3vsC05rxZ8c) (https://youtu.be/3vsC05rxZ8c)

[#73 Python Database Connection | MySQL](https://youtu.be/vR5utJvN4JY) (https://youtu.be/vR5utJvN4JY)

<https://sites.google.com/rgc.aberdeen.sch.uk/rgcahcomputingrevision/software-design/mysql-access>

<https://linuxhint.com/create-table-in-sqlite-using-if-not-exists-statement/>
<https://www.mysqltutorial.org/mysql-create-drop-database.aspx>
<https://www.w3schools.com/sql/>
<https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqldb-cursorbuffered.html>

New Skills (referenced above):

- Learned how to create global variables within a procedure so the database can be more easily connected to throughout project
- Learned how to connect to a database and run sql queries - necessary for manipulating the database at later stages in the project
- Learned the CREATE TABLE IF NOT EXISTS and CREATE DATABASE IF NOT EXISTS statements in SQL to create databases and tables if they don't already exist so I can store information in them later
- Learned the DROP TABLE statement in SQL, was useful for debugging if the program would create the table if it didn't exist (let me delete table quickly for testing)

Issues Encountered:

Issue 1: Used the wrong password for MySQL. Originally used "password".

Solution 1: Used the correct password i.e "coolpassword"

Issue 2: Initially gave the create table statement on line 75 the wrong keyword for a field that holds a string. I had 'username string' first before it was changed to 'username varchar(16)'

Further Comments:

The drop table query on line 72 was used to delete the table to be able to test that the database would be created if it didn't exist.

6

Aim: Implement the signup menu, integration with database, with validation.**Finished Code:**

```
87     def sql_commit(sql):
88         CURSOR.execute(sql)
89         DB.commit()
90
91     def sql_select(sql):
92         CURSOR.execute(sql)
93         return CURSOR.fetchall()
94
95
96     def check_length_input(min, max, user_input):
97         '''Function checks if input length is in range'''
98         if min <= len(user_input) and len(user_input) <= max:
99             #success!
100             return True
101         else:
102             #not in range...
103             return False
```

```

40 def signup_menu():
41
42     #get list of usernames from database
43     result = sql_select("SELECT usernames FROM user_details")
44     valid_usernames = []
45     if result != None:
46         for i in range(len(result)):
47             #create list of usernames to be compared to
48             valid_usernames.append(result[i][0])
49
50     cls()
51     print(BLUE_TEXT + "\n- Sign-up Menu -" + RESET_TEXT + "\n(leave blank and press enter to return to main menu)\n")
52
53     in_progress = True
54
55     '''Get Valid Username'''
56     while in_progress:
57         username = input("Enter Username (6-16 characters) > ")
58         if username == "":
59             in_progress = False
60             break
61         elif username in valid_usernames:
62             input(RED_TEXT + "Invalid! This username is already in use. Please log in or choose a new username. Press enter to retry." + RESET_TEXT)
63         elif check_length_input(6, 16, username) == False:
64             input(RED_TEXT + "Invalid! Username must be between 6-16 characters. Press enter to try again." + RESET_TEXT)
65         else:
66             break
67
68     '''Get Valid Password'''
69     while in_progress:
70         password = input("Enter Password (6-16 characters) > ")
71         if password == "":
72             in_progress = False
73             break
74         elif check_length_input(6, 16, password) == False:
75             input(RED_TEXT + "Invalid! Password must be between 6-16 characters. Press enter to try again." + RESET_TEXT)
76         else:
77             break
78
79     if in_progress:
80         #execute SQL query to add the user
81         sql_commit(f"INSERT INTO user_details (usernames, passwords, wins, losses, draws) VALUES ('{username}', '{password}', 0, 0, 0)")
82         input(GREEN_TEXT + "Sign-up successful. Press enter to return to main menu." + RESET_TEXT)
83     else:
84         input(RED_TEXT + "Sign-up cancelled. Press enter to return to main menu." + RESET_TEXT)

```

Test Evidence:

Opening the menu:

Main Menu

Player 1: **scott**

Player 2: **NOT LOGGED IN**

- [1] Log-out 'scott'
- [2] Log-in **Player 2**
- [3] Sign-up
- [4] Play Game! **[1/2 players logged in]**
- [5] View Leaderboard

Enter number from above and press enter to select.
Leave blank to exit menu.

> 3_

Opens menu successfully as shown below:

- **Sign-up Menu** -
(leave blank and press enter to return to main menu)

Enter Username (6-16 characters) >

Entering details (with validation tested) shown below:

- **Sign-up Menu** -
(leave blank and press enter to return to main menu)

Enter Username (6-16 characters) > 1
Invalid! Username must be between 6-16 characters. Press enter to try again.

Enter Username (6-16 characters) > hello
Invalid! Username must be between 6-16 characters. Press enter to try again.

Enter Username (6-16 characters) > 123456
Enter Password (6-16 characters) > 1
Invalid! Password must be between 6-16 characters. Press enter to try again.

Enter Password (6-16 characters) > hello
Invalid! Password must be between 6-16 characters. Press enter to try again.

Enter Password (6-16 characters) > 123456
Sign-up successful. Press enter to return to main menu.

Validation is successful and only allows validated usernames. A blank input like so:

- **Sign-up Menu** -
(leave blank and press enter to return to main menu)

Enter Username (6-16 characters) > 1234567
Enter Password (6-16 characters) >
Sign-up cancelled. Press enter to return to main menu.

also successfully does it's job of sending the user back to the main menu like so:

Main Menu

Player 1: **scott**

Player 2: **NOT LOGGED IN**

- [1] Log-out 'scott'
- [2] Log-in **Player 2**
- [3] Sign-up
- [4] Play Game! **[1/2 players logged in]**
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

>

New Skills/Research:

- Learned to use DB.commit() procedure to ensure the changes to accounts in in the database are saved so when fetched again they are there (<https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlconnection-commit.html>)
- Learned to iterate through a tuple to pick out existing usernames so they program knows which usernames cannot be used as they already exist (<https://stackoverflow.com/questions/16021571/iterating-quickly-through-list-of-tuples>)

Issues Encountered:

Issue 1: Originally the DB.commit() call wasn't there, so the changes (adding a new account) wasn't saved in the database

Solution 1: Added the DB.commit() call within the sql_commit() procedure

Issue 2: Forgot to implement returning the main menu upon a blank input

Solution 2: Added an extra IF statement to check if the input was blank before anything happens with the input by breaking out the while loop, exiting to the main menu

Further Comments:

The sql_commit() and sql_select() procedures were made as they will be used throughout the whole program - so made sense in the moment.

7

Aim: Implement the login menu, integration with database, with validation.**Finished Code:**

```

40 def login_menu(player_id):
41
42     #SQL: Grab usernames and passwords
43     result = sql_select("SELECT usernames, passwords FROM user_details")
44     #create blank parallel arrays
45     valid_usernames = []
46     valid_passwords = []
47
48     #if there are entries...
49     if result != None:
50         #iterate through each tuple...
51         for i in range(len(result)):
52             #append values to parallel arrays
53             valid_usernames.append(result[i][0])
54             valid_passwords.append(result[i][1])
55
56     ##Remove usernames already logged in
57     #for both records in PLAYERS array...
58     for record in PLAYERS:
59         #check if they are logged in...
60         if record.logged_in:
61             #remove from list of valid names and password
62             valid_usernames.remove(record.username)
63             valid_passwords.remove(record.password)
64
65     cls() #clearscreen
66     print(f"\n{BLUE_TEXT}- Player {player_id} Login -" + RESET_TEXT + "\nLeave blank to exit menu.\n")
67
68     #get details
69     in_progress = True
70
71     while in_progress:
72         username = input(f"Enter Username > ")
73         if username == "":
74             in_progress = False
75             break
76         elif not username in valid_usernames:
77             input(RED_TEXT + "User not found or is already logged in... Press enter to retry." + RESET_TEXT)
78         else:
79             ref_index = valid_usernames.index(username)
80             break
81
82     while in_progress:
83         password = input(f"Enter Password > ")
84         if password == "":
85             in_progress = False
86             break
87         elif password in valid_passwords:
88             if password == valid_passwords[ref_index]:
89                 break
90             input(RED_TEXT + "Incorrect password... Press enter to retry." + RESET_TEXT)
91
92     if in_progress:
93         PLAYERS[player_id-1] = Player(username, password, True)
94         input(GREEN_TEXT + "Login successful. Press enter to return to main menu." + RESET_TEXT)
95     else:
96         input(RED_TEXT + "Login cancelled. Press enter to return to main menu." + RESET_TEXT)

```

Test Evidence:

Entering the following:

Main Menu

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

- [1] Log-in Player 1
- [2] Log-in Player 2
- [3] Sign-up
- [4] Play Game! [0/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> 1

brings up the login menu successfully:

- Player 1 Login -

Leave blank to exit menu.

Enter Username > _

it successfully validates your inputs based on the existing accounts in the database successfully:

- Player 1 Login -

Leave blank to exit menu.

Enter Username > hello

User not found or is already logged in... Press enter to retry.

Enter Username > clifford

Enter Password > hello

Incorrect password... Press enter to retry.

Enter Password > 123456

Incorrect password... Press enter to retry.

Enter Password > thereddog

Login successful. Press enter to return to main menu.

Upon clicking enter above you are successfully taken back to main menu, with the player who has logged in being shown underneath the ASCII art:

Main Menu

Player 1: clifford

Player 2: NOT LOGGED IN

- [1] Log-out 'clifford'
- [2] Log-in Player 2
- [3] Sign-up
- [4] Play Game! [1/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> █

This process works just as successfully for player 2. It also ensures you don't login with a username that is already logged in - see below:

- Player 2 Login -

Leave blank to exit menu.

Enter Username > clifford

User not found or is already logged in... Press enter to retry.

Enter Username > bobthe

Enter Password > builder

Login successful. Press enter to return to main menu.

Then once logged in - you can logout again easily by choosing the log-out option:

Main Menu

Player 1: clifford

Player 2: bobthe

- [1] Log-out 'clifford'
- [2] Log-out 'bobthe'
- [3] Sign-up
- [4] Play Game! [2/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> 1

Upon above input, menu is successfully redisplayed with the player logged out:

Main Menu

Player 1: NOT LOGGED IN

Player 2: bobthe

- [1] Log-in Player 1
- [2] Log-out 'bobthe'
- [3] Sign-up
- [4] Play Game! [1/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

>

It's also worth mentioning, the number of players logged in is also successfully displayed on the message for option 4 (Play Game) throughout.

References:

No references were required here.

New Skills/Research:

No new skills present in this section that haven't already been mentioned.

Issues Encountered:

Issue 1: Error originally in my traversal of the tuple - messed up accessing the sub-tuples by index

Solution 1: Fixed the incorrect indexing so the tuple was not out of range

8

Aim: Implement the game board LOGIC (will just use a text based output for the board for now - I will turn this into a graphical display with Pygame after)

Finished Code:

```
2 import random
```

```

258 def play_game():
259     board = [[0 for c in range(7)] for r in range(6)] #generate board
260     turns = 0 #set default turns counter
261     turn_id = random.randint(0, 1) #choose which player to start with
262
263     win_state = -1 #default to ongoing game winstate
264
265     print(GREEN_TEXT + "\nConnect 4 window has opened! \nSelect the window and left click the column you'd like to place your counter in. \nGood luck!" + RESET_TEXT) #instruction message
266
267     run = True
268     while run: #ensures loop runs until told otherwise
269
270         for line in board: #this will be made into graphical menu
271             print(line) #this will be made into graphical menu
272         print(f"Player {turn_id+1}'s turn!") #this will be made into graphical menu
273
274         if win_state == -1:
275             while True:
276                 num = int(input("Which column to drop in (0-6): "))
277                 if board[0][num] == 0:
278                     drop_counter(board, num, turn_id) #drop counter
279                     break
280                 else:
281                     print("Invalid position!")
282             turns += 1 #increment turn
283
284             '''Check Win'''
285             win_state = check_win(board, turn_id, turns)
286
287             if win_state == 1: #win_state: 1 > Win
288                 #win for winner
289                 sql_commit(f"UPDATE user_details SET wins = wins + 1 WHERE usernames = '{PLAYERS[turn_id].username}'")
290                 #loss for loser
291                 sql_commit(f"UPDATE user_details SET losses = losses + 1 WHERE usernames = '{PLAYERS[1-turn_id].username}'")
292             elif win_state == 0: #win_state: 0 > Draw
293                 sql_commit(f"UPDATE user_details SET draws = draws + 1 WHERE usernames = '{PLAYERS[0].username}' OR usernames = '{PLAYERS[1].username}'")
294             else: #win_state: -1 > Continue game
295                 turn_id = 1 - turn_id #flip 0 and 1
296             else:
297                 print(f"GAME ENDED. Win state: {win_state}. Last turn by {PLAYERS[turn_id].username}.")
298                 input()
299
300 def drop_counter(board, selected_column, drop_value):
301     drop_value += 1 #make 1/2 rather than 0/1
302     for row in range(5, -1, -1):
303         if board[row][selected_column] == 0: #if column is not full...
304             board[row][selected_column] = drop_value #update 2d board array
305             return #exit function when counter has been dropped
306
307 def check_win(board, ref, turns):
308     #win_state: 1 > Win
309     #win_state: 0 > Draw
310     #win_state: -1 > Continue game
311
312     ref += 1 #make 1/2 rather than 0/1
313
314     player_won = False
315
316     '''Check Horizontal Positions'''
317     for r in range(ROWS):
318         for c in range(COLUMNS-3):
319             if ref == board[r][c] == board[r][c+1] == board[r][c+2] == board[r][c+3]:
320                 return 1
321
322     '''Verticle'''
323     for r in range(ROWS-3):
324         for c in range(COLUMNS):
325             if ref == board[r][c] == board[r+1][c] == board[r+2][c] == board[r+3][c]:
326                 return 1
327
328     '''Diagonal 1'''
329     for r in range(ROWS-3):
330         for c in range(COLUMNS-3):
331             if ref == board[r][c] == board[r+1][c+1] == board[r+2][c+2] == board[r+3][c+3]:
332                 return 1
333
334     '''Diagonal 2'''
335     for r in range(3, ROWS):
336         for c in range(COLUMNS-3):
337             if ref == board[r][c] == board[r-1][c+1] == board[r-2][c+2] == board[r-3][c+3]:
338                 return 1
339
340     ##if nothing is returned by this point nobody has won so:
341     if turns >= 42: #if board full (draw)
342         return 0
343     else:
344         return -1
345

```

```
339  ##CONSTANTS
340  COLUMNS = 7
341  ROWS = 6
```

Test Cases:

Here is evidence for the counters being dropped successfully where instructed, the turn swapping, and an example of the game recognising there is 4 counters in a row

Main Menu

Player 1: clifford

Player 2: bobthe

- [1] Log-out 'clifford'
- [2] Log-out 'bobthe'
- [3] Sign-up
- [4] Play Game! [2/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.
Leave blank to exit menu.

> 4

Connect 4 window has opened!

Select the window and left click the column you'd like to place your counter in.

Good luck!

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
```

Player 1's turn!

Which column to drop in (0-6): 0

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
```

Player 2's turn!

Which column to drop in (0-6): 1

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[1, 2, 0, 0, 0, 0, 0]
```

Player 1's turn!

Which column to drop in (0-6): 0

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 2, 0, 0, 0, 0, 0]
```

Player 2's turn!

Which column to drop in (0-6): 5

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 2, 0, 0, 0, 2, 0]
```

Player 1's turn!

Which column to drop in (0-6): 0

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 2, 0, 0, 0, 2, 0]
```

Player 2's turn!

Which column to drop in (0-6): 3

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 2, 0, 2, 0, 2, 0]
```

Player 1's turn!

Which column to drop in (0-6): 0

```
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[1, 2, 0, 2, 0, 2, 0]
[1, 2, 0, 2, 0, 2, 0]
```

Player 1's turn!

GAME ENDED. Win state: 1. Last turn by clifford.

Note: win state of 1 is a win, win state of 0 is a draw. 1 represents player 1 counter. 2 represents player 2 counter. It would be impractical to evidence every single possible win position, however here are a few examples (including a draw)

which demonstrate that the counter dropping and win detection are working as intended:

Vertical win detection:

```
Which column to drop in (0-6): 0
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[2, 2, 0, 0, 0, 0, 0]
[1, 2, 1, 0, 0, 0, 0]
[1, 2, 2, 1, 0, 0, 0]
[2, 1, 1, 2, 0, 0, 0]
Player 2's turn!'
GAME ENDED. Win state: 1. Last turn by bobthe.
```

Draw detection:

```
Which column to drop in (0-6): 0
[1, 1, 2, 1, 2, 1, 2]
[2, 1, 2, 1, 2, 1, 2]
[1, 1, 2, 1, 2, 1, 2]
[2, 2, 1, 2, 1, 2, 1]
[1, 2, 1, 2, 1, 2, 1]
[2, 2, 1, 2, 1, 2, 1]
Player 1's turn!'
GAME ENDED. Win state: 0. Last turn by clifford.
```

Diagonal win detection:

```
Which column to drop in (0-6): 3
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 2, 0, 0, 0]
[0, 0, 2, 1, 0, 0, 0]
[0, 2, 2, 1, 0, 0, 0]
[2, 1, 1, 1, 2, 0, 0]
Player 2's turn!'
GAME ENDED. Win state: 1. Last turn by bobthe.
```

Horizontal win detection:

```
Which column to drop in (0-6): 3
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 2, 0, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]
[2, 2, 2, 0, 0, 0, 0]
[1, 1, 1, 1, 0, 0, 0]
Player 1's turn!'
GAME ENDED. Win state: 1. Last turn by clifford.
```

Column full detection:

```

Which column to drop in (0-6): 0
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[2, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[2, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
Player 1's turn!
Which column to drop in (0-6): 0
[0, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[2, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[2, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
Player 2's turn!
Which column to drop in (0-6): 0
[2, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[2, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
[2, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0]
Player 1's turn!
Which column to drop in (0-6): 0
Invalid position!
Which column to drop in (0-6): █

```

^ successfully gives error and asks again when the column is full.

New Skills/Research:

- Learned how to create, traverse, and manipulate 2D arrays (<https://www.w3schools.blog/data-structure-2d-array#:~:text=Similar%20to%20one%2Ddimensional%20arrays,other%20is%20its%20column%20number>)
- Learned you can flip a variable between 0 and 1 by doing "var = 1 - var". This was useful for swapping player turns between counters being placed. (<https://stackoverflow.com/questions/4084050/can-you-use-arithmetic-operators-to-flip-between-0-and-1>)

Issues Encountered:

Issue 1: Had a few bugs with going out of range of the 2D array when checking for a win. The odd typo was causing this.

Solution 1: Corrected the typos, and ensured all types of win (up, across, diagonal) all work as intended.

Issue 2: The detection for a draw (i.e there having been 42 successful turns) would stop when there was still one space left in the board. This was a simple mistake in the range check for there having been 42 turns.

Solution 2: Changed from checking ≥ 41 turns to ≥ 42 turns

9

Aim: Implement the Connect 4 graphical display with Python including the game ended screen.**Finished Code:**

```
3 import pygame
```

```

258 def draw_window(board, turn_id, game_font, hover_column, win_state):
259     turn_id += 1 #make 1/2 instead of B/I
260
261     WIN.fill(BLACK)
262     for c in range(COLUMNS):
263         if c == hover_column:
264             if turn_id == 1:
265                 pygame.draw.circle(
266                     WIN, RED, (c*SQUARE_DIMENSIONS+(SQUARE_DIMENSIONS/2), SQUARE_DIMENSIONS/2), int(CIRCLE_RADIUS)-4)
267             else:
268                 pygame.draw.circle(WIN, YELLOW, (c*SQUARE_DIMENSIONS+(
269                     SQUARE_DIMENSIONS/2), SQUARE_DIMENSIONS/2), int(CIRCLE_RADIUS)-4)
270     for r in range(ROWS):
271         if c == hover_column:
272             pygame.draw.rect(WIN, SELECT_COLOUR, (c*SQUARE_DIMENSIONS, (r*SQUARE_DIMENSIONS) +
273                 SQUARE_DIMENSIONS, SQUARE_DIMENSIONS, SQUARE_DIMENSIONS))
274         else:
275             pygame.draw.rect(WIN, BLUE, (c*SQUARE_DIMENSIONS, (r*SQUARE_DIMENSIONS) +
276                 SQUARE_DIMENSIONS, SQUARE_DIMENSIONS, SQUARE_DIMENSIONS))
277
278     #display correct counter
279     if board[r][c] == 1:
280         pygame.draw.circle(WIN, RED, (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
281             r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)), int(CIRCLE_RADIUS))
282     elif board[r][c] == 2:
283         pygame.draw.circle(WIN, YELLOW, (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
284             r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)), int(CIRCLE_RADIUS))
285     else:
286         pygame.draw.circle(WIN, WHITE, (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
287             r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)), int(CIRCLE_RADIUS))
288
289     #black outline
290     pygame.draw.circle(WIN, BLACK, (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
291         r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)), int(CIRCLE_RADIUS), 4)
292
293     pygame.draw.rect(WIN, DARK_BLUE, (c*SQUARE_DIMENSIONS,
294         (7*SQUARE_DIMENSIONS), SQUARE_DIMENSIONS, SQUARE_DIMENSIONS/6))
295
296
297 if win_state == -1:
298     if turn_id == 1:
299         #TODO: find out what 1 means
300         label = game_font.render(f"{PLAYERS[0].username}'s Turn", 1, RED)
301     else:
302         #TODO: find out what 1 means
303         label = game_font.render(
304             f"{PLAYERS[1].username}'s Turn", 1, YELLOW)
305     text_center = label.get_rect(
306         center=(WIDTH/2, HEIGHT-(SQUARE_DIMENSIONS/2.3)))
307     WIN.blit(label, text_center)
308
309 elif win_state == 0:
310     #TODO: find out what 1 means
311     label = game_font.render(f"IT'S A DRAW!", 1, WHITE)
312     text_center = label.get_rect(center=(WIDTH/2, SQUARE_DIMENSIONS/2.3))
313     WIN.blit(label, text_center)
314
315     #TODO: find out what 1 means
316     label = game_font.render(f"Click to return to menu...", 1, WHITE)
317     text_center = label.get_rect(
318         center=(WIDTH/2, HEIGHT-(SQUARE_DIMENSIONS/2.3)))
319     WIN.blit(label, text_center)
320 else:
321     if turn_id == 1:
322         label = game_font.render(f"{PLAYERS[0].username} WINS!", 1, RED)
323     else:
324         label = game_font.render(f"{PLAYERS[1].username} WINS!", 1, YELLOW)
325     text_center = label.get_rect(center=(WIDTH/2, SQUARE_DIMENSIONS/2.3))
326     WIN.blit(label, text_center)
327
328     #TODO: find out what 1 means
329     label = game_font.render(f"Click to return to menu...", 1, WHITE)
330     text_center = label.get_rect(
331         center=(WIDTH/2, HEIGHT-(SQUARE_DIMENSIONS/2.3)))
332     WIN.blit(label, text_center)
333
334     pygame.display.update()

```

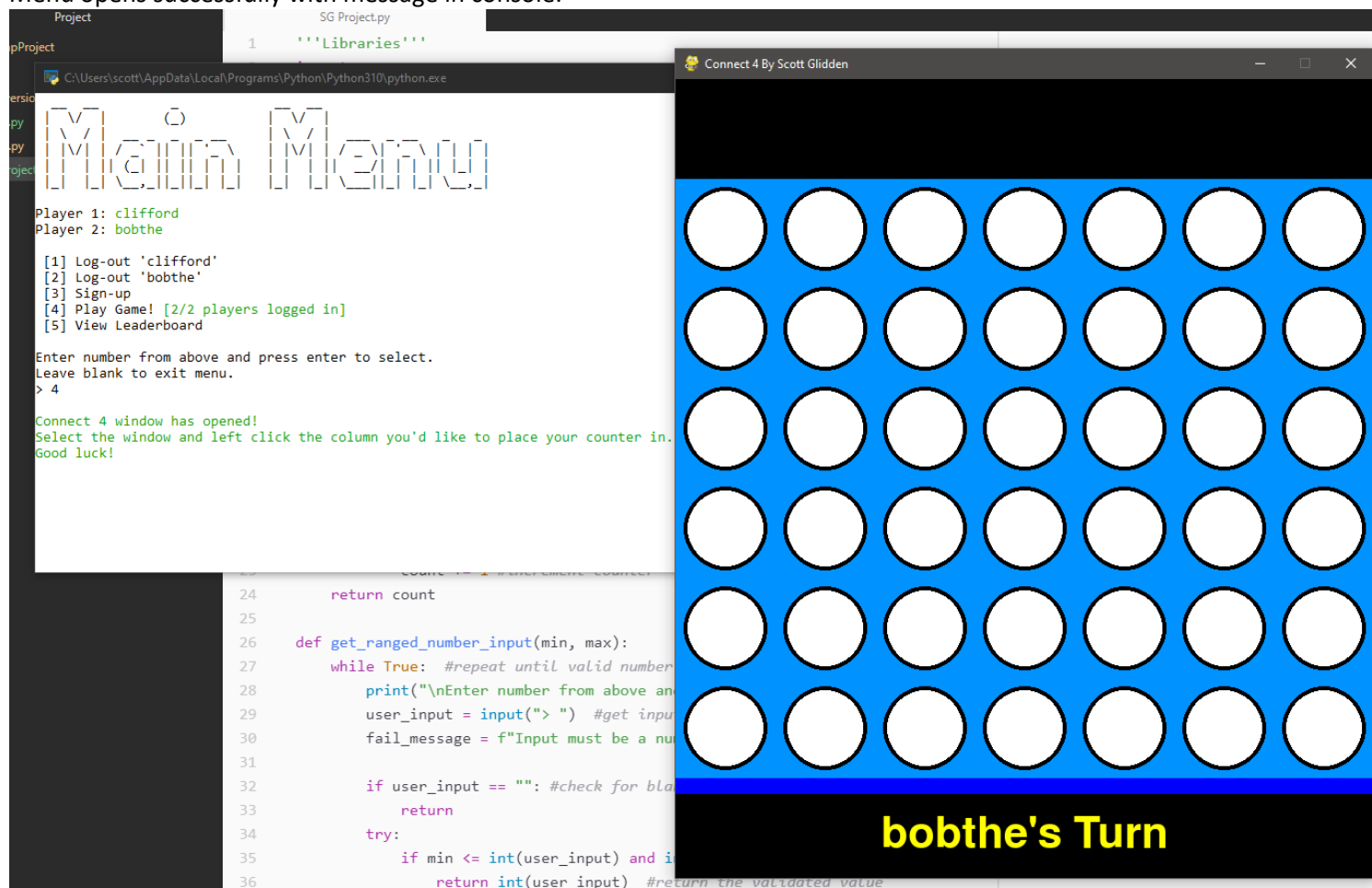
```

445  ##CONSTANTS
446  SQUARE_DIMENSIONS = 100
447  CIRCLE_RADIUS = (SQUARE_DIMENSIONS/2) - 8
448  COLUMNS = 7
449  ROWS = 6
450  WIDTH = SQUARE_DIMENSIONS * 7
451  HEIGHT = SQUARE_DIMENSIONS * 8 #8 not 6 for the space at top and bottom
453  ##PYGAME COLOUR CONSTANT TUPLES
454  WHITE = (255, 255, 255)
455  BLUE = (0, 147, 255)
456  DARK_BLUE = (0, 0, 255)
457  YELLOW = (255, 255, 0)
458  RED = (255, 0, 0)
459  BLACK = (0, 0, 0)
460  SELECT_COLOUR = (102, 190, 255)
461  FPS = 24

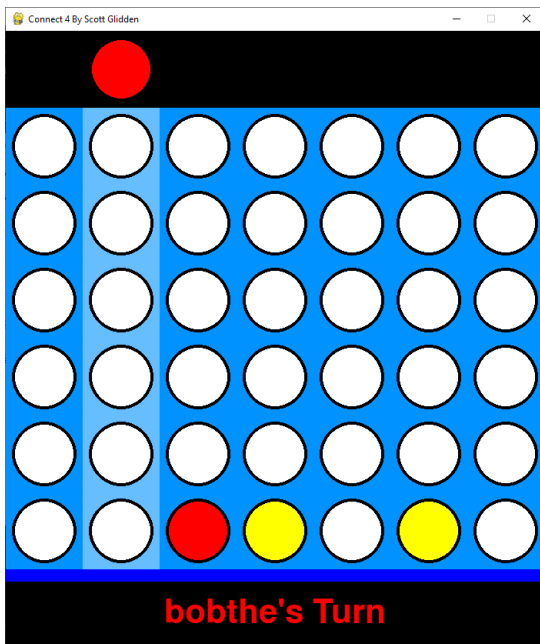
```

Test Cases:

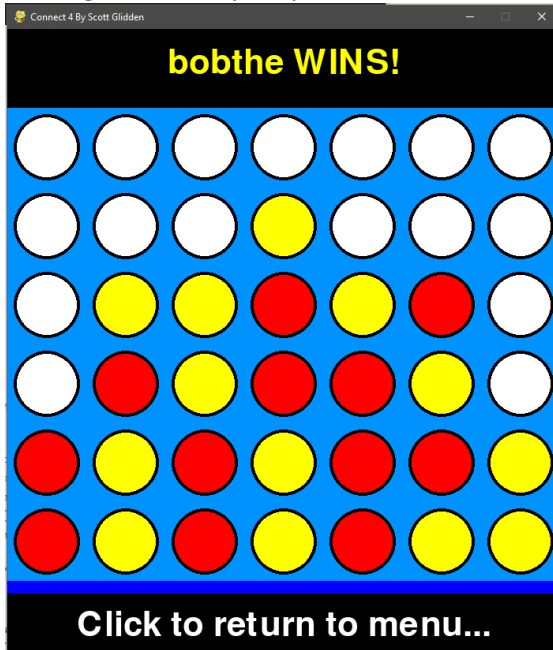
Menu opens successfully with message in console:



Colour of column changes when hovered:



Clicking successfully drops counters and win screen is shown:



Clicking after a game ends brings you back to main menu shown below:

Main Menu

Player 1: clifford

Player 2: bobthe

- [1] Log-out 'clifford'
- [2] Log-out 'bobthe'
- [3] Sign-up
- [4] Play Game! [2/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.

Leave blank to exit menu.

> _

References:

<https://www.pygame.org/docs/> (this was heavily referred to for all things pygame and was what I used to be able to implement all of the above - and what is described below)

New Skills/Research:

- Learned the basics of Pygame to implement the graphical display. This involved: creating a window, drawing shapes, drawing shapes based on variables, setting an FPS, adding mouse hover and click detection, and changing colours of shapes. This was all very foreign to me before tackling this project, so gaining the skills to create a graphical interface was a big learning curve for me. More specifically this included learning the function of:
 - pygame.time.Clock() - create clock object to keep track of time
 - pygame.font.init() & pygame.font.SysFont() - to change font of text
 - pygame.draw.circle & pygame.draw.rect - to draw circles and rectangles on window
 - WIN.fill() - to colour background of window
 - pygame.MOUSEBUTTONDOWN - detecting clicks
 - pygame.WINDOWFOCUSLOST & pygame.WINDOWLEAVE - detecting mouse leaving window
 - pygame.event.get() - necessary to run any user input detection
 - pygame.MOUSEMOTION - detecting moving mouse for hover columns
 - WIN.blit - to overlay text on screen
 - game_font.render() - to render text for being displayed on screen

Issues Encountered:

Issue 1: Inevitably while using Pygame to draw shapes on the screen, there were many miscalculations for the coordinates at which the shapes should be drawn at, along with miscalculations of their size. With displaying the text it also took some trial and error to get the text exactly where it looked best.

Solution 1: By adjusting some of the variables I was able to get the look I wanted on the Pygame window to closely match the design. There surprisingly weren't any notable problems other than the odd minor typo (not worth mentioning) while implementing this section, however some trial and error was involved in adjusting the variables to get the Pygame window looking just right.

10 **Aim:** Implement the leaderboard menu, integrating with the database.

Finished Code:

```

42 v def bubble_sort(result):
43     '''Sorts the result from SQL statement'''
44     #initial values for sort
45     n = len(result)
46     swapped = True
47 v     while swapped and n >= 0:
48         swapped = False
49 v         for i in range(n-1):
50 v             if result[i][1] < result[i+1][1]:
51                 result[i], result[i+1] = result[i+1], result[i] #swap
52                 swapped = True
53             n -= 1
54         #return sorted result
55     return result

42 def leaderboard_menu():
43     result = sql_select(f"SELECT usernames FROM user_details")
44     if result == None:
45         input(RED_TEXT + "There is no leaderboard to show as there are no signed up users yet! Sign-up in the main menu first. \n(Press enter to return to main menu)" + RESET_TEXT)
46         return
47     while result != None:
48         cls() #clear screen
49         print(BLUE_TEXT + "\n- Leaderboard Menu -" + RESET_TEXT)
50         print("[1] Sort by wins")
51         print("[2] Sort by losses")
52         print("[3] Sort by draws")
53
54         user_input = get_ranged_number_input(1, 3)
55
56         if user_input == None:
57             print("\nReturning to main menu...\n")
58             break
59         elif user_input >= 1 and user_input <= 3:
60
61             if user_input == 1:
62                 result = sql_select(f"SELECT usernames, wins, losses, draws FROM user_details")
63                 #sorting by wins uses a python bubble sort instead of relying on SQL
64                 sorted_result = bubble_sort(result)
65                 sort_by = "wins"
66             elif user_input == 2:
67                 sorted_result = sql_select(f"SELECT usernames, wins, losses, draws FROM user_details ORDER BY losses DESC")
68                 sort_by = "losses"
69             elif user_input == 3:
70                 sorted_result = sql_select(f"SELECT usernames, wins, losses, draws FROM user_details ORDER BY draws DESC")
71                 sort_by = "draws"
72
73             if sorted_result != None:
74                 table_headings = ["Username", "Wins", "Losses", "Draws"]
75                 #get largest value from each column
76
77                 #set default column widths
78                 table_width = []
79                 for item in table_headings:
80                     table_width.append(len(item))
81
82                 for row in sorted_result:
83                     for index, item in enumerate(row):
84                         if len(str(item)) > table_width[index]:
85                             table_width[index] = len(str(item))
86
87                 '''Actually print table'''
88                 print(f"\nShowing leaderboard sorted by {sort_by}:")
89                 display = BLUE_TEXT
90                 for index, item in enumerate(table_headings):
91                     display += "|" + \
92                         str(item).ljust(table_width[index], " ") + " "
93                 display += "\n\033[0;0m"
94                 print(display)
95
96                 for row in sorted_result:
97                     display = ""
98                     for index, item in enumerate(row):
99                         display += "|" + \
100                             str(item).ljust(table_width[index], " ") + " "
101                     display += "\n"
102                     print(display)
103
104                 input("\n(Press enter to return to leaderboard menu)")
105             else:
106                 print(RED_TEXT + "\n\nNobody is listed on the leaderboard yet! Use the 'sign-up' option on the main menu to create some accounts and view the leaderboard. (Press enter to return to main menu)" + RESET_TEXT)
107         ...

```

Test Cases:

Entering "5" into the main menu successfully brings up the leaderboard menu:

```

  ( )
Main Menu

```

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

```

[1] Log-in Player 1
[2] Log-in Player 2
[3] Sign-up
[4] Play Game! [0/2 players logged in]
[5] View Leaderboard

```

Enter number from above and press enter to select.

Leave blank to exit menu.

> 5

Brings up:

```

- Leaderboard Menu -

```

```

[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

```

Enter number from above and press enter to select.

Leave blank to exit menu.

> -

Each option displays a leaderboard sorted by the correct field. Sort by wins correctly displays the table sorted by wins:

- Leaderboard Menu -

- [1] Sort by wins
- [2] Sort by losses
- [3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.

> 1

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	14	4
bobthe	8	4	2
sglidden	5	6	1
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

(press enter to return to leaderboard menu)

Sort by losses correctly displays the table sorted by losses:

- Leaderboard Menu -

- [1] Sort by wins
- [2] Sort by losses
- [3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.

> 2

Showing leaderboard sorted by losses:

Username	Wins	Losses	Draws
clifford	15	14	4
fraser	1	6	1
sglidden	5	6	1
bobthe	8	4	2
orangeeater	0	1	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu)

Sort by draws correctly displays the table sorted by draws:

- Leaderboard Menu -

[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
> 3

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	14	4
bobthe	8	4	2
fraser	1	6	1
sglidden	5	6	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu)

As suggested above, pressing enter successfully brings you back to the leaderboard menu:

- Leaderboard Menu -

[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
>

New Skills/Research:

- Formatting a text based table via the use of the ljust() method to pad out a string to a set length (https://www.w3schools.com/python/ref_string_ljust.asp)

Issues Encountered:

Issue 1: Originally had an error in the SQL statement for updating the wins/losses/draws where I didn't put the username being searched for inside quotation marks

Solution 1: Put the field in quotation marks so the SQL could be executed correctly

11 **Aim:** Clean code with whitespace and commentary

Program Code:

Refer to completed program code below in next section (would be impractical, and a big waste of paper to have it twice)

Test Cases:

As this final cleanup of code didn't involve any changes in the logic of the code, no testing is necessary. Only commentary and whitespace has been added which doesn't impact the program itself - it only impacts the programmer reading the code. Extensive testing of the program will be done at a later stage.

Completed Program Code

I will annotate the completed program code shown on the next pages to draw attention to the advanced higher concepts & integration from the design.

```

1  '''Libraries'''
2  import os
3  import pygame
4  import random
5  import mysql.connector
6
7  class Player:
8      '''This initialises the player class'''
9      def __init__(self, username=None, password=None, logged_in=False):
10         self.username = username #str/None
11         self.password = password #str/None
12         self.logged_in = logged_in #bool
13
14
15     '''Menu Things'''
16     def cls():
17         '''Procedure to clear console screen'''
18         os.system('cls' if os.name == 'nt' else 'clear')
19
20     def count_players():
21         '''Function counts logged in players'''
22         #Count occurrence algorithm
23         count = 0 #set count to 0
24         for record in PLAYERS:
25             if record.logged_in: #count number logged in
26                 count += 1
27         return count
28
29     def get_ranged_number_input(min, max):
30         '''Function returns valid ranged number'''
31         while True: #repeat until valid number is received
32             print("\nEnter number from above and press enter to select.\nLeave blank to exit menu.")
33             user_input = input("> ") #get input
34             fail_message = f"Input must be a number between {min} and {max}, or blank to exit menu. Press enter to retry."
35
36             if user_input == "": #check for blank input first
37                 return #returns blank input
38             try:
39                 if min <= int(user_input) and int(user_input) <= max: #checks input is a number in range
40                     return int(user_input) #return the validated value
41                 else: #if not in range, retry
42                     input(RED_TEXT + fail_message + RESET_TEXT)
43             except: #if error has occurred above, retry
44                 input(RED_TEXT + fail_message + RESET_TEXT)
45
46     def bubble_sort(result):
47         '''Sorts the result from SQL statement by wins via Bubble Sort'''
48         n = len(result) #set length of list
49         swapped = True #assume swapped by default
50         while swapped and n >= 0: #while values are still to be sorted and algorithm hasn't reached end of list...
51             swapped = False #set swapped to false (try end loop) until told otherwise
52             for i in range(n-1): #for each item until end n
53                 if result[i][1] < result[i+1][1]: #if needing swapped...
54                     result[i], result[i+1] = result[i+1], result[i] #SWAP!
55                     swapped = True #set swapped to true to trigger the while loop again
56             n -= 1 #decrement length of list to iterate through
57         return result #return sorted result
58
59     def leaderboard_menu():
60         '''Display Leaderboard menu'''
61         result = sql_select(f"SELECT usernames FROM user_details") #run SQL
62         if result == None: #if no result in database
63             input(RED_TEXT + "There is no leaderboard to show as there are no signed up users yet! Sign-up in the main menu first. \n(Press enter to return to main menu)" + RESET_TEXT) #send error
64             return #exit menu
65         while result != None: #if there are results run leaderboard menu loop
66             cls() #clear screen

```

[illegible]


```

the 'sign-up' option on the main menu to create some accounts and
view the leaderboard. (Press enter to return to main menu)" +
RESET_TEXT)
127
128 def login_menu(player_id):
129     '''Displays login menu'''
130
131     #SQL: Grab usernames and passwords
132     result = sql_select("SELECT usernames, passwords FROM user_details")
133     #create blank parallel arrays
134     valid_usernames = []
135     valid_passwords = []
136
137     #if there are entries...
138     if result != None:
139         #iterate through each tuple...
140         for i in range(len(result)):
141             #append values to parallel arrays
142             valid_usernames.append(result[i][0])
143             valid_passwords.append(result[i][1])
144
145     ##Remove usernames already logged in
146     #for both records in PLAYERS array...
147     for record in PLAYERS:
148         #check if they are logged in...
149         if record.logged_in:
150             #remove from list of valid names and password
151             valid_usernames.remove(record.username)
152             valid_passwords.remove(record.password)
153
154     cls() #clearscreen
155     print(f"\n{BLUE_TEXT}- Player {player_id} Login -" + RESET_TEXT + "\nLeave blank
to exit menu.\n") #instructions for logging in
156
157     in_progress = True #default to true
158
159     while in_progress: #unless told otherwise from a blank input
160         username = input(f"Enter Username > ") #get username
161         if username == "": #exit to main menu if blank input
162             in_progress = False
163             break
164         elif not username in valid_usernames: #if invalid details or already logged in
165             input(RED_TEXT + "User not found or is already logged in... Press enter
to retry." + RESET_TEXT) #error message
166         else: #if valid
167             #grab index of username for checking against password
168             ref_index = valid_usernames.index(username)
169             break #exit while loop as valid input
170
171     while in_progress: #unless told otherwise from a blank input
172         password = input(f"Enter Password > ") #get password
173         if password == "": #exit to main menu if blank input
174             in_progress = False
175             break
176         elif password in valid_passwords: #if password exists
177             if password == valid_passwords[ref_index]: #if password matches
178                 break #exit while loop asking for user to re-enter
179             input(RED_TEXT + "Incorrect password... Press enter to retry." + RESET_TEXT)
            #re-enter if invalid password and repeat
180
181     if in_progress: #if both details are valid
182         PLAYERS[player_id-1] = Player(username, password, True) #update global
            PLAYERS record
183         input(GREEN_TEXT + "Login successful. Press enter to return to main menu." +
            RESET_TEXT) #success message
184     else: #cancel message if blank input was detected
185         input(RED_TEXT + "Login cancelled. Press enter to return to main menu." +
            RESET_TEXT) #exiting message
186
187 def signup_menu():
188     '''Display signup menu'''
189     ##Get list of usernames from database

```



```

190 result = sql_select("SELECT usernames FROM user_details") #get list of usernames
    in_database
191 valid_usernames = [] #blank list
192 if result != None: #if results found
193     for i in range(len(result)): #for each username...
194         valid_usernames.append(result[i][0]) #append to list for comparing later
195
196 cls() #clear screen
197 print(BLUE_TEXT + "\n- Sign-up Menu -" + RESET_TEXT + "\n(leave blank and press
    enter to return to main menu)\n") #signup instructions
198
199 in_progress = True #default to true
200
201 '''Get Valid Username'''
202 while in_progress: #unless told to exit back to menu...
203     username = input("Enter Username (6-16 characters) > ") #get username
204     if username == "": #exit to main menu if blank input
205         in_progress = False
206         break
207     elif username in valid_usernames: #if username exists give error
208         input(RED_TEXT + "Invalid! This username is already in use. Please log
            in or choose a new username. Press enter to retry." + RESET_TEXT)
209     elif check_length_input(6, 16, username) == False: #if username not within
        length range, give error
210         input(RED_TEXT + "Invalid! Username must be between 6-16 characters.
            Press enter to try again." + RESET_TEXT)
211     else: #if valid
212         break #leave while loop
213
214 '''Get Valid Password'''
215 while in_progress: #unless told to exit back to menu
216     password = input("Enter Password (6-16 characters) > ") #get password
217     if password == "": #exit to main menu if blank input
218         in_progress = False
219         break
220     elif check_length_input(6, 16, password) == False: #if not within length
        range, give error
221         input(RED_TEXT + "Invalid! Password must be between 6-16 characters.
            Press enter to try again." + RESET_TEXT)
222     else: #if valid
223         break #leave while loop
224
225 if in_progress: #if username and password value
226     sql_commit(f"INSERT INTO user_details (usernames, passwords, wins, losses,
        draws) VALUES ('{username}', '{password}', 0, 0, 0)") #add user to database
227     input(GREEN_TEXT + "Sign-up successful. Press enter to return to main menu."
        + RESET_TEXT) #success message
228 else: # cancel message if blank input
229     input(RED_TEXT + "Sign-up cancelled. Press enter to return to main menu." +
        RESET_TEXT)
230
231 def sql_commit(sql):
232     '''Run SQL statement that updates database'''
233     CURSOR.execute(sql)
234     DB.commit()
235
236 def sql_select(sql):
237     '''Run a fetch SQL statement'''
238     CURSOR.execute(sql)
239     return CURSOR.fetchall()
240
241 def check_length_input(min, max, user_input):
242     '''Function checks if input length is in range'''
243     if min <= len(user_input) and len(user_input) <= max: #check in specified range
244         return True #if in range return True
245     else:
246         return False #if not in range return False
247
248 def connect_db():
249     '''Procedure connects to the database'''
250     ##Database connection credentials
251     mysql_user = "root"

```

```

252     mysql_password = "coolpassword"
253
254     #setup global database variables
255     global DB
256     global CURSOR
257
258     try:
259         #Checks details are correct
260         DB = mysql.connector.connect(
261             host='localhost', user=mysql_user, passwd=mysql_password)
262     except mysql.connector.Error as err:
263         #If details are incorrect
264         print("Error with connection: {}".format(err))
265         input("Incorrect details or MySQL not setup correctly. Exiting program...")
266         quit()
267     else:
268         #Try connect
269         #Connect to SQL with details
270         DB = mysql.connector.connect(
271             host='localhost', user=mysql_user, passwd=mysql_password)
272         #create database if not existent
273         CURSOR = DB.cursor()
274         #create database if not existent
275         CURSOR.execute("CREATE DATABASE IF NOT EXISTS stg_connect4")
276
277         #Now connect SPECIFICALLY to database that we now know exists
278         DB = mysql.connector.connect(
279             host='localhost', user=mysql_user, passwd=mysql_password,
280             database="stg_connect4")
281         CURSOR = DB.cursor(buffered=True)
282
283         #DEBUG
284         #CURSOR.execute("DROP TABLE user_details")
285
286         #Create the table if not existent
287         CURSOR.execute("CREATE TABLE IF NOT EXISTS user_details (usernames
288             varChar(16) PRIMARY KEY, passwords varChar(16), wins int, losses int, draws
289             int)")
290
291     def menu():
292         '''Display main menu'''
293         while True: #continually run program until quit
294             cls() #clear screen
295             print(ASCII_MAIN_MENU) #Main Menu ASCII art
296
297             '''Display Login Status'''
298             for index, record in enumerate(PLAYERS): #for each player
299                 if record.logged_in == False: #if not logged in message
300                     print(f"Player {index+1}: " + RED_TEXT + "NOT LOGGED IN" + RESET_TEXT)
301                 else: #logged in message
302                     print(f"Player {index+1}: " + GREEN_TEXT + record.username +
303                         RESET_TEXT)
304
305             print("") #new line for neatness
306
307             '''Login/Logout Messages'''
308             for index, record in enumerate(PLAYERS): #for each player
309                 if record.logged_in == True: #if logged in message
310                     print(f" [{index+1}] Log-out '{record.username}'")
311                 else: #if logged out in message
312                     print(f" [{index+1}] Log-in {BLUE_TEXT}Player {index+1}\033[0;0m")
313
314             print(" [3] Sign-up") #display option 3
315
316             logged_in_players = count_players() #count players logged in
317
318             if logged_in_players >= 2: #show 2 players logged in in GREEN (indicating
319                 you can start game)
320                 print(f" [4] Play Game! \u001B[32m[{logged_in_players}/2 players logged
321                     in]\033[0;0m")
322             else: #display how many people logged in in RED (indicating you cannot start
323                 game yet)

```



```

317         print(f" [4] Play Game! \u001b[0;31;40m[{logged_in_players}/2 players
          logged in]\u033[0;0m")
318
319     print(" [5] View Leaderboard") #display option 5
320
321     menu_input = get_ranged_number_input(1, 5) #get valid menu input
322
323     if menu_input == None: #quit game if blank
324         quit()
325     if menu_input >= 1 and menu_input <= 2: #if login selected
326         if PLAYERS[menu_input-1].logged_in == False: #if player not logged in
327             login_menu(menu_input) #open login menu for appropriate player
328         else: #if player logged in
329             PLAYERS[menu_input-1] = Player() #logout appropriate player
330
331     elif menu_input == 3: #if signup selected...
332         signup_menu() #open signup menu
333
334     elif menu_input == 4:
335         if PLAYERS[0].logged_in == True and PLAYERS[1].logged_in == True: #if 2
          player logged in start game
336             play_game()
337         else: #give error if not enough players logged in
338             input(RED_TEXT + "Error: There are not 2 unique players logged in.
          Press enter to retry." + RESET_TEXT)
339
340     elif menu_input == 5: #if leaderboard selected
341         leaderboard_menu() #open leaderboard menu
342
343     '''Conect 4 Board'''
344     def draw_window(board, turn_id, game_font, hover_column, win_state):
345         '''Draw pygame window'''
346         turn_id += 1 #make 1/2 instead of 0/1
347
348         WIN.fill(BLACK) #make background black by default
349         for c in range(COLUMNS): #for each column
350             if c == hover_column: #if hovered over
351                 if turn_id == 1: #draw red counter at top of screen
352                     pygame.draw.circle(WIN, RED,
          (c*SQUARE_DIMENSIONS+(SQUARE_DIMENSIONS/2), SQUARE_DIMENSIONS/2),
          int(CIRCLE_RADIUS)-4)
353                 else: #draw yellow counter at top of screen
354                     pygame.draw.circle(WIN, YELLOW,
          (c*SQUARE_DIMENSIONS+(SQUARE_DIMENSIONS/2), SQUARE_DIMENSIONS/2),
          int(CIRCLE_RADIUS)-4)
355         for r in range(ROWS): #for each row
356             if c == hover_column: #if in hovered column
357                 pygame.draw.rect(WIN, SELECT_COLOUR, (c*SQUARE_DIMENSIONS,
          (r*SQUARE_DIMENSIONS) + SQUARE_DIMENSIONS, SQUARE_DIMENSIONS,
          SQUARE_DIMENSIONS)) #draw square of highlight colour
358
359             else: #else if not in hovered column
360                 pygame.draw.rect(WIN, BLUE, (c*SQUARE_DIMENSIONS,
          (r*SQUARE_DIMENSIONS) + SQUARE_DIMENSIONS, SQUARE_DIMENSIONS,
          SQUARE_DIMENSIONS)) #draw square of board colour
361
362         ##This code draws the circles required to make the grid
363         if board[r][c] == 1: #if player 1 slot
364             pygame.draw.circle(WIN, RED,
          (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
          r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)),
          int(CIRCLE_RADIUS)) #draw red counter circle
365         elif board[r][c] == 2: #if player 2 slot
366             pygame.draw.circle(WIN, YELLOW,
          (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
          r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)),
          int(CIRCLE_RADIUS)) #draw yellow counter circle
367         else:
368             #if unoccupied, draw white circle (to show a blank spot)
369             pygame.draw.circle(WIN, WHITE,
          (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
          r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)),

```

```

        int(CIRCLE_RADIUS))
373     #draw black outline circle over red, yellow, or white circle just drawn
374     pygame.draw.circle(WIN, BLACK,
        (int(c*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2), int(
375         r*SQUARE_DIMENSIONS+SQUARE_DIMENSIONS+SQUARE_DIMENSIONS/2)),
        int(CIRCLE_RADIUS), 4)
376
377     #add dark blue line on bottom of board for style
378     pygame.draw.rect(WIN, DARK_BLUE, (c*SQUARE_DIMENSIONS,
379         (7*SQUARE_DIMENSIONS), SQUARE_DIMENSIONS,
        SQUARE_DIMENSIONS/6))
380
381     if win_state == -1: #if game hasn't been won
382         if turn_id == 1: #if player 1 turn
383             label = game_font.render(f"{PLAYERS[0].username}'s Turn", 1, RED) #set
            text to player 1's name
384         else: #if player 2 turn
385             label = game_font.render(f"{PLAYERS[1].username}'s Turn", 1, YELLOW)
            #set text to player 2's name
386             text_center = label.get_rect(center=(WIDTH/2,
            HEIGHT-(SQUARE_DIMENSIONS/2.3))) #position text
387             WIN.blit(label, text_center) #draw text
388
389     elif win_state == 0: #if draw
390         label = game_font.render(f"IT'S A DRAW!", 1, WHITE) #set text to draw message
391         text_center = label.get_rect(center=(WIDTH/2, SQUARE_DIMENSIONS/2.3))
            #position text
392         WIN.blit(label, text_center) #draw text
393
394         label = game_font.render(f"Click to return to menu...", 1, WHITE) #set text
            to click to return message
395         text_center = label.get_rect(center=(WIDTH/2,
            HEIGHT-(SQUARE_DIMENSIONS/2.3))) #position text
396         WIN.blit(label, text_center) #draw close/return message
397
398     else: #if win
399         if turn_id == 1: #if player 1
400             label = game_font.render(f"{PLAYERS[0].username} WINS!", 1, RED) #set
            text to player 1 name
401         else: #if player 2
402             label = game_font.render(f"{PLAYERS[1].username} WINS!", 1, YELLOW) #set
            text to player 2 name
403             text_center = label.get_rect(center=(WIDTH/2, SQUARE_DIMENSIONS/2.3))
            #position text
404             WIN.blit(label, text_center) #draw win message on screen
405
406             label = game_font.render(f"Click to return to menu...", 1, WHITE) #set text
            to click to return message
407             text_center = label.get_rect(center=(WIDTH/2,
            HEIGHT-(SQUARE_DIMENSIONS/2.3))) #position text
408             WIN.blit(label, text_center) #draw close/return message
409
410     pygame.display.update() #actually update the connect 4 window with what has been
        drawn
411
412     def play_game():
413         '''Initiate instance of Connect 4'''
414
415         board = [[0 for c in range(7)] for r in range(6)] #generate board
416         turns = 0 #set default turns counter
417         turn_id = random.randint(0, 1) #choose which player to start with
418
419         hover_column = -1 #default column to highlight (None)
420
421         ##PYGAME Bootup Variables
422         global WIN #create global window variable
423         WIN = pygame.display.set_mode([WIDTH, HEIGHT]) #set global variable to window
424         pygame.display.set_caption("Connect 4 By Scott Glidden") #set window title
425         clock = pygame.time.Clock() #initialise pygame clock
426         pygame.font.init() #initialise fonts
427         game_font = pygame.font.SysFont("Arial Rounded MT Bold", 65) #set font
428

```

```

429 win_state = -1 #default win state - i.e game in progress no winners
430
431 #console success message
432 print(GREEN_TEXT + "\nConnect 4 window has opened! \nSelect the window and left
click the column you'd like to place your counter in. \nGood luck!" +
RESET_TEXT) #instruction message
433
434 run = True #default to true
435 while run: #ensures loop runs until told otherwise
436     pygame.event.pump() #doesn't do anything to the screen, but is required to
ensure the window won't stop responding if window left alone
437     clock.tick(FPS) #ensures the while loop runs 24 times a second (every
exactly 1/24 of a second)
438
439     if win_state == -1: #if game not won
440         for event in pygame.event.get(): #fetch python events so they can be
accessed below
441             if event.type == pygame.QUIT: #if close button clicked
442                 run = False #stop while loop
443                 pygame.quit() #quit pygame
444             if event.type == pygame.WINDOWFOCUSLOST or event.type ==
pygame.WINDOWLEAVE: #if mouse leaves window
445                 hover_column = -1 #reset hover column
446             if event.type == pygame.MOUSEMOTION: #if mouse has moved
447                 hover_column = int(event.pos[0]/SQUARE_DIMENSIONS) #computing
which segment of screen it is in (which column)
448             if event.type == pygame.MOUSEBUTTONDOWN: #if click
449                 click_column = int(event.pos[0]/SQUARE_DIMENSIONS) #calculate
clicked column
450                 if click_column >= 0 and click_column <= 6 and
board[0][click_column] == 0: #if counter can be dropped (i.e
column isn't full)
451
452                     drop_counter(board, click_column, turn_id) #drop counter in
column selected
453                     turns += 1 #increment turn counter
454
455                     win_state = check_win(board, turn_id, turns) #check for win
after new counter dropped
456
457                     if win_state == 1: #win_state: 1 > Win
458                         #+ win score for winner
459                         sql_commit(f"UPDATE user_details SET wins = wins + 1
WHERE usernames = '{PLAYERS[turn_id].username}'")
460                         #+ loss score for loser
461                         sql_commit(f"UPDATE user_details SET losses = losses + 1
WHERE usernames = '{PLAYERS[1-turn_id].username}'")
462                     elif win_state == 0: #win_state: 0 > Draw | +draw for both
players
463                         sql_commit(f"UPDATE user_details SET draws = draws + 1
WHERE usernames = '{PLAYERS[0].username}' OR usernames =
'{PLAYERS[1].username}'")
464                     else: #win_state: -1 > Continue game as no draw, no win
465                         turn_id = 1 - turn_id #flip 0 and 1 - i.e change which
players turn it is
466
467         else:
468             hover_column = -1 #reset hover column
469             for event in pygame.event.get(): #get python events so they can be used
below
470                 if event.type == pygame.MOUSEBUTTONDOWN: #check for mouse click when
program on game-ended screen
471                     run = False #stop while loop
472                     pygame.display.quit() #shut down window
473                     pygame.quit() #quit pygame
474             if run: #if the game hasn't ended - draw board on screen
475                 draw_window(board, turn_id, game_font, hover_column, win_state)
476
477 def drop_counter(board, selected_column, drop_value):
478     '''Function drops counter in board'''
479     drop_value += 1 #make 1 and 2 rather than 0 and 1 for easier comparison with
drop_value
480     for row in range(5, -1, -1): #for each row from 5 to 0 (inclusive)

```

```

480         if board[row][selected_column] == 0: #if column is not full...
481             board[row][selected_column] = drop_value #update 2d board array with
482                 newly dropped counter
483             return #exit function when counter has been dropped
484
485 def check_win(board, ref, turns):
486     '''Function checks for win on board'''
487     #win_state: 1 > Win
488     #win_state: 0 > Draw
489     #win_state: -1 > Continue game
490
491     ref += 1 #make 1 and 2 rather than 0 and 1
492
493     ## The functions below all iterate through the board checking every possible
494     ## position for a 4 in the row ONLY for the player who has just dropped a counter
495     ## This is as only the player who has just dropped a counter can win in that turn
496     '''Check Horizontal Positions'''
497     for r in range(ROWS):
498         for c in range(COLUMNS-3):
499             if ref == board[r][c] == board[r][c+1] == board[r][c+2] == board[r][c+3]:
500                 return 1 #return player won value
501
502     '''Verticle'''
503     for r in range(ROWS-3):
504         for c in range(COLUMNS):
505             if ref == board[r][c] == board[r+1][c] == board[r+2][c] == board[r+3][c]:
506                 return 1 #return player won value
507
508     '''Diagonal 1'''
509     for r in range(ROWS-3):
510         for c in range(COLUMNS-3):
511             if ref == board[r][c] == board[r+1][c+1] == board[r+2][c+2] ==
512                 board[r+3][c+3]:
513                 return 1 #return player won value
514
515     '''Diagonal 2'''
516     for r in range(3, ROWS):
517         for c in range(COLUMNS-3):
518             if ref == board[r][c] == board[r-1][c+1] == board[r-2][c+2] ==
519                 board[r-3][c+3]:
520                 return 1 #return player won value
521
522     ##if nothing is returned by this point nobody has won so:
523     if turns >= 42: #if board full (draw)
524         return 0 #return draw value
525     else:
526         return -1 #return continue game value
527
528 ##PYGAME CONSTANTS
529 SQUARE_DIMENSIONS = 100
530 CIRCLE_RADIUS = (SQUARE_DIMENSIONS/2) - 8
531 COLUMNS = 7
532 ROWS = 6
533 WIDTH = SQUARE_DIMENSIONS * 7
534 HEIGHT = SQUARE_DIMENSIONS * 8 #8 not 6 for the space at top and bottom
535
536 ##PYGAME COLOUR CONSTANT TUPLES
537 WHITE = (255, 255, 255)
538 BLUE = (0, 147, 255)
539 DARK_BLUE = (0, 0, 255)
540 YELLOW = (255, 255, 0)
541 RED = (255, 0, 0)
542 BLACK = (0, 0, 0)
543 SELECT_COLOUR = (102, 190, 255)
544 FPS = 24
545
546 ##TEXT COLOUR CONSTANTS
547 RED_TEXT = "\u001b[0;31;40m"
548 BLUE_TEXT = "\u001b[36m"
549 GREEN_TEXT = "\u001b[32m"
550 RESET_TEXT = "\033[0;0m"

```

```
548  ##ASCII ART
549  ASCII_MAIN_MENU = '''
550  | \ / |   ( )   _ _   | \ / |   _ _   |
551  | \ / |   ( )   _ _   | \ / |   _ _   |
552  | \ / |   ( )   _ _   | \ / |   _ _   |
553  | \ / |   ( )   _ _   | \ / |   _ _   |
554  | \ / |   ( )   _ _   | \ / |   _ _   |
555  , , ,
556  # https://patorjk.com/software/taag/#p=display&f=Big&t=Sign%20-%20up%20%20%20Menu
557
558  #setup array of records
559  PLAYERS = [Player()] * 2
560
561  '''Main Program'''
562  connect_db()
563  menu()
564
```

Program User Interface

Here is evidence of each menu in the game being printed as detailed at the design stage - the next section “Project Testing” will cover the menu in much more detail.

[Main Menu](#)

Main Menu

Player 1: NOT LOGGED IN

Player 2: NOT LOGGED IN

```
[1] Log-in Player 1
[2] Log-in Player 2
[3] Sign-up
[4] Play Game! [0/2 players logged in]
[5] View Leaderboard
```

Enter number from above and press enter to select.

Leave blank to exit menu.

Sign-up Menu

- Sign-up Menu -

```
(leave blank and press enter to return to main menu)
```

```
Enter Username (6-16 characters) > 1
```

Invalid! Username must be between 6-16 characters. Press enter to try again.

```
Enter Username (6-16 characters) > hello
```

Invalid! Username must be between 6-16 characters. Press enter to try again.

```
Enter Username (6-16 characters) > 123456
```

```
Enter Password (6-16 characters) > 1
```

Invalid! Password must be between 6-16 characters. Press enter to try again.

```
Enter Password (6-16 characters) > hello
```

Invalid! Password must be between 6-16 characters. Press enter to try again.

```
Enter Password (6-16 characters) > 123456
```

Sign-up successful. Press enter to return to main menu.

Log-in Menu

- Player 1 Login -

Leave blank to exit menu.

```
Enter Username > hello
```

```
User not found or is already logged in... Press enter to retry.
```

```
Enter Username > clifford
```

```
Enter Password > hello
```

```
Incorrect password... Press enter to retry.
```

```
Enter Password > 123456
```

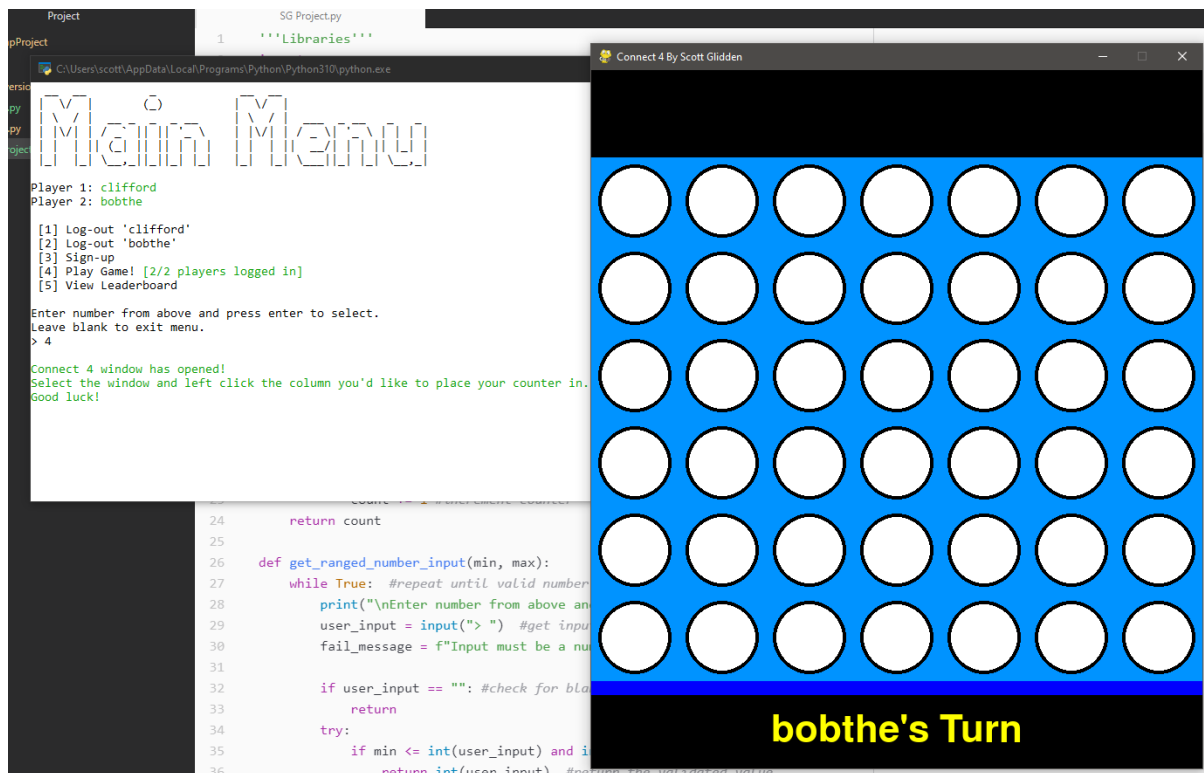
```
Enter Password > 123456
Incorrect password... Press enter to retry.
```

Enter Password > thereddog

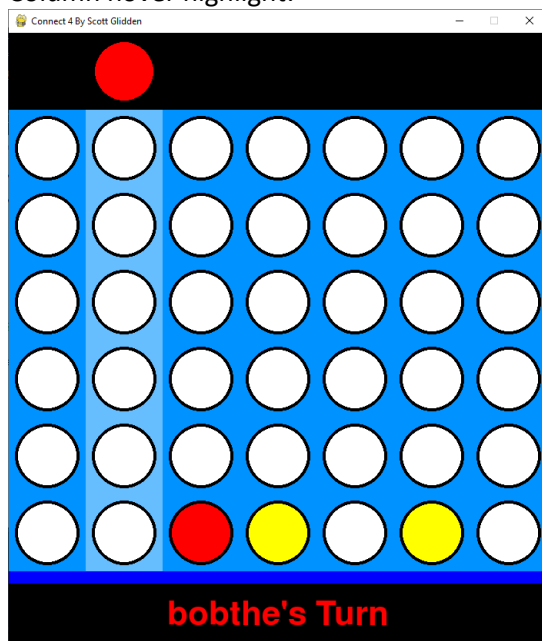
```
Enter Password > thereddog
Login successful. Press enter to return to main menu.
```


Connect 4 Board

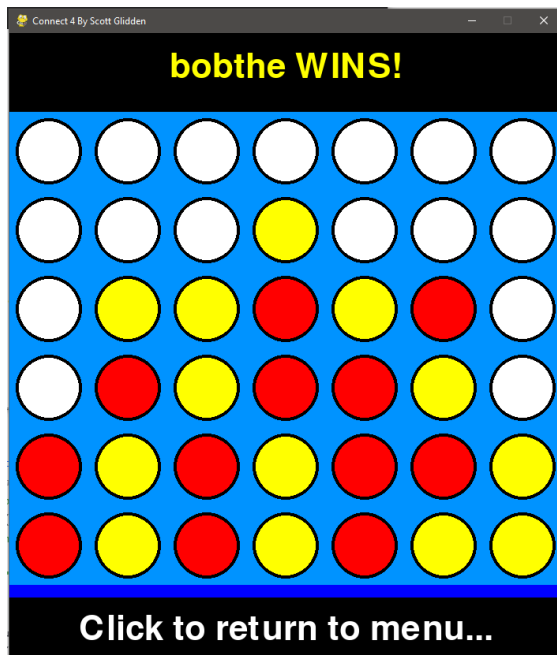
Menu opens:



Column hover highlight:



Win detection and screen:



Leaderboard Menu

- Leaderboard Menu -

- [1] Sort by wins
- [2] Sort by losses
- [3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.

> 1

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	14	4
bobthe	8	4	2
sglidden	5	6	1
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

(press enter to return to leaderboard menu)

```
- Leaderboard Menu -
[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
> 2

Showing leaderboard sorted by losses:
| Username | Wins | Losses | Draws |
| clifford | 15 | 14 | 4 |
| fraser | 1 | 6 | 1 |
| sglidden | 5 | 6 | 1 |
| bobthe | 8 | 4 | 2 |
| orangeeater | 0 | 1 | 0 |
| 123456 | 0 | 0 | 0 |
| 123456789012345678901234 | 0 | 0 | 0 |
| 246810 | 0 | 0 | 0 |
| coolusername | 1 | 0 | 0 |
| sglidden2 | 0 | 0 | 0 |
| teigs2408 | 0 | 0 | 0 |
| test me | 0 | 0 | 0 |
| Vamperina | 1 | 0 | 0 |

(press enter to return to leaderboard menu) _
```

- Leaderboard Menu -

- [1] Sort by wins
- [2] Sort by losses
- [3] Sort by draws

Enter number from above and press enter to select.

Leave blank to exit menu.

> 3

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	14	4
bobthe	8	4	2
fraser	1	6	1
sglidden	5	6	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu) _

Database Entries

Table "user_details" viewed in MySQL Workbench:

Query 1 stg_connect4 user_details								
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants	DDL	
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges	Extra	
draws	int		YES			select,insert,update,references		
losses	int		YES			select,insert,update,references		
passwords	varchar(24)		YES	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
usernames	varchar(24)		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
wins	int		YES			select,insert,update,references		

Overview of table "user_details" with entries viewed in MySQL workbench:

Query 1 x stg_connect4.user_details stg_connect4 stg_connect4.user_details

Limit to 1000 rows

```

1 select *
2 from stg_connect4.user_details

```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:

	usernames	passwords	wins	losses	draws
	123456	123456	0	0	0
	123456789012345678901234	123456789012345678901234	0	0	0
	246810	47575845785	0	0	0
	bobthe	builder	10	4	2
	clifford	thereddog	15	16	4
	coolusername	coolpassword	1	0	0
	fraser	123456	1	6	1
	orangeeater	151104keeks	0	1	0
	sglidden	123456	5	6	1
	sglidden2	123456	0	0	0
	teigs2408	123456	0	0	0
	test me	testme	0	0	0
	Vamperina	thewitch	1	0	0
*	NULL	NULL	NULL	NULL	NULL

Project Testing

Test Plan

These test plan scenarios detailed below will cover all functional and end-user requirements detailed within the [Requirement Specification](#) from the Analysis stage of the project. The personas below will be used for some of the tests:

Persona 1 (aka Player 1 & Score Checker)

Credentials Of Persona:

Username: marktheshark
Password: inthesea

Traits:

- Adept with technology
- Familiar with Connect 4 rules
- Impatient
- Competitive
- Wants to check leaderboard to see how many they are beat\ing and what his goal is to beat

Persona 2 (aka Player 2)

Credentials Of Persona:

Username: sidthesloth Password: globalwarming
--

Traits:

- Adept with technology
- Familiar with Connect 4 rules
- Casual player (just looking to have fun)
- Unfussed by the scores, don't intend to look at leaderboard

These traits have been chosen as they contrast. This contrast will help convey the idea that the program is suitable for people with different attitudes towards Connect 4.

I will test each of the main modules of the program individually ensuring they behave as intended by checking all relevant requirements. This includes:

1. Main Menu
2. Leaderboard Menu
3. Sign-up Menu
4. Log-in Menu
5. Connect 4 Board

The specific test plan for each section will be detailed at the start of each section, along with evidence for the inputs/outputs being entered for these scenarios, as well as the results/outcome of the tests being detailed.

Note: In testing each of the above sections I will indirectly be testing every subroutine in the program, including code relevant to the advanced higher concepts and integration. If testing the above uncovers any errors, then the specific code related to the section will be delved into more detail for further testing. Otherwise, if the end result is correct and no errors have arisen, it will be assumed all code relating to the successful output is also correct.

[Testing: Main Menu](#)

Python Requirements Tested

Inputs	Processes	Outputs
Validated user menu navigation inputs	Validation of user inputs Open connection with database (on program start)	Text-based menus & submenus displayed based on input: - Main Menu - Sign-up Menu - Log-in Menu - Leaderboard Menu Display who is logged in on main-menu Number of users logged in on main-menu Validation error messages Quitting game

Database Requirements Tested

Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	X	X

End User Requirements Tested
Users must be able to easily navigate the menus

Test Script (meeting the above requirements):

1. Every menu will attempt to be navigated to and confirm the input has been validated and accepted and has had the correct outcome (appropriate submenu/message)
2. Enter extreme and exceptional data to ensure the validation filters incorrect inputs (negative testing).

The below tests happen to fulfil some of the requirements for the login system testing. The only outcome I'm interested in right now is the main menu updating the logged in player information as that is what is relevant to the main menu system. The login system requirements will be tested in detail later.

3. Log-in no players and check main menu logged in information accurate
4. Log-in Persona 1 with their details and check main menu logged in information accurate
5. Log-in Persona 2 with their details and check main menu logged in information accurate
6. Log-in Persona 1 and Persona 2 and check main menu logged in information accurate

In testing all the above, opening the connection with the database will be indirectly tested as if it cannot open the above tests are not possible. Assuming the above is successful, we can determine that the program has connected to the database.

User Input	Program Output	Outcome/Summary
------------	----------------	-----------------

Entering "1" on main menu: <pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 1_</pre>	<pre>- Player 1 Login - Leave blank to exit menu. Enter Username ></pre>	Pass. Accepted input "1" and opened the login submenu. Works as intended.
Entering "2" on main menu: <pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 2</pre>	<pre>- Player 2 Login - Leave blank to exit menu. Enter Username ></pre>	Pass. Accepted input "2" and opened the login submenu. Works as intended.

Entering "3" on main menu: <pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 3_</pre>	<pre> - Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) ></pre>	Pass. Accepted input "3" and opened the signup submenu. Works as intended.
Entering "4" on main menu: <pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 4_</pre>	<pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 4 Error: There are not 2 unique players logged in. Press enter to retry._</pre>	Pass. Accepted input "4" and recognised that the player wanted to start the game. It gives the correct error message to the user as there are no players logged in. This is an intended error when you try to play a game without 2 players logged in - so this test is still a success as the program recognised the input "4" and gave the correct error output. Therefore works as intended.

Entering "5" on main menu: <pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 5_ </pre>	<pre> - Leaderboard Menu - [1] Sort by wins [2] Sort by losses [3] Sort by draws Enter number from above and press enter to select. Leave blank to exit menu. > </pre>	Pass. Accepted input "5" and opened the leaderboard submenu. Works as intended.
Entering "" on main menu (blank input): <pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > </pre>	<pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > Process returned 0 (0x0) execution time : 5934.188 s Press any key to continue . . . </pre>	Pass. Accepted input "" and quit the python program (see the process return sentence at bottom of program output). Works as intended.

Trying extreme and exceptional data to ensure input validation rejects invalid inputs (inputted extreme and exceptional values and the output of program visible below):

```

  0
M  a  M
i  n  i  m  e  n  u

```

```

Player 1: clifford
Player 2: bobthe

```

```

[1] Log-out 'clifford'
[2] Log-out 'bobthe'
[3] Sign-up
[4] Play Game! [2/2 players logged in]
[5] View Leaderboard

```

Enter number from above and press enter to select.

Leave blank to exit menu.

> one

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> ""

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> hello

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> 6

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> 0

Input must be a number between 1 and 5, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> -

Passed.

Extreme and exceptional data has been handled with correct error handling. All invalid values tried have been successfully rejected. Therefore, the program works as intended with correct error handling.

Just starting up the program, no logging in.	<pre> Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > </pre>	Pass. Main menu displays nobody as logged in as Player 1/Player 2 and displays 0/2 players logged in. Menu displays as intended.
Persona 1 logging in and exiting back to menu: - Player 1 Login - Leave blank to exit menu. Enter Username > marktheshark Enter Password > inthesea Login successful. Press enter to return to main menu.	<pre> Main Menu Player 1: marktheshark Player 2: NOT LOGGED IN [1] Log-out 'marktheshark' [2] Log-in Player 2 [3] Sign-up [4] Play Game! [1/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > </pre>	Pass. Main menu displays Player 1 logged in only and displays 1/2 players logged in. Menu displays as intended.

Persona 2 logging in and exiting back to menu: <pre>- Player 2 Login - Leave blank to exit menu. Enter Username > sidthesloth Enter Password > globalwarming Login successful. Press enter to return to main menu.</pre>		Pass. Main menu displays Player 2 logged in only and displays 1/2 players logged in. Menu displays as intended.
Persona 1 logging in and exiting back to menu, then Persona 2 logging in and exiting back to menu: <pre>- Player 1 Login - Leave blank to exit menu. Enter Username > marktheshark Enter Password > inthesea Login successful. Press enter to return to main menu. - Player 2 Login - Leave blank to exit menu. Enter Username > sidthesloth Enter Password > globalwarming Login successful. Press enter to return to main menu.</pre>		Pass. Main menu displays Player 1&2 logged in and displays 2/2 players logged in. Menu displays as intended.
End User Requirement Outcomes From Above Testing: As the navigation of the main menu successfully works as intended with simple number inputs, the end-user requirement of “Users must be able to easily navigate the menu” is fulfilled. Testing Summary: The above evidence shows that the program successfully meets all the stated requirements set out.		

[Testing: Leaderboard Submenu](#)**Python Requirements Tested**

Inputs	Processes	Outputs
Validated user menu navigation inputs Leaderboard query results	Validation of user inputs Sort “win” sql results	Sorted leaderboard table based on input Return to main menu Validation error messages

Database Requirements Tested

Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	Sort “losses” and “draws” sql results	Leaderboard query results

End User Requirements Tested

Users must be able to easily navigate the menus
Users must be able to choose between the leaderboard being shown sorted by wins, losses, or draws

Test Script:

- Enter appropriate input for each of the leaderboard menu options (1-3 or a blank input) and confirm the input has been validated and accepted. Then enter extreme and exceptional data to ensure the validation filters incorrect inputs (negative testing).
- Attempt to open the leaderboard menu when the table is empty.

For reference here is the leaderboard menu unsorted (this is referred to to ensure the sorted values correspond to the correct user):

Username	Wins	Losses	Draws
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
bobthe	10	4	2
clifford	15	16	4
coolusername	1	0	0
fraser	1	6	1
orangeeater	0	1	0
sglidden	5	6	1
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

In testing all these steps, we will also be ensuring the relevant SQL statements in the program have worked correctly. If they haven't - we will see errors in the following testing.

User Input	Program Output	Outcome
------------	----------------	---------

User inputting "1" on leaderboard submenu:

- Leaderboard Menu -

[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
> 1

- Leaderboard Menu -

[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
> 1

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

(press enter to return to leaderboard menu)

Pass.

The program has accepted input "1" and program has printed formatted leaderboard sorted (via bubblesort) by wins correctly (15, 10, 5, 1, 1, 1, 0, 0 etc) with the numbers corresponding to the correct player. The message is also correctly displaying "sorted by wins". Therefore works as intended.

User inputting "2" on leaderboard submenu:

- Leaderboard Menu -

- [1] Sort by wins
- [2] Sort by losses
- [3] Sort by draws

Enter number from above and press enter to select.

Leave blank to exit menu.

> 2

- Leaderboard Menu -

- [1] Sort by wins
- [2] Sort by losses
- [3] Sort by draws

Enter number from above and press enter to select.

Leave blank to exit menu.

> 2

Showing leaderboard sorted by losses:

Username	Wins	Losses	Draws
clifford	15	16	4
fraser	1	6	1
sglidden	5	6	1
bobthe	10	4	2
orangeeater	0	1	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu) _

Pass.

The program has accepted input "2" and program has printed formatted leaderboard sorted by losses correctly (16, 6, 6, 4, 1, 0, 0, 0 etc) with the numbers corresponding to the correct player. The message is also correctly displaying "sorted by losses". Therefore works as intended.

User inputting "3" on leaderboard submenu:

```
- Leaderboard Menu -
```

```
[1] Sort by wins
[2] Sort by losses
[3] Sort by draws
```

```
Enter number from above and press enter to select.
Leave blank to exit menu.
> 3
```

```
- Leaderboard Menu -
```

```
[1] Sort by wins
[2] Sort by losses
[3] Sort by draws
```

```
Enter number from above and press enter to select.
Leave blank to exit menu.
> 3
```

```
Showing leaderboard sorted by draws:
```

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
fraser	1	6	1
sglidden	5	6	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

```
(press enter to return to leaderboard menu)
```

Pass.

The program has accepted input "3" and program has printed formatted leaderboard sorted by draws correctly (4, 2, 1, 1, 0, 0, 0 etc) with the numbers corresponding to the correct player. The message is also correctly displaying "sorted by draws". Therefore works as intended.

User inputting "" on leaderboard submenu (blank input): <pre>- Leaderboard Menu - [1] Sort by wins [2] Sort by losses [3] Sort by draws Enter number from above and press enter to select. Leave blank to exit menu. ></pre>		Pass. The program has accepted blank input and has taken user back to the main menu as intended. Works as intended.
Trying extreme and exceptional data to ensure input validation rejected invalid inputs (inputted extreme and exceptional values and the output of program visible below):		Passed. Extreme and exceptional data has been handled with correct error handling. All invalid values tried have been successfully rejected. Therefore, program works as intended.

- Leaderboard Menu -

[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.

Leave blank to exit menu.

> Hello

Input must be a number between 1 and 3, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> 0

Input must be a number between 1 and 3, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> 4

Input must be a number between 1 and 3, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> 4.5

Input must be a number between 1 and 3, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> ""

Input must be a number between 1 and 3, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

>

Input must be a number between 1 and 3, or blank to exit menu. Press enter to retry.

Enter number from above and press enter to select.

Leave blank to exit menu.

> h

Input must be a number between 1 and 3, or blank to exit menu. Press enter to retry.■

End User Requirement Outcomes From Above Testing:

- As the navigation of the leaderboard menu entirely works as intended with simple number inputs, the end-user requirement of “Users must be able to easily navigate the menu” is fulfilled.
- As the leaderboard can be sorted by wins/losses/draws successfully as proved above, the end-user requirement “Users must be able to choose between the leaderboard being shown sorted by wins, losses, or draws” is fulfilled.

Testing Summary: The above evidence shows that the program successfully meets all the stated requirements set out.

Testing: Signup Menu

Python Requirements Tested

Inputs	Processes	Outputs
Validated inputted usernames (6-16 characters) Validated inputted passwords (6-16 characters) Existing user details SQL results	Validation of user inputs (6-16 characters and ensuring user doesn't already exist using SQL results)	Returning to main menu Sign-up success message Validation error messages

Database Requirements Tested

Inputs (Forms)	Processes	Outputs
----------------	-----------	---------

	(Queries)	(Reports)
Inserting new account details (username and password)	X	Existing user details SQL results
End User Requirements Tested		
Users must be able to easily navigate the menus Users needs their accounts to be password protected		

Test Script:

1. Persona 1 and Persona 2 will enter their valid details to ensure an account is created as expected. The success of an account being made will be judged by the success message given and also checking if the new user has appeared in the leaderboard menu with all stats at 0 when displayed (to show that the SQL queries have successfully taken and stored the new information)
2. Persona 1 will enter a variety of invalid details to test if the input validation suggested in the requirements rejects invalid values. This will be done for both the username and password.
3. Persona 1 and Persona 2 will attempt to re-enter their details into the sign-up menu after the accounts have already been created to ensure duplicate accounts with same username are not created.
4. A blank input will be entered as the username, and as the password to exit back to main menu

In testing all these steps, we will also be ensuring the relevant SQL statements in the program have worked correctly. If they haven't - we will see errors in the following testing.

User Input	Program Output	Outcome
Persona 1 entering their details: - Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) > marktheshark Enter Password (6-16 characters) > inthesea	<pre> - Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) > marktheshark Enter Password (6-16 characters) > inthesea Sign-up successful. Press enter to return to main menu. </pre>	Pass. The program has accepted the valid details, displayed a success message, and the new account has showed up in the leaderboard. All stats have defaulted to 0. Works as intended.

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
marktheshark	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

(press enter to return to leaderboard menu)

Persona 2 entering their details:

```
- Sign-up Menu -
(leave blank and press enter to return to main menu)

Enter Username (6-16 characters) > sidthesloth
Enter Password (6-16 characters) > globalwarming
```

```
- Sign-up Menu -
(leave blank and press enter to return to main menu)

Enter Username (6-16 characters) > sidthesloth
Enter Password (6-16 characters) > globalwarming
Sign-up successful. Press enter to return to main menu.
```

Pass.

The program has accepted the valid details, displayed a success message, and the new account has showed up in the leaderboard. All stats have defaulted to 0. Works as intended.

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
marktheshark	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
sidthesloth	0	0	0
teigs2408	0	0	0
test me	0	0	0

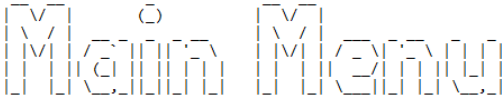
(press enter to return to leaderboard menu)

Persona 1 entering a variety of incorrect details to test inputs are rejected as they should be (note "validusername" is typed there on purpose to be a valid username so the password input validation could also be tested. This is as the password can only be typed after a valid username):

Passed.

Extreme and exceptional data has been handled with correct error handling. All invalid values tried have been successfully rejected. Therefore, program works as intended.

<pre> - Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) > 12345 Invalid! Username must be between 6-16 characters. Press enter to try again. Enter Username (6-16 characters) > 123456789123456789 Invalid! Username must be between 6-16 characters. Press enter to try again. Enter Username (6-16 characters) > hello Invalid! Username must be between 6-16 characters. Press enter to try again. Enter Username (6-16 characters) > "" Invalid! Username must be between 6-16 characters. Press enter to try again. Enter Username (6-16 characters) > validusername Enter Password (6-16 characters) > 12345 Invalid! Password must be between 6-16 characters. Press enter to try again. Enter Password (6-16 characters) > 123456789123456789 Invalid! Password must be between 6-16 characters. Press enter to try again. Enter Password (6-16 characters) > hello Invalid! Password must be between 6-16 characters. Press enter to try again. Enter Password (6-16 characters) > "" Invalid! Password must be between 6-16 characters. Press enter to try again. Enter Password (6-16 characters) > </pre>	
Persona 1 re-entering details after an account with their username already exists: <pre> - Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) > marktheshark Invalid! This username is already in use. Please log in or choose a new username. Press enter to retry. </pre>	Passed. The already existing username has been rejected. Works as intended.
Persona 2 re-entering details after an account with their password already exists (this time after an invalid value has been detected first):	Passed. The already existing username has been rejected . Works as

<pre>- Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) > marktheshark Invalid! This username is already in use. Please log in or choose a new username. Press enter to retry. Enter Username (6-16 characters) > sidthesloth Invalid! This username is already in use. Please log in or choose a new username. Press enter to retry.</pre>		intended.
User putting blank input at the username and password stage: Username Stage: <pre>- Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) > Sign-up cancelled. Press enter to return to main menu.</pre> Password Stage: <pre>- Sign-up Menu - (leave blank and press enter to return to main menu) Enter Username (6-16 characters) > validusername Enter Password (6-16 characters) > Sign-up cancelled. Press enter to return to main menu._</pre>	(both exit back to this main menu screen) <pre>Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > _</pre>	Passed. The user is brought back to the main menu upon a blank input, cancelling the signup process)
End User Requirement Outcomes From Above Testing: - As the navigation of the sign-up menu clearly prompts the user of how to progress through or exit menu, the end-user requirement of “Users must be able to easily navigate the menu” is fulfilled. - As the above system requires users to have a password with their account the end-user requirement “Users needs their accounts to be password protected” is on track to being fulfilled. Log-in testing is required before I can confidently say that the accounts are all successfully password protected.		
Testing Summary: The above evidence shows that the program successfully meets all the stated requirements set out.		

Testing: Log-in Menu

Requirements:

Python Requirements Tested

Inputs	Processes	Outputs
Validated inputted usernames and passwords which already exist in database Existing user details SQL results	Validation of user inputs (via checking entered details exist in database)	Returning to main menu Log-in success message Validation error messages

Database Requirements Tested

Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	X	Existing user details SQL results

End User Requirements Tested

Users must be able to easily navigate the menus
Users needs their accounts to be password protected

Test Script:

1. Log-in no players and check main menu logged in information accurate
2. Log-in Persona 1 with their details and check main menu logged in information accurate

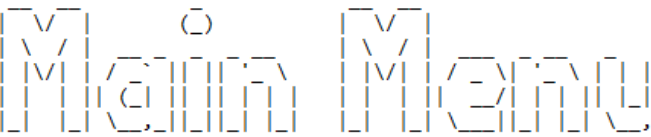
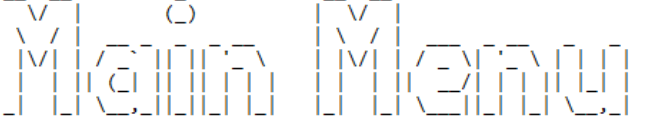
3. Log-in Persona 2 with their details and check main menu logged in information accurate
4. Log-in Persona 1 and Persona 2 and check main menu logged in information accurate (then another test to ensure both can be logged out)

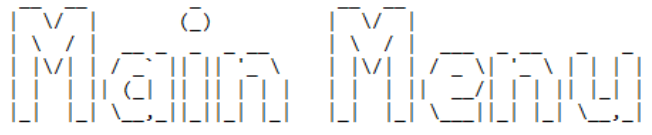
^^^ **NOTE:** These above tests overlap with some of the Main Menu testing, but have been included here too for clarity.

5. Log-in Persona 1 as player 1 and attempt to login as player 2 using Persona 1's details to ensure the same player can't be logged in as both players
6. Persona 1 will attempt to log-in with a collection of invalid details to ensure the validation doesn't wrongly accept incorrect values.

In testing all these steps, we will also be ensuring the relevant SQL statements in the program have worked correctly. If they haven't - we will see errors in the following testing.

User Input	Program Output	Outcome
Just starting up the program, no logging in.	<pre> O Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. ></pre>	Pass. Player 1 and Player 2 are correctly listed as not logged in - and therefore option 1 and 2 are giving the user the option to login. 0/2 players are listed as logged in. Login system works as intended.

Persona 1 logging in and exiting back to menu: - Player 1 Login - Leave blank to exit menu. Enter Username > marktheshark Enter Password > inthesea Login successful. Press enter to return to main menu.■	 Player 1: marktheshark Player 2: NOT LOGGED IN [1] Log-out 'marktheshark' [2] Log-in Player 2 [3] Sign-up [4] Play Game! [1/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. >	Pass. Player 1 is listed as logged in with Persona 1 username, Player 2 is listed as not logged in. This is correct. Therefore program option 1 is giving the user the option to log out as player 1, and log in as player 2. 1/2 players are listed as logged in. Login system works as intended.
Persona 1 logging in and exiting back to menu: - Player 2 Login - Leave blank to exit menu. Enter Username > sidthesloth Enter Password > globalwarming Login successful. Press enter to return to main menu.	 Player 1: NOT LOGGED IN Player 2: sidthesloth [1] Log-in Player 1 [2] Log-out 'sidthesloth' [3] Sign-up [4] Play Game! [1/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > ■	Pass. Player 2 is listed as logged in with Persona 2 username, Player 1 is listed as not logged in. This is correct. Therefore program option 2 is giving the user the option to log out as player 2, and log in as player 1. 1/2 players are listed as logged in. Login system works as intended.

Persona 1 logging in and exiting back to menu, then Persona 2 logging in and exiting back to menu: - Player 1 Login - Leave blank to exit menu. Enter Username > marktheshark Enter Password > inthesea Login successful. Press enter to return to main menu. - Player 2 Login - Leave blank to exit menu. Enter Username > sidthesloth Enter Password > globalwarming Login successful. Press enter to return to main menu.	 Player 1: marktheshark Player 2: sidthesloth [1] Log-out 'marktheshark' [2] Log-out 'sidthesloth' [3] Sign-up [4] Play Game! [2/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. >	Pass. Player 1&2 is listed as logged in with corresponding Persona 1 username and Persona 2 username. Both are listed as logged in. Therefore program option 1&2 is giving the user the option the log out as player 1 or player 2. 1/2 players are listed as logged in. Login system works as intended.
Expanding on the above test, logging out player 1 and player 2:  Player 1: marktheshark Player 2: sidthesloth [1] Log-out 'marktheshark' [2] Log-out 'sidthesloth' [3] Sign-up [4] Play Game! [2/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 1	 Player 1: NOT LOGGED IN Player 2: sidthesloth [1] Log-in Player 1 [2] Log-out 'sidthesloth' [3] Sign-up [4] Play Game! [1/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. >	Pass. When player 1 logs out, the main menu information is updated to reflect this. When player 2 logs out, the main menu information is updated to reflect there being no logged in players. Therefore the logging out works. So works as intended! Huzzah!

 <pre>Main Menu Player 1: NOT LOGGED IN Player 2: sidthesloth [1] Log-in Player 1 [2] Log-out 'sidthesloth' [3] Sign-up [4] Play Game! [1/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 2</pre>	 <pre>Main Menu Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 2</pre>	
Log-in persona 1 as player 1: <pre>- Player 1 Login - Leave blank to exit menu. Enter Username > marktheshark Enter Password > inthesea Login successful. Press enter to return to main menu.</pre> Then attempt to log-in persona 1 but as player 2: <pre>- Player 2 Login - Leave blank to exit menu. Enter Username > marktheshark</pre>	<pre>- Player 2 Login - Leave blank to exit menu. Enter Username > marktheshark User not found or is already logged in... Press enter to retry.</pre>	Pass. The already logged in user cannot log in again. Therefore, works as intended!
Persona 1 logging in with all sorts of invalid usernames, then typing correct username, then trying invalid passwords to further test input validation, before logging in with correct password:		Pass. All the invalid usernames and passwords have been rejected. This includes adding "" or typing other users passwords which don't correspond to the username. Therefore, works as intended.

- Player 1 Login -
Leave blank to exit menu.

```
Enter Username > 1
User not found or is already logged in... Press enter to retry.
Enter Username > Hello
User not found or is already logged in... Press enter to retry.
Enter Username > validusername
User not found or is already logged in... Press enter to retry.
Enter Username > "marktheshark"
User not found or is already logged in... Press enter to retry.
Enter Username > marktheshark
Enter Password > 1
Incorrect password... Press enter to retry.
Enter Password > Hello
Incorrect password... Press enter to retry.
Enter Password > validusername
Incorrect password... Press enter to retry.
Enter Password > marktheshark
Incorrect password... Press enter to retry.
Enter Password > sidthesloth
Incorrect password... Press enter to retry.
Enter Password > globalwarming
Incorrect password... Press enter to retry.
Enter Password > "inthesea"
Incorrect password... Press enter to retry.
Enter Password > inthesea
Login successful. Press enter to return to main menu.
```

Note: For this test I temporarily deleted all entries from the table (so I could test trying to access a blank leaderboard)
Attempt to open menu:

Pass.

The leaderboard has not opened as it is empty. Works as intended.

```
Main Menu
```

```
Player 1: NOT LOGGED IN
Player 2: NOT LOGGED IN
```

```
[1] Log-in Player 1
[2] Log-in Player 2
[3] Sign-up
[4] Play Game! [0/2 players logged in]
[5] View Leaderboard
```

```
Enter number from above and press enter to select.
Leave blank to exit menu.
```

```
> 5
```

```
There is no leaderboard to show as there are no signed up users yet! Sign-up in the main menu first.
(Press enter to return to main menu)
```

Upon clicking enter you successfully return to main menu:

```
Main Menu
```

```
Player 1: NOT LOGGED IN
Player 2: NOT LOGGED IN
```

```
[1] Log-in Player 1
[2] Log-in Player 2
[3] Sign-up
[4] Play Game! [0/2 players logged in]
[5] View Leaderboard
```

```
Enter number from above and press enter to select.
Leave blank to exit menu.
```

```
> 5
```

End User Requirement Outcomes From Above Testing:

- As the navigation of the log-in menu clearly prompts the user of how to progress through or exit menu, the end-user requirement of “Users must be able to easily navigate the menu” is fulfilled.
- As the above system successfully ensures the correct password for the user is logged in, the end-user requirement “Users needs their accounts to be password protected” has been fulfilled.

Testing Summary: The above evidence shows that the program successfully meets all the stated requirements set out.

Testing: Connect 4 Board

Requirements:**Python Requirements Tested**

Inputs	Processes	Outputs
Left clicking window	Check 2 players are logged in before game starts Validation of user inputs (left clicking) on the Connect 4 Board including: - Dropping counter where valid - Exiting to menu after game-ended screen Automatically detect wins and draws on Connect 4 board Randomise which players drops first counter	Successful starting game message Launching graphically displayed Connect 4 board Update following on the board window: - Highlighting columns when hovered - Displaying dropped counters dropped by player - Post-game win/draw screen Exiting back to main menu

Database Requirements Tested

Inputs (Forms)	Processes (Queries)	Outputs (Reports)
----------------	---------------------	-------------------

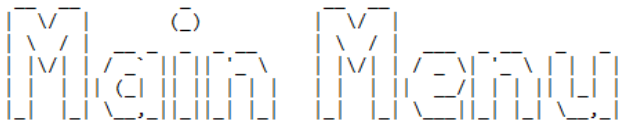
X	Update player statistics based on how the game ends (editing wins, losses, and draws for both logged in players)	X
End User Requirements Tested		
1. Users must drop a counter by clicking the wanted column – not typing in the number corresponding to the column 2. Users must be able to continue playing multiple games without needing to restart the program		

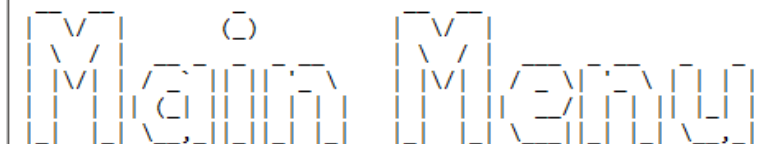
Test Script:

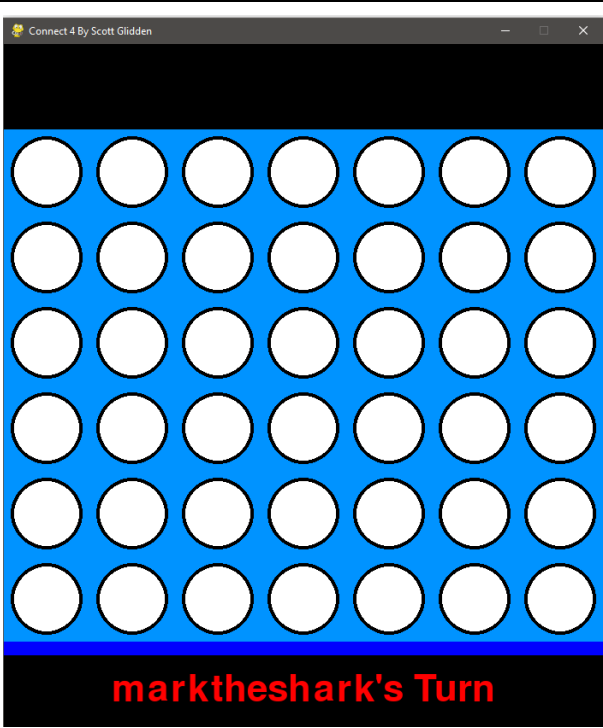
1. Attempt to start game with no players logged in
2. Attempt to start game with only 1 Persona 1 logged in
3. Start game with Persona 1 and Persona 2 both logged in (the only time it should be able to launch menu) with the console good luck message as well as the graphical display opening.
4. Evidence both Persona 1 and Persona 2 having the first turn of the game to show it is random
5. Hover over each column to ensure the correct column highlights with correct counter above
6. a
7. Check attempting dropping counter in a full column doesn't work and ignores the click from user - suggesting they click a valid column.
8. Click the end-screen of a won/draw game to return to main menu
9. Create sample games of both personas winning with 4 same coloured counters: (all updating the leaderboard correctly)
 - a. In a vertical pattern
 - b. In horizontal pattern
 - c. In a diagonally left pattern
 - d. In a diagonally right pattern
10. Play-test mock game of both personas drawing the game - and the leaderboard updating correctly
11. Exit graphical display before game end-screen to cancel game and exit back to menu
12. Closing window after a game-endscreen verdict has been reached rather than clicking screen

In testing all these steps, we will also be ensuring the relevant SQL statements in the program have worked correctly. If they haven't - we will see errors in the following testing.

User Input	Program Output	Outcome
------------	----------------	---------

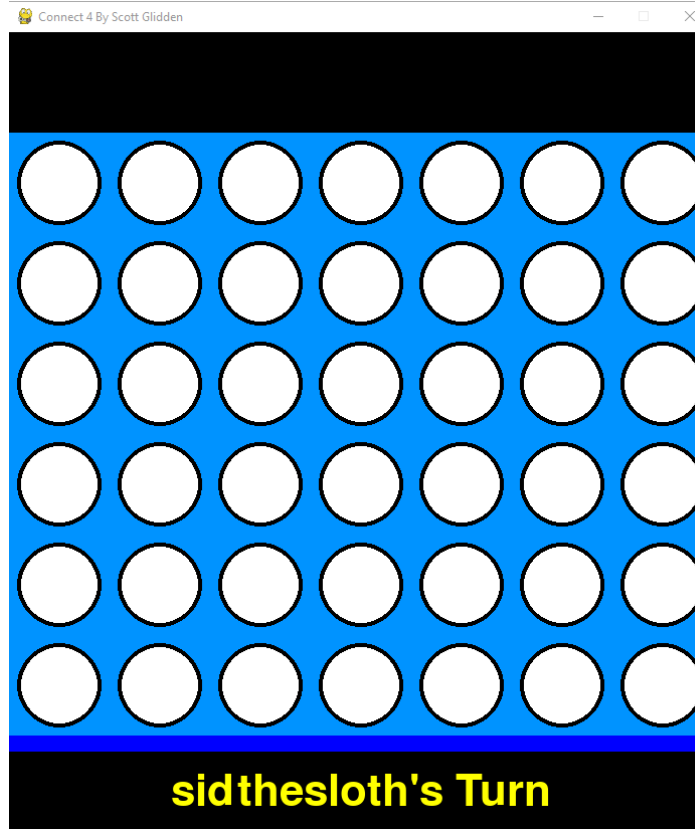
Start game with 0 players logged in:  Player 1: NOT LOGGED IN Player 2: NOT LOGGED IN [1] Log-in Player 1 [2] Log-in Player 2 [3] Sign-up [4] Play Game! [0/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 4 Error: There are not 2 unique players logged in. Press enter to retry.	Pass. The game did not start as there were not 2 unique players logged in. Works as intended.
Start game with only Persona 1 logged in:  Player 1: marktheshark Player 2: NOT LOGGED IN [1] Log-out 'marktheshark' [2] Log-in Player 2 [3] Sign-up [4] Play Game! [1/2 players logged in] [5] View Leaderboard Enter number from above and press enter to select. Leave blank to exit menu. > 4 Error: There are not 2 unique players logged in. Press enter to retry.	Pass. The game did not start as there were not 2 unique players logged in. Works as intended.

Start game with Persona 1 and Persona 2 logged in:  Player 1: marktheshark Player 2: sidthesloth <pre>[1] Log-out 'marktheshark' [2] Log-out 'sidthesloth' [3] Sign-up [4] Play Game! [2/2 players logged in] [5] View Leaderboard</pre> Enter number from above and press enter to select. Leave blank to exit menu. > 4	Message printed in console:  Player 1: marktheshark Player 2: sidthesloth <pre>[1] Log-out 'marktheshark' [2] Log-out 'sidthesloth' [3] Sign-up [4] Play Game! [2/2 players logged in] [5] View Leaderboard</pre> Enter number from above and press enter to select. Leave blank to exit menu. > 4 Connect 4 window has opened! Select the window and left click the column you'd like to place your counter in. Good luck! ■ Graphical display opened (with marktheshark randomly selected to start):	Pass. The console good luck message and graphical display opened only when 2 players are logged in. In this case marktheshark was randomly picked to start. Therefore: Works. As. Intended!
---	---	--



Like the above, starting the game with 2 players logged in.

Evidence of sidthesloth (persona 2) being randomly picked to start first.

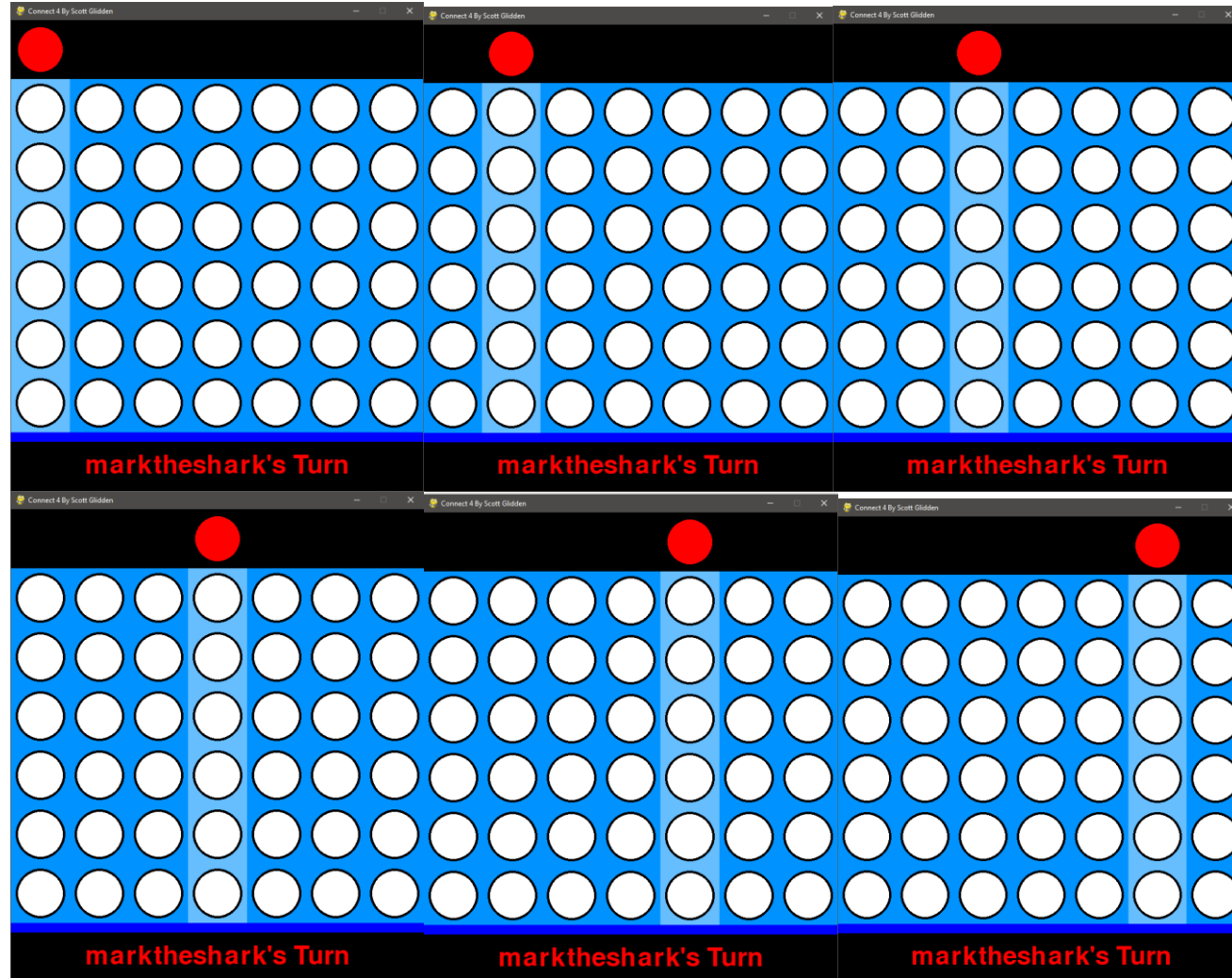


Pass.

The game board can also randomly select persona 2 to start - so randomisation works.

Works as intended.

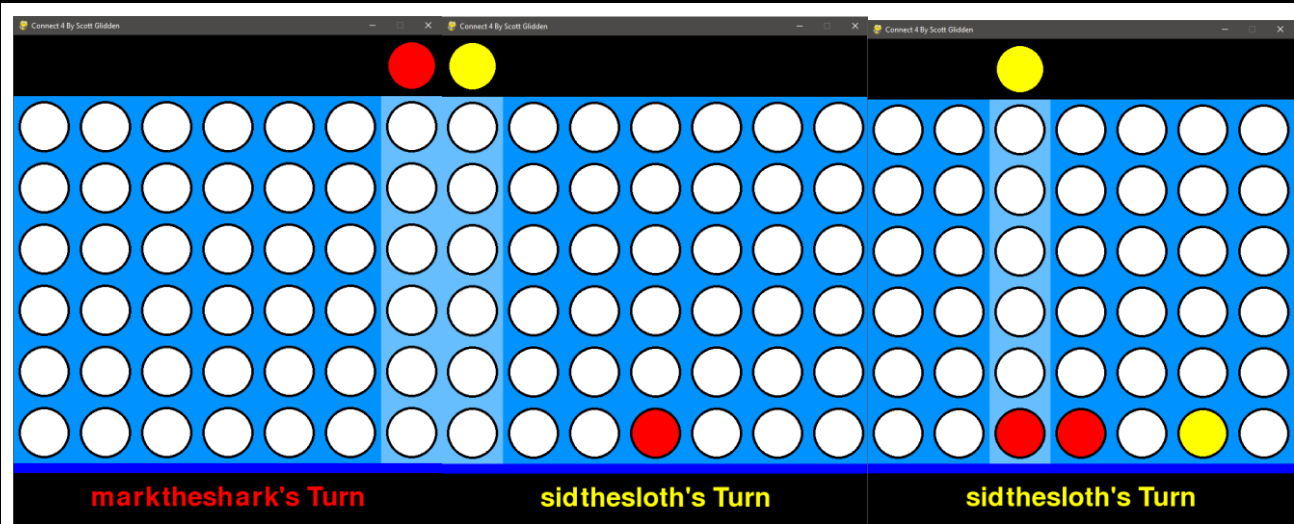
Hovering over each column with mouse (Note: I couldn't get any software that would screen AND show the mouse, so for clarification when a column is highlighted my cursor is in the menu hovering anywhere in the column INCLUDING the black top and bottom bars. I've also thrown in a couple of examples of the columns being hovered over by the other player demonstrating the counter colour changes too):



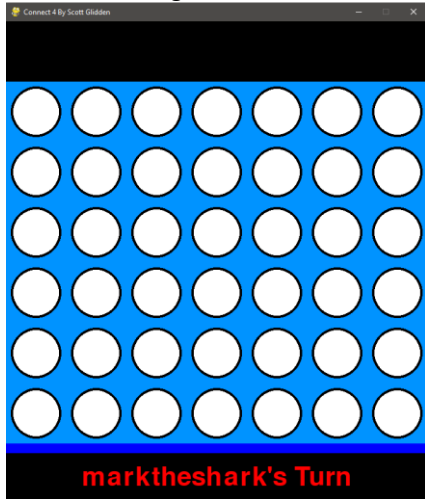
Pass.

All 7 columns successfully light up when hovered over with the correct colour counter floating on top.

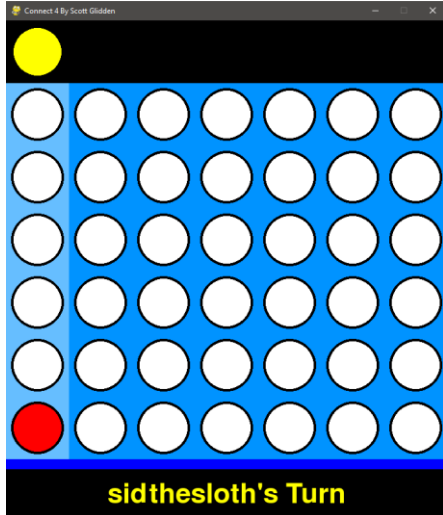
Works as intended.



Here is a mock game to demonstrate the counters fall where asked to. Counter is dropped in column by left clicking. Output:



marktheshark starts and drops in column 1 by left clicking column. Output:



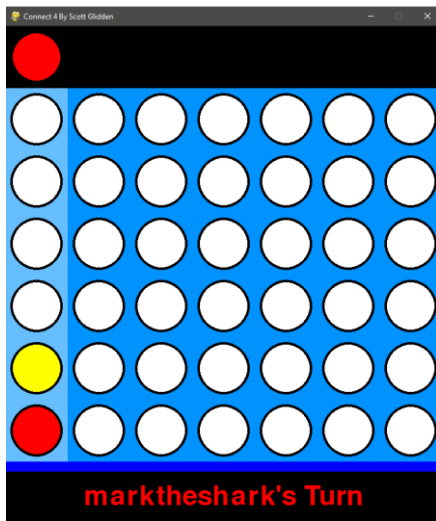
sidthesloth drops in column 1 also by left clicking column. Output:

Pass.

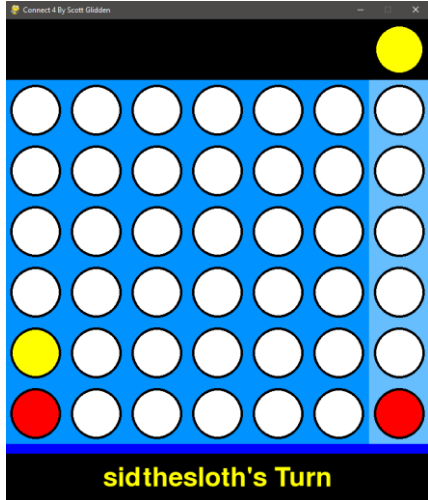
The counters fall to the bottom of the clicked column as expected, including landing on top of other counters. The turns swap after each counter is dropped.

(Note: for the majority of the images the mouse isn't moved after a counter has been dropped in the column hence why the column stays highlighted after the counter has been dropped there. The 5th image shows that the highlighted column does indeed adjust to where your mouse is even after dropping counter, and does not highlight the column which has just had a counter dropped. This is a co-incidence based on where the mouse was sitting when screenshotting)

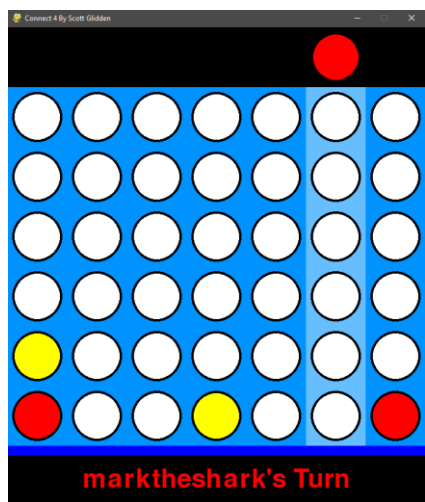
Works as intended.



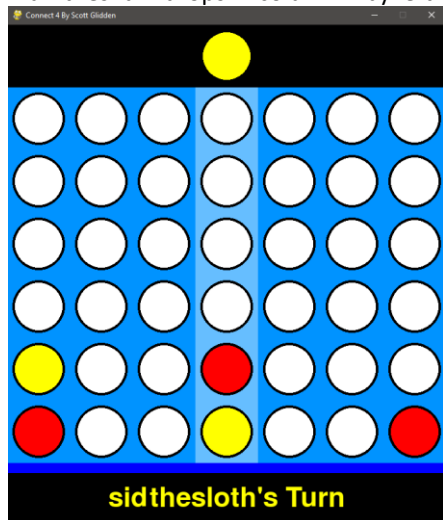
marktheshark drops in column 7 by left clicking column. Output:



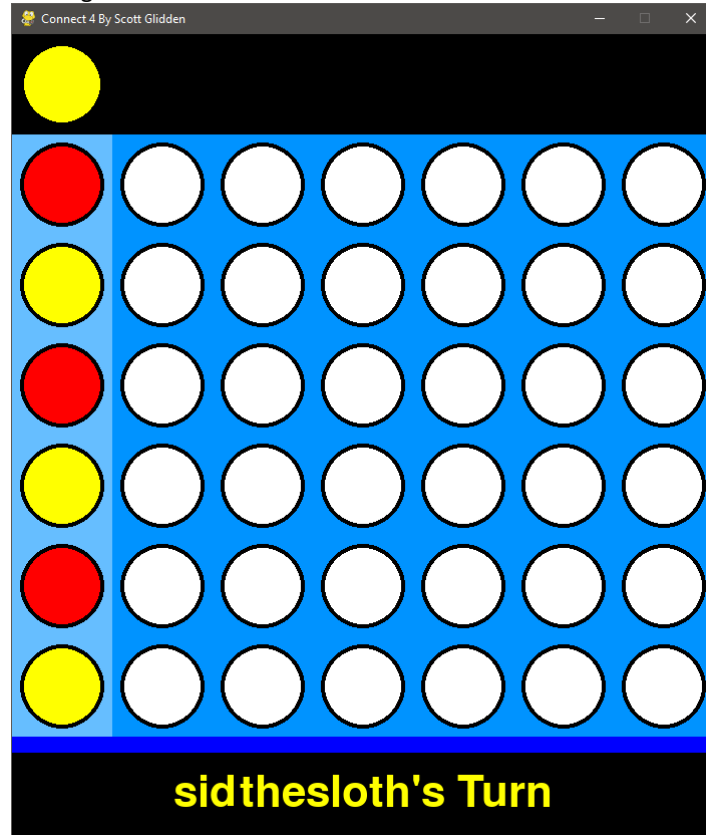
sidthesloth drops in column 4 by left clicking column: Output:



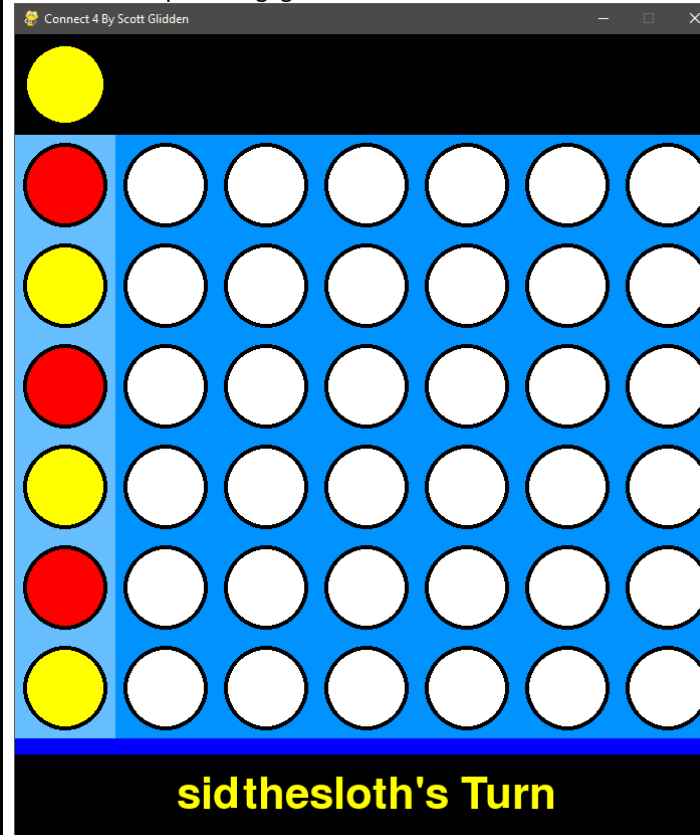
marktheshark drops in column 4 by left clicking column. Output:



Clicking the first column when full:



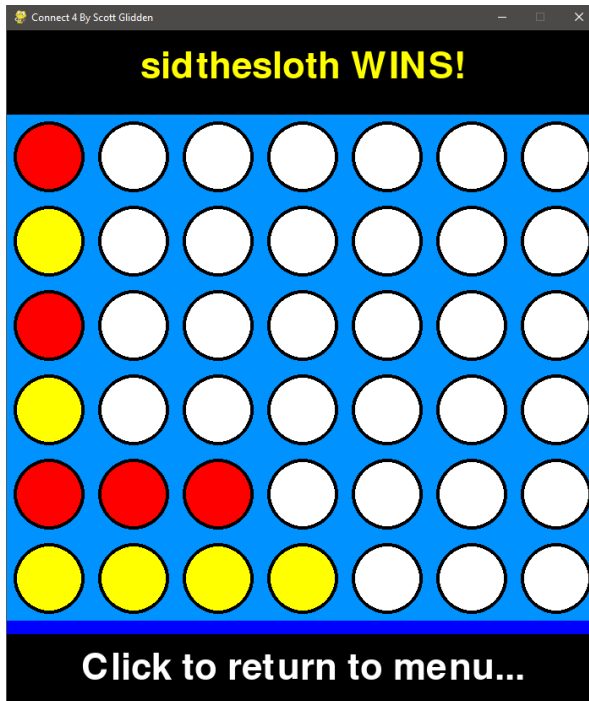
Evidence of input being ignored:



Pass.

Clicking to drop a counter in a full column doesn't do anything and ignores the input.

Works as intended.



Clicking the end-screen of a game (above screen)

Exits back to menu:

Main Menu

Player 1: marktheshark

Player 2: sidthesloth

- [1] Log-out 'marktheshark'
- [2] Log-out 'sidthesloth'
- [3] Sign-up
- [4] Play Game! [2/2 players logged in]
- [5] View Leaderboard

Enter number from above and press enter to select.
Leave blank to exit menu.

> _

Pass.

When the game board is clicked after the game is over, you are taken back to the menu successfully.

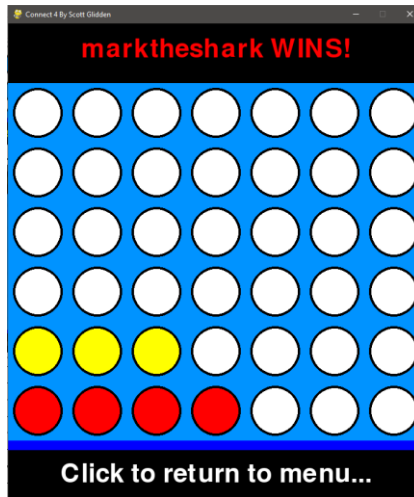
Works as intended.

Leaderboard before game & horizontal 4 in a row for Persona 1:

```
- Leaderboard Menu -
[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
> 1
```

```
Showing leaderboard sorted by wins:
Username Wins Losses Draws
clifford 15 16 4
bobthe 10 4 2
sglidden 5 6 1
coolusername 1 0 0
fraser 1 6 1
marktheshark 1 1 0
sidthesloth 1 1 0
Vamperina 1 0 0
123456 0 0 0
123456789012345678901234 0 0 0
246810 0 0 0
orangeeater 0 1 0
sglidden2 0 0 0
teigs2408 0 0 0
test me 0 0 0
```



Leaderboard after game:

```
- Leaderboard Menu -
[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
> 1
```

```
Showing leaderboard sorted by wins:
Username Wins Losses Draws
clifford 15 16 4
bobthe 10 4 2
sglidden 5 6 1
marktheshark 2 1 0
coolusername 1 0 0
fraser 1 6 1
sidthesloth 1 2 0
Vamperina 1 0 0
123456 0 0 0
123456789012345678901234 0 0 0
246810 0 0 0
orangeeater 0 1 0
sglidden2 0 0 0
teigs2408 0 0 0
test me 0 0 0
```

(press enter to return to leaderboard menu)_

Pass.

Horizontal 4 in a row detected successfully.
Wins for marktheshark (persona 1) has incremented by 1.
Losses for sidthesloth (persona 2) has incremented by 1

Works as intended.

Leaderboard before game & vertical 4 in a row for Persona 1:

```
- Leaderboard Menu -
[1] Sort by wins
[2] Sort by losses
[3] Sort by draws

Enter number from above and press enter to select.
Leave blank to exit menu.
> 1
```

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
marktheshark	2	1	0
coolusername	1	0	0
fraser	1	6	1
sidthesloth	1	2	0
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

(press enter to return to leaderboard menu)_



Leaderboard after game:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
marktheshark	3	1	0
coolusername	1	0	0
fraser	1	6	1
sidthesloth	1	3	0
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

Pass.

Vertical 4 in a row detected successfully.

Wins for marktheshark (persona 1) has incremented by 1.

Losses for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Leaderboard before game & diagonal left 4 in a row for Persona 1:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
marktheshark	3	1	0
coolusername	1	0	0
fraser	1	6	1
sidthesloth	1	3	0
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0



Leaderboard after game:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
marktheshark	4	1	0
coolusername	1	0	0
fraser	1	6	1
sidthesloth	1	4	0
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

Pass.

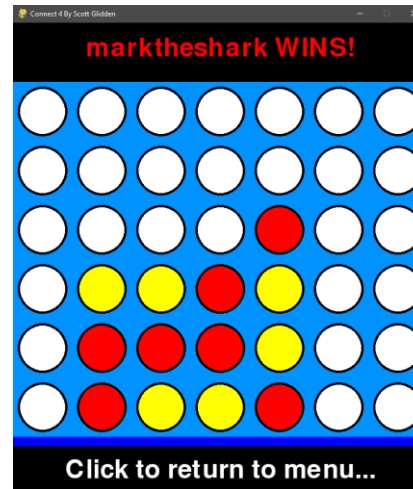
Diagonal left 4 in a row detected successfully.
Wins for marktheshark (persona 1) has incremented by 1.
Losses for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Leaderboard before game & diagonal right 4 in a row for Persona 1:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
sglidden	5	6	1
marktheshark	4	1	0
coolusername	1	0	0
fraser	1	6	1
sidthesloth	1	4	0
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0



Leaderboard after game:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	1	0
sglidden	5	6	1
coolusername	1	0	0
fraser	1	6	1
sidthesloth	1	5	0
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

Pass.

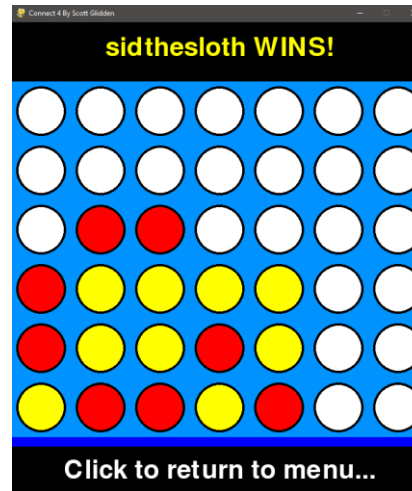
Diagonal right 4 in a row detected successfully.
Wins for marktheshark (persona 1) has incremented by 1.
Losses for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Leaderboard before game & horizontal 4 in a row for Persona 2:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	1	0
sglidden	5	6	1
coolusername	1	0	0
fraser	1	6	1
sidthesloth	1	5	0
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0



Leaderboard after game:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	2	0
sglidden	5	6	1
sidthesloth	2	5	0
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

Pass.

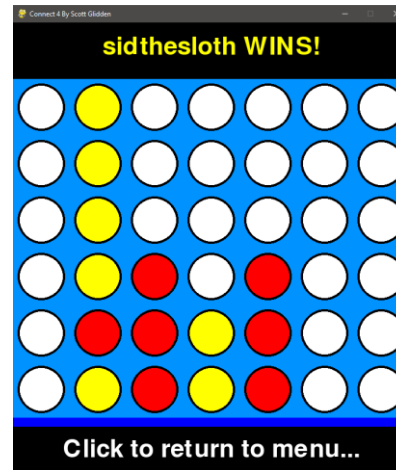
Horizontal 4 in a row detected successfully.
Losses for marktheshark (persona 1) has incremented by 1.
Wins for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Leaderboard before game & vertical 4 in a row for Persona 2:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	2	0
sglidden	5	6	1
sidthesloth	2	5	0
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0



Leaderboard after game:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	3	0
sglidden	5	6	1
sidthesloth	3	5	0
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

Pass.

Vertical 4 in a row detected successfully.

Losses for marktheshark (persona 1) has incremented by 1.

Wins for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Leaderboard before game & diagonal left 4 in a row for Persona 2:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	3	0
sglidden	5	6	1
sidthesloth	3	5	0
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0



Leaderboard after game:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	4	0
sglidden	5	6	1
sidthesloth	4	5	0
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

Pass.

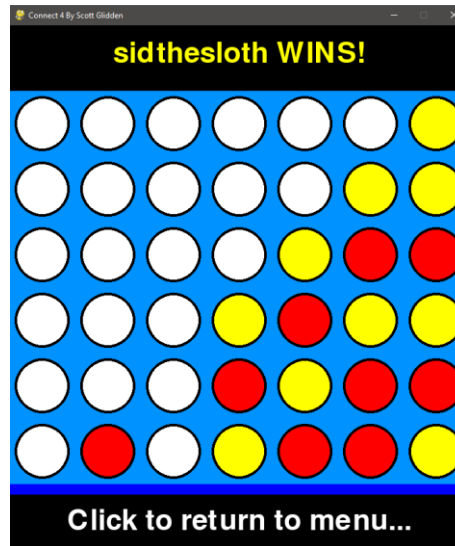
Diagonal left 4 in a row detected successfully.
Losses for marktheshark (persona 1) has incremented by 1.
Wins for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Leaderboard before game & diagonal right 4 in a row for Persona 2:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	4	0
sglidden	5	6	1
sidthesloth	4	5	0
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0



Pass.

Diagonal right 4 in a row detected successfully.
Losses for marktheshark (persona 1) has incremented by 1.
Wins for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Leaderboard after game:

Showing leaderboard sorted by wins:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
marktheshark	5	5	0
sglidden	5	6	1
sidthesloth	5	5	0
coolusername	1	0	0
fraser	1	6	1
Vamperina	1	0	0
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0

Leaderboard before game and draw end game screen:

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
fraser	1	6	1
sglidden	5	6	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
marktheshark	5	6	0
orangeeater	0	1	0
sglidden2	0	0	0
sidthesloth	6	5	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

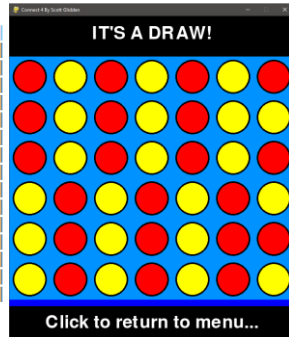
(press enter to return to leaderboard menu)

Leaderboard after game:

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
fraser	1	6	1
marktheshark	5	6	1
sglidden	5	6	1
sidthesloth	6	5	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu) _



Pass.

Draw detected successfully and both marktheshark (persona 1) and sidthesloth (persona 2) have had their draw counter incremented in the database.

Works as intended.

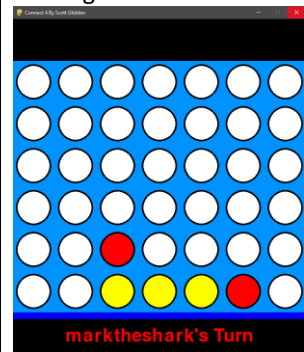
Leaderboard before exiting connect 4 menu before game verdict:

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
fraser	1	6	1
marktheshark	5	6	1
sglidden	5	6	1
sidthesloth	6	5	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu)

Clicking the close window button before the game verdict:



Leaderboard and main menu after this:

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
fraser	1	6	1
marktheshark	5	6	1
sglidden	5	6	1
sidthesloth	6	5	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu)

Main Menu

Player 1: marktheshark
 Player 2: sidthesloth
 [1] Log-out 'marktheshark'
 [2] Log-out 'sidthesloth'
 [3] Sign-up
 [4] Play Game! [2/2 players logged in]
 [5] View Leaderboard
 Enter number from above and press enter to select.
 Leave blank to exit menu.
 >

As expected, when the game was exited before the program decided a verdict for the game, the leaderboard remains untouched and you are taken right back to the main menu.

Works as intended.

Leaderboard before closing connect 4 window after game verdict rather than clicking:

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
fraser	1	6	1
marktheshark	5	6	1
sglidden	5	6	1
sidthesloth	6	5	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test me	0	0	0
Vamperina	1	0	0

(press enter to return to leaderboard menu).

Mock game closed by clicking the X close button in the top right rather than clicking the screen:



Leaderboard after clicking X to close window:

As expected, whether or not you click the screen to close the connect 4 game window or **use the X red button** the leaderboard has still correctly updated.

Wins for marktheshark (persona 1) has incremented by 1.
Losses for sidthesloth (persona 2) has incremented by 1.

Works as intended.

Showing leaderboard sorted by draws:

Username	Wins	Losses	Draws
clifford	15	16	4
bobthe	10	4	2
fraser	1	6	1
marktheshark	6	6	1
sglidden	5	6	1
sidthesloth	6	6	1
123456	0	0	0
123456789012345678901234	0	0	0
246810	0	0	0
coolusername	1	0	0
orangeeater	0	1	0
sglidden2	0	0	0
teigs2408	0	0	0
test_me	0	0	0
Vamperina	1	0	0

End User Requirement Outcomes From Above Testing:

- The above system uses a graphical user interface with left clicking to drop a counter in the board therefore the end-user requirement “Users must drop a counter by clicking the wanted column – not typing in the number corresponding to the column” is fulfilled successfully
- As the above system proves that you exit back to the main menu when a game ends, the end-user requirement “Users must be able to continue playing multiple games without needing to restart the program” is successfully fulfilled.

Testing Summary: The above evidence shows that the program successfully meets all the stated requirements set out.

PROJECT EVALUATION

Fitness For Purpose Evaluation

The complete project, as evidenced from the results of testing, meets all original functional and end-user requirements. Users can sign-up, login, view and sort the leaderboard, and play fully functional games of Connect 4 on a graphical interface. All inputs have been validated along with the advanced higher concepts (2D array, array of records, & integration with database) are all met successfully. Using the created personas and test cases all tasks/functions of the program were able to be carried out successfully without any errors. All test cases in my test plan have been carried out, evidenced, and results discussed. I am certain my solution meets all requirements from my requirement specification.

I have successfully finished my project by the end of week 24 (04/04/2022) as intended by my Gantt Chart in time for the deadline. The project flawlessly follows the design which was created, resulting in a final product that is exactly as intended. All the scope, boundaries, and constraints have been delivered/achieved exactly as were set out during the Analysis phase of my project.

For clarification, I have copied my end-user and functional requirements below for reference. I have gone through each requirement and scored out each requirement that has been met to ensure none have been missed during testing.

Main Menu		
Python Requirements		
Inputs	Processes	Outputs
Validated user menu navigation inputs	Validation of user inputs Open connection with database (on program start)	Text-based menus & submenus displayed based on input: — Main Menu — Sign-up Menu — Log-in Menu — Leaderboard Menu Display who is logged in on main-menu Number of users logged in on main-menu Validation error messages Quitting game
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	X	X
End User Requirements		

Users must be able to easily navigate the menus

Log-in Submenu		
Python Requirements		
Inputs	Processes	Outputs
Validated inputted usernames and passwords which already exist in database Existing user details SQL results	Validation of user inputs (via checking entered details exist in database)	Returning to main menu Log in success message Validation error messages
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	X	Existing user details SQL results
End User Requirements		
Users must be able to easily navigate the menus Users needs their accounts to be password protected		

Sign-up Submenu		
Python Requirements		
Inputs	Processes	Outputs
Validated inputted usernames (6-16 characters) Validated inputted passwords (6-16 characters) Existing user details SQL results	Validation of user inputs (6-16 characters and ensuring user doesn't already exist using SQL results)	Returning to main menu Sign up success message Validation error messages
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
Inserting new account details (username and password)	X	Existing user details SQL results

End User Requirements
Users must be able to easily navigate the menus Users needs their accounts to be password protected

Connect 4 Board		
Python Requirements		
Inputs	Processes	Outputs
Left clicking window	Check 2 players are logged in before game starts Validation of user inputs (left clicking) on the Connect 4 Board including: —— Dropping counter where valid —— Exiting to menu after game ended screen Automatically detect wins and draws on Connect 4 board Randomise which players drops first counter	Successful starting game message Launching graphically displayed Connect 4 board Update following on the board window: —— Highlighting columns when hovered —— Displaying dropped counters dropped by player —— Post game win/draw screen Exiting back to main menu
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	Update player statistics based on how the game ends (editing wins, losses, and draws for both logged in players)	X
End User Requirements		
Users must drop a counter by clicking the wanted column — not typing in the number corresponding to the column Users must be able to continue playing multiple games without needing to restart the program		

Leaderboard Submenu

Python Requirements		
Inputs	Processes	Outputs
Validated user menu navigation inputs Leaderboard query results	Validation of user inputs Sort "win" sql results	Sorted leaderboard table based on input Return to main menu Validation error messages
Database Requirements		
Inputs (Forms)	Processes (Queries)	Outputs (Reports)
X	Sort "losses" and "draws" sql results	Leaderboard query results
End User Requirements		
Users must be able to easily navigate the menus Users must be able to choose between the leaderboard being shown sorted by wins, losses, or draws		

As evidenced above, all requirements have successfully been met. This project has met all criteria set out, so is definitely suitable for purpose.

Maintainability Evaluation

It may be useful to refer to the implemented program code to get a wider idea of the program, however I will provide examples in code examples below. I have ensured my solution is maintainable by doing the following:

Commentary & Whitespace

Every function/procedure in my program has a docstring at the top describing the purpose of the subroutine. I have purposefully spaced out subsections of code with lines of whitespace inside of functions as well as having commentary describing each line of code so a reasonably competent programmer can more easily understand what the code is doing and how it works. Some examples of this on next page (however this style of commentary and whitespace is continued throughout the whole program)

```

def leaderboard_menu():
    '''Display leaderboard menu'''
    result = sql_select(f"SELECT usernames FROM user_details") #run SQL
    if result == None: #if no result in database
        input(RED_TEXT + "There is no leaderboard to show as there are no signed up users yet! Sign-up in the main menu first. \n(Press enter to return to main menu)" + RESET_TEXT) #send error
        return #exit menu
    while result != None: #if there are results run leaderboard menu loop
        cls() #clear screen

        ##Print options
        print(BLUE_TEXT + "\n- Leaderboard Menu -" + RESET_TEXT)
        print("[1] Sort by wins")
        print("[2] Sort by losses")
        print("[3] Sort by draws")

        user_input = get_ranged_number_input(1, 3) #get valid input

        if user_input == None: #exit menu if empty input
            print("\nReturning to main menu...\n")
            break
        elif user_input >= 1 and user_input <= 3: #run appropriate query based on input

            if user_input == 1: #sort by wins
                result = sql_select(f"SELECT usernames, wins, losses, draws FROM user_details")
                #sorting by wins uses a python bubble sort instead of relying on SQL's ORDER BY
                sorted_result = bubble_sort(result) #sort by wins via bubble sort
                sort_by = "wins" #displayed sorter
            elif user_input == 2:
                sorted_result = sql_select(f"SELECT usernames, wins, losses, draws FROM user_details ORDER BY losses DESC") #get SQL result sorted
            by losses
                sort_by = "losses" #displayed sorter
            elif user_input == 3:
                sorted_result = sql_select(f"SELECT usernames, wins, losses, draws FROM user_details ORDER BY draws DESC") #get SQL result sorted
            by draws
                sort_by = "draws" #displayed sorter

            if sorted_result != None: #ensure there is a result to be printed
                table_headings = ["Username", "Wins", "Losses", "Draws"] #setup heading names

                ##Create list from table headings of length of heading
                table_width = []
                for item in table_headings:
                    table_width.append(len(item))

                ##Get largest value from each column fetched from database & add to list
                for row in sorted_result:
                    for index, item in enumerate(row):
                        if len(str(item)) > table_width[index]: #if length greater
                            table_width[index] = len(str(item)) #update

                ##Print table headings with correct length
                print(f"\nShowing leaderboard sorted by {sort_by}:")
                display = BLUE_TEXT #set display to blue colour
                for index, item in enumerate(table_headings): #for each heading
                    display += "|" + str(item).ljust(table_width[index], " ") + " " #add next column to string
                display += "\033[0;0m" #reset text colour and close table
                print(display) #print line

                for row in sorted_result: #for each row in table
                    display = "" #reset display string
                    for index, item in enumerate(row): #for each item in row
                        display += "|" + str(item).ljust(table_width[index], " ") + " " #add next column to string
                    display += "|" #close table
                    print(display) #print row

                input("\n(Press enter to return to leaderboard menu)") #wait for input before returning to main leaderboard menu

            else:
                #error when nobody exists on leaderboard yet
                print(RED_TEXT + "\n\nNobody is listed on the leaderboard yet! Use the 'sign-up' option on the main menu to create some accounts and view the leaderboard. (Press enter to return to main menu)" + RESET_TEXT)

```

```

def get_ranged_number_input(min, max):
    '''Function returns valid ranged number'''
    while True: #repeat until valid number is received
        print("\nEnter number from above and press enter to select.\nLeave blank to exit menu.")
        user_input = input("> ") #get input
        fail_message = f"Input must be a number between {min} and {max}, or blank to exit menu. Press enter to retry."

        if user_input == "": #check for blank input first
            return #returns blank input
        try:
            if min <= int(user_input) and int(user_input) <= max: #checks input is a number in range
                return int(user_input) #return the validated value
            else: #if not in range, retry
                input(RED_TEXT + fail_message + RESET_TEXT)
        except: #if error has occurred above, retry
            input(RED_TEXT + fail_message + RESET_TEXT)

```

Modularity

In Atom, it allows you to collapse subroutines. Here is a screenshot of the modules collapsed so you can more easily see the module names:

```
1  '''Libraries'''
2  import os
3  import pygame
4  import random
5  import mysql.connector
6
7  class Player:
8      '''This initialises the player class'''
9      def __init__(self, username=None, password=None, logged_in=False):
10         self.username = username #str/None
11         self.password = password #str/None
12         self.logged_in = logged_in #bool
13
14
15  '''Menu Things'''
16 > def cls():=
19
20 > def count_players():=
28
29 > def get_ranged_number_input(min, max):=
45
46 > def bubble_sort(result):=
58
59 > def leaderboard_menu():=
127
128 > def login_menu(player_id):=
186
187 > def signup_menu():=
230
231 > def sql_commit(sql):=
235
236 > def sql_select(sql):=
240
241 > def check_length_input(min, max, user_input):=
247
248 > def connect_db():=
287
288 > def menu():=
343  '''Conect 4 Board'''
344 > def draw_window(board, turn_id, game_font, hover_column, win_state):=
411
412 > def play_game():=
475
476 > def drop_counter(board, selected_column, drop_value):=
483
484 > def check_win(board, ref, turns):=
```

```

524 ##PYGAME CONSTANTS
525 SQUARE_DIMENSIONS = 100
526 CIRCLE_RADIUS = (SQUARE_DIMENSIONS/2) - 8
527 COLUMNS = 7
528 ROWS = 6
529 WIDTH = SQUARE_DIMENSIONS * 7
530 HEIGHT = SQUARE_DIMENSIONS * 8 #8 not 6 for the space at top and bottom
531
532 ##PYGAME COLOUR CONSTANT TUPLES
533 WHITE = (255, 255, 255)
534 BLUE = (0, 147, 255)
535 DARK_BLUE = (0, 0, 255)
536 YELLOW = (255, 255, 0)
537 RED = (255, 0, 0)
538 BLACK = (0, 0, 0)
539 SELECT_COLOUR = (102, 190, 255)
540 FPS = 24
541
542 ##TEXT COLOUR CONSTANTS
543 RED_TEXT = "\u001b[0;31;40m"
544 BLUE_TEXT = "\u001b[36m"
545 GREEN_TEXT = "\u001b[32m"
546 RESET_TEXT = "\033[0;0m"
547
548 ##ASCII ART
549 ASCII_MAIN_MENU = '''
550 |  \  |      ( )      |  \  |
551 |  \ /  |  _ _ _ _ _  |  \ /  |  _ _ _ _ _
552 |  | \  | / _ | | | |  \  |  | \  | / _ | | | |  \  |
553 |  |  | | ( _ | | | |  |  |  | |  _ / | | | |  |  |
554 |  |  | \ _ , _ | | | |  |  |  | \ _ | | | |  |  | \ _ ,
555 '''
556 # https://patorjk.com/software/taag/#p=display&f=Biq&t=Sign%20-%20up%20%20%20Menu
557
558 #setup array of records
559 PLAYERS = [Player()] * 2
560
561 '''Main Program'''
562 connect_db()
563 menu()

```

As evidenced above, this program has been programmed modularly - with each of the modules having meaningful variable names. I have also split the modules by menu and the code relevant to the graphical display/playing game. In programming modularly it allows me to make changes to modules and have them come into effect across the whole program with any code that references it. Modularity in itself also breaks down the program into more manageable chunks, more readable by a programmer. It also allows code to be reused, which keeps code less cluttered, and more maintainable as a whole.

The `cls()` function is a good example of this which adds a level of convenience and clarity for the programmer. Rather than writing this code every time I want to clear the screen:

```
18 os.system('cls' if os.name == 'nt' else 'clear')
```

By putting this code within a procedure, I can simply call the `cls()` function rather than typing out the full line of code each time. Much more convenient and less repeated code. It also means if a new python version breaks the clear screen function, it only needs to be replaced in one place by the programmer.

By programming this whole project modularly, this aids in the maintainability of the project for the programmer for reasons described above.

Variable Names

Throughout the program I have used meaningful variable names (and module names) to help keep code clear. I have also stuck to 2 standard formats for variables to keep them consistent.

Any global constant variables were written all block capital (form of: "VARIABLE")

Any other variable has been written all lower case with underscores separating words (form of: "variable_name")

Here is a snippet of code demonstrating the above (text colours are constants, the rest are local variables):

```
def signup_menu():
    '''Display signup menu'''
    ##Get list of usernames from database
    result = sql_select("SELECT usernames FROM user_details") #get list of usernames in database
    valid_usernames = [] #blank list
    if result != None: #if results found
        for i in range(len(result)): #for each username...
            valid_usernames.append(result[i][0]) #append to list for comparing later

    cls() #clear screen
    print(BLUE_TEXT + "\n- Sign-up Menu -" + RESET_TEXT + "\n(leave blank and press enter to return to main menu)\n")
    #signup instructions

    in_progress = True #default to true

    '''Get Valid Username'''
    while in_progress: #unless told to exit back to menu...
        username = input("Enter Username (6-16 characters) > ") #get username
        if username == "": #exit to main menu if blank input
            in_progress = False
            break
        elif username in valid_usernames: #if username exists give error
            input(RED_TEXT + "Invalid! This username is already in use. Please log in or choose a new username. Press enter to retry." + RESET_TEXT)
        elif check_length_input(6, 16, username) == False: #if username not within length range, give error
            input(RED_TEXT + "Invalid! Username must be between 6-16 characters. Press enter to try again." + RESET_TEXT)
        else: #if valid
            break #leave while loop

    '''Get Valid Password'''
    while in_progress: #unless told to exit back to menu
        password = input("Enter Password (6-16 characters) > ") #get password
        if password == "": #exit to main menu if blank input
            in_progress = False
            break
        elif check_length_input(6, 16, password) == False: #if not within length range, give error
            input(RED_TEXT + "Invalid! Password must be between 6-16 characters. Press enter to try again." + RESET_TEXT)
        else: #if valid
            break #leave while loop

    if in_progress: #if username and password value
        sql_commit(f"INSERT INTO user_details (usernames, passwords, wins, losses, draws) VALUES ('{username}', '{password}', 0, 0, 0)") #add user to database
        input(GREEN_TEXT + "Sign-up successful. Press enter to return to main menu." + RESET_TEXT) #success message
    else: #cancel message if blank input
        input(RED_TEXT + "Sign-up cancelled. Press enter to return to main menu." + RESET_TEXT)
```

By using a consistent format for the variables as well as meaningful names, the code is kept clearer for any programmer looking at my code, aiding in maintainability.

Robustness Evaluation

As can be seen from my testing section of the project, I have been unable to break any input validation or cause any errors in my program. So due to the fact I have been unable to find any bugs or errors in my program from my testing, I am confident in saying my program is as robust as it can be. As an example - here is the code that is reused throughout the menus to validate users number inputs:

```
def get_ranged_number_input(min, max):  
    '''Function returns valid ranged number'''  
    while True: #repeat until valid number is received  
        print("\nEnter number from above and press enter to select.\nLeave blank to exit menu.")  
        user_input = input("> ") #get input  
        fail_message = f"Input must be a number between {min} and {max}, or blank to exit menu. Press enter to retry."  
  
        if user_input == "": #check for blank input first  
            return #returns blank input  
        try:  
            if min <= int(user_input) and int(user_input) <= max: #checks input is a number in range  
                return int(user_input) #return the validated value  
            else: #if not in range, retry  
                input(RED_TEXT + fail_message + RESET_TEXT)  
        except: #if error has occurred above, retry  
            input(RED_TEXT + fail_message + RESET_TEXT)
```

It's validation such as this that helps ensure the program doesn't break - and keeps the program robust.